

BLM5126

İleri Yazılım Mimarisi

2025-2026 GÜZ YARIYILI

DR.ÖĞR.ÜYESİ GÖKSEL BİRİCİK

Ön Bilgi

Pazartesi 09.00-11.50

D-106

gbiricik@yildiz.edu.tr

Ön Koşul:

- Yazılım mühendisliği temelleri ve/veya sektörde yazılım geliştirme deneyimi
- En azından bir dilde program kodlayabilme

Kapsam:

- Detaylı tasarım yerine yüksek seviyeli mimari tasarıma,
- Nesneye yönelik tasarıma odaklanacağız.

Ders Hedefleri

Dönemi tamamladığımızda;

1. Bir uygulamanın analiz ve tasarımını UML ile ifade edebilmeyi,
 2. Uygulamanın işlevsel anlam tanımlarını OCL ile yapabilmeyi,
 3. Yazılım mimarilerini tanımlayıp değerlendirebilmeyi,
 4. Uygun mimari biçimi seçip kullanabilmeyi,
 5. Nesneye yönelik tasarım tekniklerini anlayıp uygulayabilmeyi,
 6. Uygun yazılım tasarım kalıplarını seçip kullanabilmeyi,
 7. Tasarım gözden geçirimini anlayıp katılabilmeyi
 8. Modern yazılım mimarilerini ve problemleri çözüm yöntemlerini
- Öğreneceğimizi hedefliyoruz.

Ders Akışı

1.Bölüm

- Giriş: Önce birkaç giriş dersi yapacağız

2.Bölüm

- UML ve Analiz: Bir tasarım problemi nasıl analiz edilir, UML ve OCL ile nasıl tanımlanır, analizin statik ve dinamik yönleri nelerdir öğreneceğiz.

3.Bölüm

- Yazılım Mimarisi: Mimariler hakkında düşünmeyi, onları ifade etmeyi, işlevsel olmayan gereksinimlerin mimari belirlenmesine olan etkilerini, mimari tasarımların gerçekleştirilecek modüller haline nasıl getirileceğini göreceğiz.

4.Bölüm

- Yazılım Tasarımı: Tasarımla ilgili nesne tasarımı, tasarım prensipleri, tasarım gözden geçirimi gibi özel detaylara bakacağız.

5.Bölüm

- Modern yazılım mimarileri ile yazılım çözümlerinin nasıl geliştirildiğini inceleyeceğiz.

Ders Planı

Hafta	Tarih	Konu
1	29.09.2025	Giriş, Ders planı, Genel bilgiler, İlk örnek tasarım
2	06.10.2025	Tasarım kavramları, UML, Nesneye yönelik analiz, UML sınıf modelleri
3	13.10.2025	Tasarım çalışmaları, formal tanımlama
4	20.10.2025	OCL
5	27.10.2025	Davranış modelleme
6	03.11.2025	Yazılım mimarisi, Mimari görünüm
7	10.11.2025	Mimari biçimler, İşlevsel olmayan gereksinimler

Ders Planı

Hafta	Tarih	Konu
8	17.11.2025	Ara Sınav
9	24.11.2025	Öğrenci Sunumları
10	01.12.2025	Öğrenci Sunumları
11	08.12.2025	Öğrenci Sunumları
12	15.12.2025	Öğrenci Sunumları
13	22.12.2025	Öğrenci Sunumları
14	29.01.2025	Öğrenci Sunumları
15	05.01.2025	Öğrenci Sunumları

Değerlendirme Kriterleri

1 ara sınav (%30)

Sunum (%30)

- Modern yazılım mimarileri, sorunlar, çözüm yaklaşımları ve mimari tasarım kalıpları ile ilgili seçilen konularda 10-15 dakikalık bir sunum yapacaksınız.
- Sunumun akademik raporunu da yazacaksınız.

Final projesi (%40)

Dönem içi %60, dönem sonu projesi %40 etkileyecek.

Kaynak Kitaplar



Hızlı Bir Başlangıç

Tasarım, öğretmekten çok öğrenilir.

Basit bir metin görüntüleyici tasarlayalım.

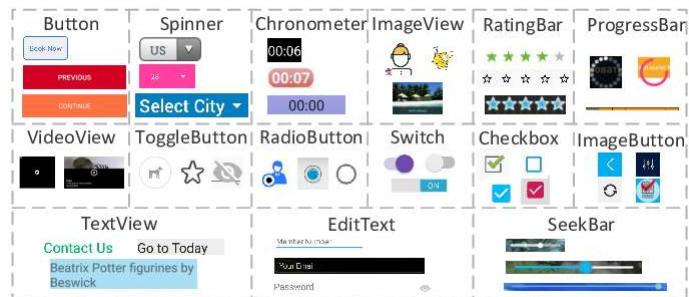
- Problem: Bir dosyadaki metin içeriğini ekranda göstermek.
- Kısıt: Kullanıcı grafik arayüzü araçlarında (GUI toolkit) bunu tek başına yapabilen bir bileşen yok.
- İstek: Temiz bir şekilde yapılandırılmış bir çözüm tasarlayacağız.

İlk Soru

Swing ya da SWT gibi bir GUI kütüphaneniz olsun.

MetinGörüntüleyici tasarlamak için hangi atomik GUI elemanlarına ihtiyaç duyarsınız?

- Checkbox
- Window
- Progress Bar
- Combo Box
- Scroll Bar
- Radio Button
- ...



Devam edelim

Göstermek için bir *pencere*

Ekrana sığmayacak kadar uzun metinler için de bir *kaydırma çubuğu*

Peki metni bize kim sağlayacak? GUI'de yok. Ama metne erişebilmeliyiz.

- *DosyaYönetici* bileşeni
- Kısıtlarımız:
 - Dosyanın tüm içeriğini hafızada tutamıyoruz.
 - Kalanı için diske erişip okuma yapmak gerekli.
 - İşletim sistemi seviyesinde dosyaya satır satır erişimimiz var. (Kütüphanemizde dosyadan satır okuma modülümüz mevcut. İstedikimizde belirli bir uzunluğa kadar ardışık satırları çekip bize verebiliyor. Dosya açma, kapatma gibi detayları hallediyor.)

Penceremiz

Metni grafik olarak gösterebilecek bir penceremiz olmalı.

Adına *Gösterici* diyelim.

Tamsayı (integer) satır sayısı gösterebiliyor.

1 ile 100 satır arasında boyutunu değiştirebiliyoruz.

Örneğimizin basit olması açısından, tüm metin aynı yazıtipinde ve aynı boyutta, değiştiremiyoruz.

Kaydırma Çubuğumuz

Dosyanın değişik yerlerini gösterebilmek için ihtiyacımız var.

Alışık olduğumuz, pencerenin sağında duran, yukarı-aşağı hareket ettirilebilen parçası olan bir çubuk.

Bu parçaya «kulp» adını verelim. Kulbun hareket ettiği alana da «tabla» diyelim.

- Kullanıcı tablada kulbu hareket ettirerek dosyadaki gördüğü pozisyonu değiştirebilsin.
- Kulbun bulunduğu pozisyona göre Göstericide dosyanın ilgili parçası gösterilmelidir.
 - Kulp en üstteyken dosyanın başı gösterilmeli.
 - Kulp en alttayken dosyanın sonu gösterilmeli.
- Kulbun boyutunun tabla boyutuna olan oranı, dosyanın Göstericide gösterilen parçasının boyutuna denk düşecektir.
 - Dosyanın tüm içeriği Göstericiye sığıyorsa, kulp tüm tablayı kaplar.
 - Dosya çok büyükse veya Gösterici çok küçükse, incecik bir kulp görürüz.

Kullanım Senaryoları

Elimizde üç adet yapısal eleman adayı mevcut.

- Gösterici, KaydırmaÇubuğu, DosyaYönetici

Metin Görüntüleyici'nin davranışsal tarafına bakalım.

Davranış modellemenin yolu, kullanıcının planlanan çözümü nasıl kullanacağını düşünmektir.

- Bunlara kullanım senaryoları adını veriyoruz.

Metin Görüntüleyici uygulamamız için hangi kullanım senaryoları olabilir?

Metin Görüntüleyici Kullanım Senaryoları

Metin Görüntüleme

Kulbu hareket ettirme

Pencere (Gösterici) boyutunu değiştirme

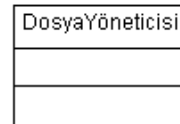
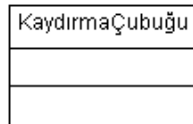
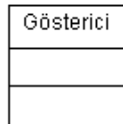
Analiz Modeli

Ana yapısal elemanlar ve davranışlar hakkında karar verdikten sonra, analiz modelini oluşturabiliriz.

UML sınıf diyagramı ile analizi ifade edebiliriz.

Bileşenin adı, özellikleri, davranışları bir arada üç bölmeli dikdörtgende gösterilir.

- Bileşenler arası çizgiler, ilişkileri gösterir



İşlemler

Kullanıcıların Metin Görüntüleyici ile olan etkileşimini sağlayan aksiyonlardan oluşur.

Henüz Analiz modelinde olduğumuzu unutmayalım. Metot implementasyonları ile ilgilenmiyoruz.

- Kullanıcının neler yapabileceğini buluyoruz.

Metin görüntüleyicinin tepki vereceği, dışarıdan görülebilecek öz davranışları nelerdir?

- Metni Gösterme: Bunu yapmak için uygulamanın tepki vereceği herhangi bir olay bulunmamakta (Tabii uygulamanın düzgün şekilde başladığını kabul ediyoruz)
- Kulbu Hareket Ettirme
- Göstericiyi Yeniden Boyutlandırma

İşlem Parametreleri

Bunlar için parametreleri de tanımlayabiliriz:

- Göstericinin boyutları neler olmalı?
- Tablada kulbun pozisyonu nedir?

Boyutlandır davranışının geri dönüş değeri ne olabilir?

- İşlemi dönüş değeri için değil, etkisi için yapıyorsak "void"
- Sadece UML veri tipleri, gerçekleştirme detaylarına girmiyoruz.

Kaydır davranışının geri dönüş değeri ne olabilir?

Gösterici
boyutlandır(yeniBoyut)

KaydırmaÇubuğu
kaydır(yeniKonum)

DosyaYöneticisi

Gereksinimlerimizi Hatırlayalım

Gösterici

- Tamsayı (integer) satır sayısı gösterebiliyor.
- 1 ile 100 satır arasında boyutunu değiştirebiliyoruz.

UML diyagram notasyonunda bunları ifade edemeyiz.

GUI kütüphanemiz büyük ihtimalle piksel pozisyonu döndürecektir, şimdilik o detaylarla ilgilenmiyoruz. Sadece 1-100 arası tamsayı olarak ifade ediyoruz.

Bu gibi engeller hep olacaktır, sadece en önemli detaylara odaklanıp, geri kalanını gerçekleştirme aşamasına bırakmak gerekir.

Görünen Özellikler

Uygulamaya ait özelliklerden, kullanıcının görebildikleri

Algı (percept) adı da verilir.

Algılanan her kavramı bulup özellik olarak modellemeye çalışırız.

Göstericinin görünen özellikleri nelerdir?

- Kulbun konumu
- Göstericinin boyutları
- Kulbun boyutu
 - Değişik boyutlu bir dosyayı gösterirsek ya da göstericinin boyutlarını değiştirirsek, bu da mutlaka değişmeli !
- Ve tabii ki, Metin !!

Bu özellikleri, gerçekleştireceğimiz bileşenlere yerleştireceğiz.

- Gösterici: boyut, metin.
- KaydırmaÇubuğu: kulpKonumu, kulpBoyutu

Görünen Özellikler: DosyaYönetici

Aslında kullanıcı dosya yöneticisi bileşenini direk olarak görmüyor.

Bileşen, dosyadaki metin içeriğini bize sağlıyor.

İşletim sistemi aracılığı ile dosya sistemine erişiyor.

Belge, işletim sistemi aktörü tarafından sağlanan bir görünen özellik

Sistemi kuş bakışı incelediğimizde, neler iç, neler dış paydaştır?

- Kullanıcılar dış paydaş: Sistemden esas faydayı sağlıyorlar
- Dosya sistemi (daha genel olarak işletim sistemi) de dış paydaştır. Dosya Yönetici bu aktörle konuşur.

İlişkiler

Buraya kadar, sınıfları, davranışlarını, görünen özelliklerini oluşturduk. Bu, kolay kısmıydı.

Zor olan, bileşenler arası ilişkileri ortaya çıkarmaktır.

UML analiz modelinde 3 ilişki türü bulunur:

- İşbirliği (association)
- Birleşme (aggregation)
- Genelleme (generalization)

Hangi bileşenlerin kullanıcı aksiyonlarını karşılamak için sorumlulukları olduğunun bulunması ile ortaya çıkarılabilir.

Kullanım senaryoları ve ilgili işlemler bu sorulara cevap verebilir.

İlişkiler

"boyutlandır()" ve "kaydır()" işlemlerimiz için hangi alt eylemlerin olmasını bekleriz?

- Boyutlandır işleminde sadece pencerenin boyutu değişmez.
- Kaydır'da sadece kulbun yeri değişmez.
- Uygulamanın geri kalanının da bir şekilde tepki göstermesini bekleriz.

Boyutlandır:

- Farklı sayıda metin satırına ihtiyaç doğar.
- Kulbun boyutu/konumu değişir.
- Gösterici'nin boyutları değişir.

Kaydır:

- Gösterilecek farklı metin satırlarına ihtiyaç doğar
- Kulbun konumu değişir.
- Gösterici'deki metin değişir.

Bu tepkilerin hepsi, ilgili bileşenler arasındaki ilişkileri ifade eder.

Gösterici-DosyaYönetici İlişkisi: Görünen Satır Sayısı

Herhangi bir anda, Gösterici'de görünen metin satırı sayısını, pencere boyutu ve dosyadaki satır sayısının bir fonksiyonu olarak nasıl ifade edebiliriz?

Pencere Boyutu: Pencerede görünen satır sayısı

Dosya Satır Sayısı: Dosyadaki satır sayısı

Eğer dosyada yeteri kadar satır varsa, Pencere Boyutu kadar satır gösterilir.

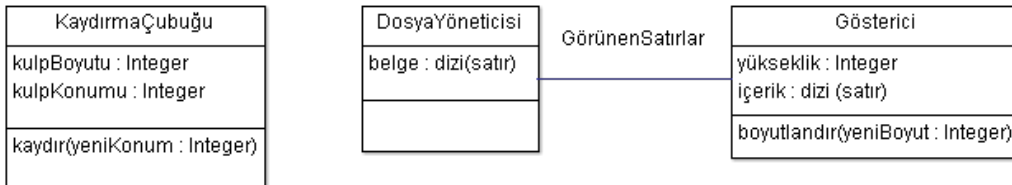
Dosyanın satır sayısı Gösterici'den azsa, Dosya Satır Sayısı kadar gösterilir.

Cevap: minimum(Pencere Boyutu, Dosya Satır Sayısı)

Ama bunu diyagramda gösteremeyiz. Başka bir mekanizmaya ihtiyacımız var:

- OCL (Object Constraint Language)
- UML'in metin tipinde alt parçası
- Yerine getirmemiz gereken çeşitli gereksinimleri tanımlayabilmemize olanak sağlar

Görünen Satır Sayısı İlişkisi



Hangi Satırlar Görünecek?

GörünenSatırlar ilişkisi, satırların DosyaYöneticisi'nden gelmesi gerektiğini gösteriyor.

Ama hangi satırlar?

Hangi satırlar olduğu, kaydırma çubuğunun kulbunun konumu ile belirlenir.

Gösterilecek satır sayısını biliyoruz.

Gösterilecek ilk satırı belirlersek, gerisi kolay.

- Kulbun tepesinin konumu, gösterilecek ilk satırı gösterir.
 - Örneğin tam yarıdaysak, tam ortadaki satırdan başlamalıyız.
 - Sonrasında Göstericiye sığan satır sayısı kadar metin gösteririz.

Hangi Satırlar Görünecek?

Görünecek En üst Satır + Görünen Satır Sayısı

Bunu belirlerken dayanak aldığımız özellikler:

- Gösterici: Boyut,
- DosyaYönetici: belgeBoyutu
- KaydırmaÇubuğu: kulpKonumu

Burada üç bileşen arasında üçlü ilişki bulunmaktadır: "Görünür"

- Gösterici, DosyaYöneticinin sağladığı, KaydırmaÇubuğu kulpKonumu tarafından belirlenen satırları gösterir.

KaydırmaÇubuğu Kulp Boyutu

Bileşenlerimiz arası ilişkiler şeklinde nasıl ifade edebiliriz?

- kulpOranı=1 ise, tüm tablayı kaplar.

GörünenSatırSayısı / dosyaBoyutu

100 satır görünüyor ve 1000 satır varsa, kulp tablanın %10'unu kaplar.

DosyaYöneticisi, Gösterici ve KaydırmaÇubuğu arasında yine üçlü ilişki bulunuyor. KulpBoyutOranı ilişkisi.

İnce Detaylar

Belgenin satır sayısı 0 ise ne olacak?

- Sıfıra bölüyoruz. Özel durum.

Kulp boyutu derken tam olarak ne kastediyoruz?

- GUI kütüphanesi büyük ihtimalle piksel değerleri döndürecek, bunları oran ya da tamsayıya çevirmemiz gerek.

Dosyanın en sonuna gittik diyelim. Sonra Göstericiyi daha büyük olacak şekilde boyutlandırdık. Göstericide ne görürüz?

- Aynı satırlar, ilaveten sonda boş satırlar
- En alt satır yine en altta, üstte daha fazla satır

Uygulamayı biz tasarlıyoruz, hangisinin olacağına biz karar vermeliyiz.

Bu şekilde modelleme ile hiç düşünemeyeceğimiz şeyleri de göz önüne alabiliriz.

- Sizi, ince detaylar üzerinde düşünmeye zorlar.

Ne yaptık?

Basit bir metin görüntüleyici problemini çözmek için analiz modeli oluşturduk.

Henüz problemi çözmedik, tasarımda çözeceğiz.

Kendimize sormamız gereken soru:

- Herhangi bir Metin Görüntüleyici uygulamasının ele alması gereken anahtar tasarım sorusu nedir?

Tasarım Kavramları

Tasarım Her Yerdedir

Uluslararası Uzay İstasyonu

Haftaya planladığınız aile yemeği

Biz yazılım mimarisi ve tasarımına odaklansak da, genel tasarım kavramlarının çoğu yazılımda da rol oynar.

Bazı tanımlar

Tasarım: bilinçli, amaçlı planlama

Mühendislik: bilimsel ve matematiksel temelleri uygulayarak becerili veya sanatsal üretim

Zanaat: yetenekli meslek (yetenek deneyimden gelir)

Sanat: Beceri, tat ve hayal gücünü estetik nesnelerin üretiminde kullanma

Yazılım Tasarımı ve Programlama arasındaki farklar

Ölçek:

- Büyük bir çözüm gerektiren büyük bir problemle uğraşıyorsunuzdur.
- Programlarken, 100-1000-10000 satır kodla problemi çözersiniz.
- Uluslararası Uzay İstasyonunda 30 milyon satır kod var.

Fonksiyonel olmayan gereksinimlere önem vermek

- Programlarken, en fazla performansla ilgilenebilirsiniz.
- Tasarımda, işlevsel olmayan gereksinimler arası dengeyi sağlamalısınız.

Yazılım Tasarımı

Problemin işlevsel gereksinimlerini yerine getirirken işlevsel olmayan gereksinimleri de atlamadan yazılım üretimi

Birşeylerden ödün vermek gerekecek ama hangisinden ve nasıl?

- Performans ve Kaynak Kullanımı?

Performans için tasarım yapsaydınız, dizi mi kullanırsınız nesne koleksiyonu mu?

- Hava tahmininde komşuluk gridi değişirse?

Mimari Tasarım:

- Bir yazılımın modül veya bileşenlerine davranışsal yönleriyle belirlenmiş sorumlulukları atama işlemi
- Program parçaları bileşenler halinde birleştirilir, her bileşene davranış şeklinde sorumlulukları yüklenir, bileşenlerin nasıl etkileşime geçeceği belirlenir.

Detaylı Tasarım:

- Mimari tasarımıda belirlenen davranışların teker teker bileşenlere veri yapıları ve algoritmaları da düşünülerek tanımlanması

Detaylı Tasarım Dili

Pseudo Code (sözde kod) Program Tasarım Dili (PDL)

- Anahtar kelimeler, doğal dilde serbest ifadeler, veri tanımları, alt program tanımı ve çağrıları

Yapısal Programlama

- Sıralı ifadeler, koşullar, döngüler, bölütleme (alt program)

Akış Diyagramı

- Yönlü graflar, düğümler işlem birimleri, bağlantılar veri akışları

Karar Tabloları

- Kurallar, koşullar, aksiyonlar

Yazılım Tasarımı Yaklaşımları

Bir sistemi en iyi nasıl yapılandırırız sorusunu cevaplayanlar

- Nesneye yönelik tasarım
- Yapısal tasarım
- Rol tabanlı tasarım

Belirli bir sınıfa ait yazılımlara odaklananlar

- Gerçek zamanlı sistemler

Uygulamanın sadece belirli bir kısmına odaklananlar

- UI/UX tasarımı

Yazılım Tasarımı Yaklaşımları

Tüm yaklaşımlarda ortak olan 3 yön:

- Tasarım yöntemi:
 - Tasarımcıların bir problemi çözmek için uyguladıkları sistematik ve sıralı işlem adımları.
 - Genellikle problemi göstermek için belirli bir yol önerir.
 - Örneğin Nesneye yönelik tasarımda, problem işbirliği yapan nesneler şeklinde tanımlanır.
 - Yöntem genel olarak tüm takıma düşünce ve aktivitelerini belirli şekilde yapmaları için disiplin sağlar
- Tasarım gösterimi
 - Görsel veya yazılı ifadelerle tasarımı ifade etme
- Tasarım geçerleme

Yazılım Tasarım İkilemleri

Mimar/tasarımcı olarak kararlar vermeniz gerekir.

- İşleri yukarıdan aşağıya doğru mu, aşağıdan yukarı doğru mu, içten dışa doğru mu yapacaksınız?
- Yordam ve fonksiyonlar mı düşüneceksiniz, kavramlar ve davranışlar mı?

Yazılım üretiminde kaçınılmaz sorun: Tasarım zaman alır.

- Uzun dönem yönetilebilir yapı mı, kısa süreli takvimli işi yetiştirmek mi?

Hangi araçlar kullanılacak?

Tasarım gözden geçirme (geçerleme)

Tasarım yönteminiz var.

Tasarım gösteriminizi de seçtiniz.

Tasarımınızı yaptınız.

Bir takım, gereksinimlerle uyumlu olup olmadığını gözden geçirir.

Bu aşamada gözden geçirmeye gerek var mı?

- Zaten programı gerçekleştireceğiz, (pek çoğu otomatikleşmiş) testler yapacağız?
- En sonda gözden geçirsek?

Problemi ne kadar erken tespit ederseniz, o kadar ucuz ve hızlı çözersiniz.

Tasarım Geçerleme

Bir takım tarafından tasarım gözden geçirilir. Kullandığınız araçlar da bir takım kontrolleri sizin için yapabilir.

Önemli hususlar:

Geçerleyenlerin bağımsızlığı

- Tasarlayan ve geçerleyen aynı kişiler olursa, belirli şeyleri gözden kaçırabilirler
- Tasarlarken düşünmedikleri detayları, geçerlemede de düşünemezler.

Tasarım yöntemine bağımlılık

- Yapısal tasarımda tasarım sonuçlarıyla metrikleri eşleştiren kurallar kümesi mevcuttur

Tasarlarken geçerleme / Bitirince geçerleme

- Günlük veya haftalık olarak yaptıklarınızı gözden geçirip düzeltmek
- Kilometre taşına kadar üretip, geçerleme ve düzeltmeleri yapmak

Diğer tasarım unsurları

Mimari ve detaylı tasarımın sınırları nerededir?

İşlevsel ve işlevsel olmayan gereksinimlere ne kadar ağırlık verilmelidir?

"Ne" ve "nasıl" dengesi nasıl kurulacak?

- Tanımlama (ne)
- Tasarlama (nasıl)

Uygulama özelinde tasarım işlemlerini etkileyen faktörler nelerdir?

- Belirli kaynak ve önceden hazırladığımız çözümleri kullanmak isteyebiliriz

Tasarım belgelendirme

Büyük sistemler karmaşıktır.

Ölçek ve karmaşıklık, iyi belgelendirme ihtiyacını arttırır.

Tüm gücümüzü geliştirme için kullanmış olalım.

- Sistem bir süre iyi çalışacaktır.
- Sonrasında bakıma girecektir.
- Tasarım belgelendirme, taraflar arası iletişimi kolaylaştırır

Formal belgeler ya da basit notlar, hatta sunumlar bile olabilir.

Geleneksel Tasarım Belgeleri

Sistemi bir araya getiren bileşenler

- Sorumlulukları
- Veri akışları

Kontrol akışı

Performans kriterleri

Kaynak kullanımı (hafıza, harici birimler, bant genişliği, ...)

Bunlar yetmiyorsa, IEEE 1016 standardı

- Tasarımcı kim, parçalar arası bağımlılıklar neler, gizli varsayımlar var mı, işlevsel olmayan gereksinimler arası ödünleşmelere nasıl karar verdiniz, kullanıcı/teknoloji/donanım varsayımlarınız neler, ...

Tasarım gerekçelendirme

Bu kadar detaylı bilgileri bir araya getirerek tasarım gerekçeleri oluşturulur

Tasarım çözümümüzde neler yaptık, neden yaptık

- Ör: Hız ve boyut arasında nasıl bir seçim yaptık, neye göre yaptık

Ne kadar açık sebeplere dayalı seçimler yaparsak, ileride sistemin bakımını yapacak taraflara o kadar kolaylık sağlarız.

Tasarımda Anahtar Kavramlar

Kavramsal Bütünlük

- Descartes: Farklı ellerden çıkan çok parçalı işlerde mükemmellik, tek bir ustanın elinden çıkana oranla çok az görünür.
- Fred Brooks (The Mythical Man-Month): Bağımsız ve koordine olmamış kavramlar yerine bir tasarım fikir kümesinden çıkmak kaydıyla, alışılmadık özellikler ve gelişmeler sağlayan sistemler iyidir

Az Bağımlılık

- Sisteminizin bileşenleri birbirine bağımlıdır. Başarılı çalışma için iki bileşenin mümkün olduğunca az bağımlı olmaları istenir.
- Çok bağımlı olurlarsa, birindeki değişiklik diğerini de değiştirmenizi gerektirir.

İyi Uyum

- Bir bileşenin belirli bir amacı ya da hizmeti olması iyidir.
- İyi uyuma sahip birimleri tekrar kullanma olasılığı daha yüksektir.

Java paketleri az bağımlılığı mı iyi uyumu mu destekler?

Java sınıf kalıtımı bağımlılığı arttırır mı azaltır mı?

Tasarımda Anahtar Kavramlar

Bilgi Gizleme (David Parnas)

- Belirli bir modülün yapabildiklerini, soyut bir arayüz arkasında saklamak. Kullanıcılar sadece görünen arayüzü bilebilir.
- Gerçekleştirme detaylarımızı değiştirebilmemizi sağlar.
- Erişim gizliliği (donanım, veritabanına, sunucuya, ...) sağlar

Soyutlama (abstraction)

- Karmaşık sistemdeki problemleri ayrıştırma, belirli noktalara odaklanma
- Analizi tanımlarken tasarım detaylarından soyutlanmak
- Programlama dillerinde dizi, struct, nesne gibi yapıların içeriği soyutlaması
- Nesneye yönelik genelleştirme: özel durumlar yerine sınıf davranışı
- Parametreler: Farklı tipler ve detaylar yerine sadece isimlerle kullanım

Tasarımda Anahtar Kavramlar

Estetik

- Aquinas (James Joyce çevirisi): Güzellik için üç şey gerekir: Bütünlük, ahenk, saflık
- Yazılım karşılıkları: Bütünlük (completeness), tutarlılık (consistency), kavramsal bütünlük (conceptual integrity)
- Pascal: Bu mektubu çok uzun yazdım çünkü daha kısa yapabilecek kadar vaktim yoktu.
- Karmaşık gereksinimleri yerine getirecek basit ve etkin bir çözüm geliştirmek için zaman ve enerji harcamanız gerekir.

Tasarım Felsefesi (Piet Mondrian)

- Yazılım tasarımı dört filozof ile ilişkilendirilir.
- Descartes: Analitik geometri. Analiz, model kullanımı
- Marx: Sosyal etki. Kullanıcı merkezli tasarım, tasarımın sosyal etkileri (hatta kullanıcıları tasarıma dahil etmek)
- Martin Heidegger: Hedeflere ulaşmak için araç kullanımı. CASE, IDE
- Wittgenstein: Dil oyunları. Bir kelimenin keşfi, bir kavramla ilgili düşünmenizi sağlar. Desktop, folder, trash, client/server, icon, firewall...

Tasarım

Yazılım geliřtirmenin en yaratıcı kısmıdır.

Sistemin genel kalitesi, üretilen tasarımlara direk bağımlıdır.

Tasarım kalitesini yansıtan önemli bir gösterge, tasarım ekibinin benzer projelerdeki deneyimidir.

UML Diyagramları

Diyagramlar

Tasarımlarımızı bir şekilde göstermemiz gereklidir.

- Bunu yapmanın en popüler yolu diyagramlardır.

Bu derste UML'e odaklanarak,

- Nasıl kullanıldığını,
- UML diyagramlarının neler olduğunu,
- Nasıl oluşturulduğunu gözden geçireceğiz.
- Pek çok UML diyagramı mevcut, hepsini kullanmayacağız.



Diyagramlar

Diyagramlar yazılım geliştirmede en başından beri oldukça popülerdir.

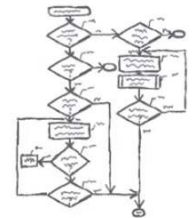
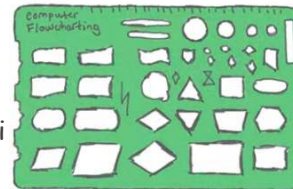
- Algoritmaların akış diyagramları için şablon kullanırız

Zaman içinde çok çeşitli karmaşık diyagram teknikleri ortaya atılmıştır

- SADT
- Jackson Design
- Structure Design, ...

'80'lerin sonlarından itibaren nesneye yönelik tekniklerin yaygınlaşmasıyla diyagramlar da değişti

- OMT



OMT

Object Management Technique

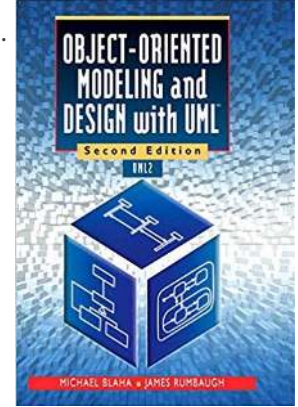
General Electric'de James Rumbaugh ve ekibi tarafından geliştirildi.

Sonuçta bir zar içeren kitap kapağı ortaya çıktı.

- Bir yüzünde Sınıf diyagramı
 - Yapısal yön
- Bir yüzünde Durum-Geçiş diyagramı
 - Davranışsal Yön
- Bir yüzünde Ardışıl Diyagram
 - İşlevsel yön

OMT, sisteme 3 bakış açısını uyum sağlayan bir görünümle birleştirdi.

- Rakipleri de geliyordu.



UML

OMT, değişik teknikleri bir araya toplama çalışmalarının gelişmesine neden oldu. En bilineni, Tümleştirilmiş Modelleme dili olan UML (Unified Modeling Language)

OMG Object Management Group tarafından standart hale getirildi.

- Teknik şartnameleri burada bulabilirsiniz.

IBM tarafından da desteklendi.

- Rational firması, Rational Rose ürünü

Pek çok ticari/bedava ürün mevcut.

UML'in üç ana mimarı vardır. «Three Amigos»

- James Rumbaugh
- Grady Booch
- Ivar Jacobson
- UML dili dışında, 3 kitap, 1 elkitabı, 1 kullanıcı kılavuzu... üretmişlerdir.

UML diyagramları hem analiz, hem de tasarım için kullanılabilir. Ana ayrım,

- Analiz çözülen problemle ilgili,
- Tasarım problemin çözümü ile ilgilidir

UML

UML 2.0: 14 çeşit diyagram

- Ders kapsamında 3-4 tanesine odaklanacağız.

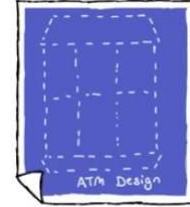
İki ana diyagram kategorisi:

Yapısal

- Sistemin her zaman olan parçalarını ve aralarındaki ilişkileri tanımlar

Davranışsal

- Sistemin çalışma şekilleri
- Her diyagramda bir çalışma gösterilir.



Tasarım Diyagramları

Tasarım diyagramları kullanmanın

Faydaları:

- İletişim: Bilgiyi grubun diğer elemanlarına da aktarabilirsiniz
 - Bu elemanlar sizin ekip arkadaşlarınız da olabilir, projede hiç konuşmadığınız insanlar da olabilir.
 - Hatta 5 yıl boyunca uygulamanın bakımını yapacaklar bile olabilir.
- Mevcut yöntemlere verdiği destek
 - Kullandığınız yazılım geliştirme metodolojisini destekler. Örneğin tasarım gözden geçirmeyi yapılandırır.
- Araç desteği
 - Kullandığınız araçlar, diyagramlardaki eksikler veya hatalarda size yardımcı olur.

Zararı:

- Tüm diyagramları güncel tutmaya çalışmanız gerekir. Sistemin geri kalanı değiştiğinde gözden geçirip güncellemelisiniz.

Yapısal Diyagramlar

Sınıf Diyagramı

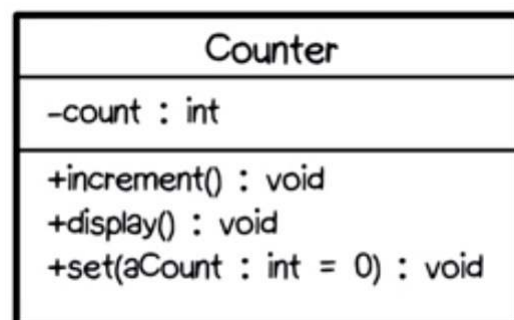
En popüler diyagram

Statik model ve sınıf yapı diyagramı olarak da bilinir.

Sistemin yapısını gösterir.

Sınıflar ve ilişkileri yer alır.

- Sınıflarda isim, özellikleri (oluşum değişkenleri) (instance variables), metod ve işlemler yer alır.



Sınıf Diyagramında İlişkiler

Bağımlılık (Dependency)

- X, Y'yi, kullanır.



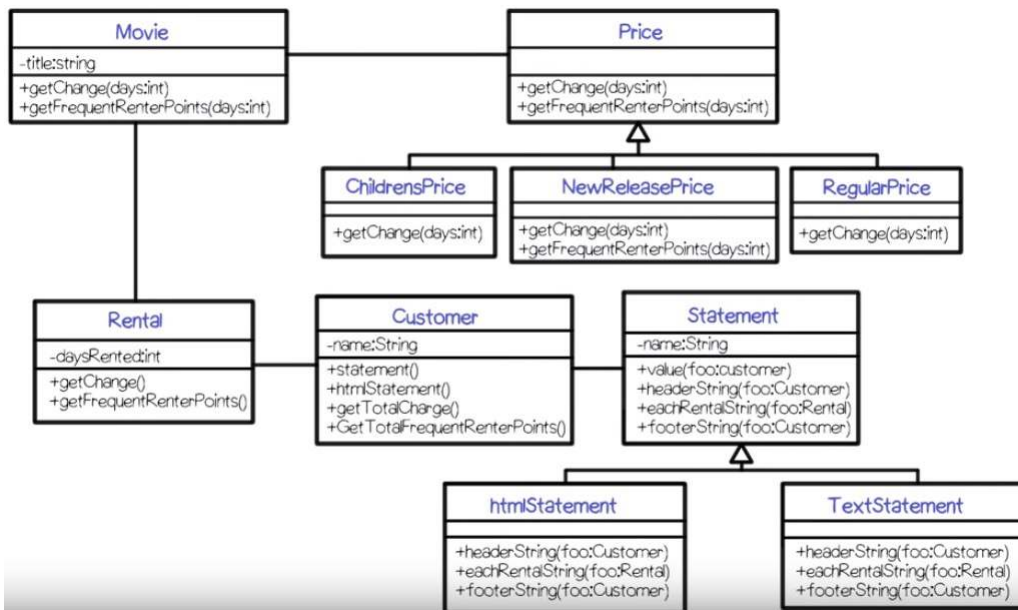
İlişki (Association)

- X, Y'yi etkiler ya da Y'den bir örnek içerir.
- Ucunda eşkenar dörtgen de koyulabilir. Birleştirme (aggregation), (has a) anlamı verir.



Genelleştirme (Generalization)

X bir Y tipindedir.



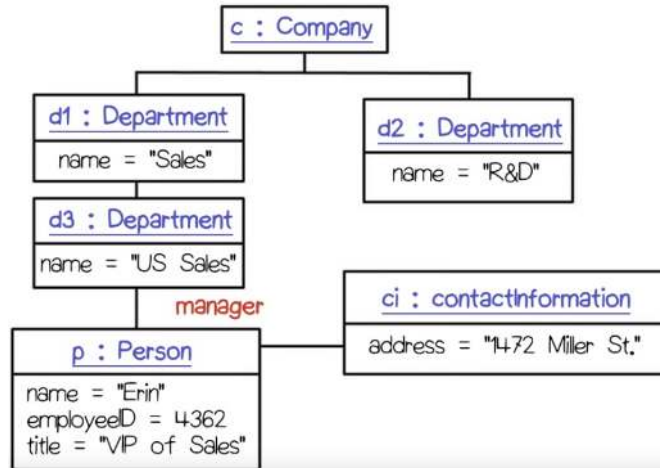
Nesne Diyagramı

Sınıflar yerine türetilmiş nesne örneklerini içerir.

İsmin altı çizili olarak yazılır.

Türetilen nesnenin adı ve sınıfı bulunur.

Belirli bir nesne olduğundan, özelliklerin değerleri de yer alır.



Kompozit Yapı Diyagramı

Daha az bilinir ve daha az kullanılır.

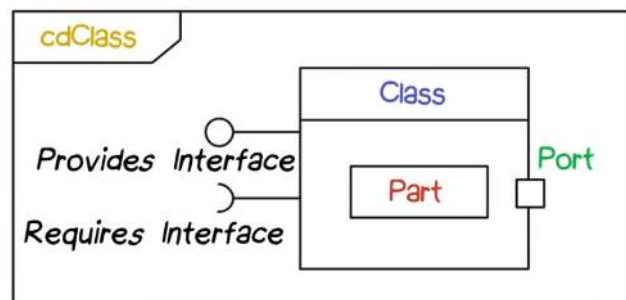
Sınıfın iç yapısını göstermek için çizilir.

Bizim için ilgili yanı, arayüzlerdir.

- Sağladığı arayüz: Bu sınıf dünyaya bazı yetenekler sunuyor
- Gerektirdiği arayüz: Bu sınıf, dünyadan ne bekliyor?

Sağladığı ve gerektirdiği arayüzleri üzerinden sınıflar birbirine bağlanır.

Bir yazılım mimarisinin parçalarını birleştirmenin yollarından biridir.



Bileşen (Component) Diyagramı

Bileşen diyagramı da tam olarak bunu yapar.

Bir sistemin bileşenlerinin nasıl bir araya geleceğini gösteren statik bir gerçekleştirme görünümü sunar.

UML'de *bileşen*, sistemin fiziksel ve değiştirilebilir bir parçasıdır. Bileşen gerçekleştirmeyi kapsar ve bir arayüz kümesinin gerçekleştirmesine hem uyar, hem de katkı sağlar.

UML Reference Manual: A physical, replaceable part of a system that packages implementation and conforms to and provides a realization of the set of interfaces.

Genellikle kütüphane şeklinde kod varlıklarının modellenmesinde kullanılır.

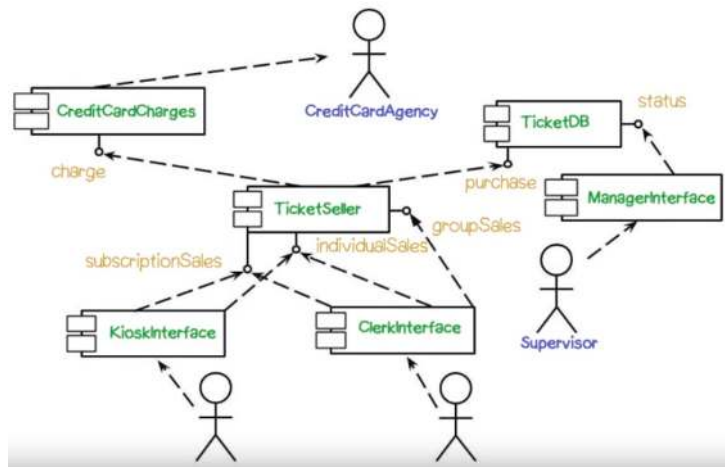
Diyagramdaki ilişkiler, bir bileşenin diğer bileşene ait servisleri kullandığını gösterir.

Bileşen Diyagramı

Bileşen sembolü Bouche'nin Bouche'gram'larından gelir.

Çöp adamlar sistemin aktörleridir

Kesik çizgiler bileşenlerin bağlantı yerlerini oluşturur



Barındırma (Deployment) Diyagramı

Karmaşık sistemlerde parçalar farklı işlem birimlerinde çalışabilir.

Çalışma zamanı işlem birimlerinin (türetilmiş bileşenlerden oluşan) konfigürasyonları ve nasıl etkileştikleri bu diyagram ile resmedilir.

UML Reference Manual: Configuration of run-time processing nodes and the component instances and objects that live on them.

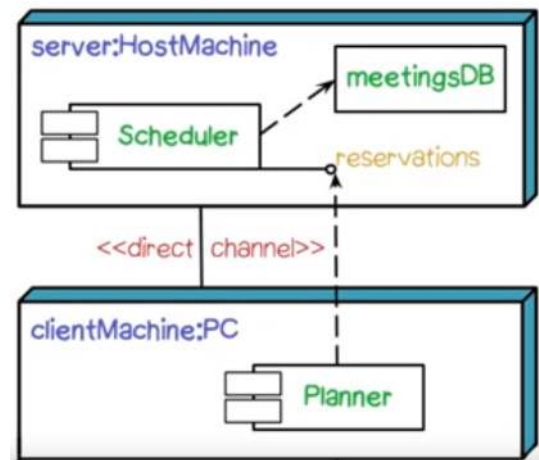
Düğümler işlem cihazlarını, kenarlar da haberleşmeyi betimler.

Barındırma Diyagramı

İki işlem ünitemiz var.

İçlerinde bileşenler yer alıyor.

Hem fiziksel bağlantıları, hem de mantıksal bağlantıları (birbiri ile kenetlenmiş arayüzler) gösteriyor.



Paketler

UML Java'daki gibi paket yapısını da destekler.

Genel amaçlı düzenleme mekanizmalarıdır.

UML2 ile ayrı bir diyagram haline geldi.

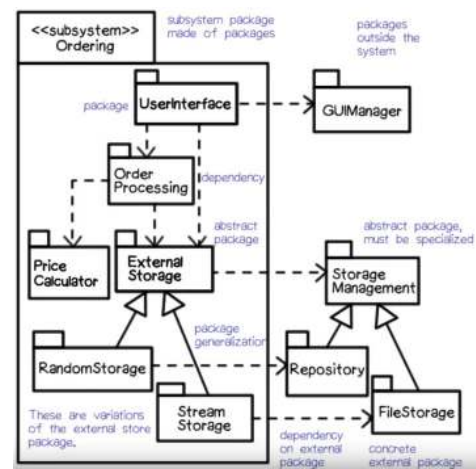
«Namespace» mantığını sağladığı için her paket çakışmaya neden olmadan kendi isim kümesine sahip olabilir.

Paket içindeki parçalar arasında bağımlılık ilişkisi varsa, bu paketler arasında gösterilir.

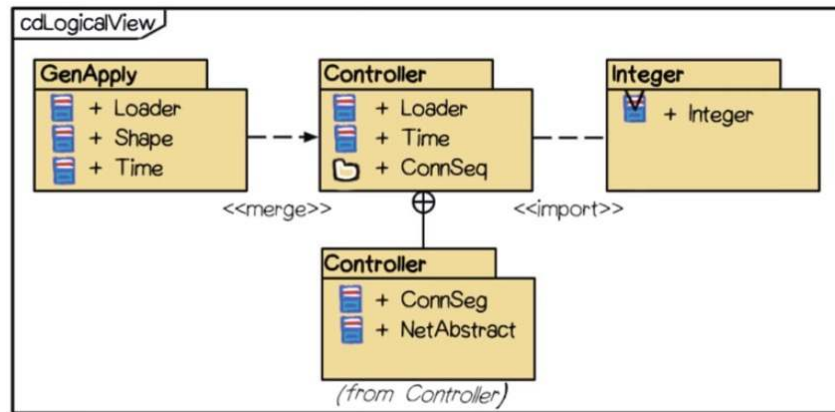
UML 1.5'te Paketler

Solt üst köşedeki çıkıntıda etiketi ile gösterilir.

Paketler arası bağımlılıklar kesikli oklar ile gösterilir



UML 2.0 Paket Diyagramı



Profil Diyagramı

Bahsedeceğimiz son yapısal diyagram.

UML sınıf, bağlantılar gibi çeşitli parçaları olan bir dildir. Bu parçalar bir sistemi tanımlar. UML'i UML ile tanımlayabilirsiniz.

- UML meta-model

Daha da soyut olan, tasarımcı olarak UML modelini genişletebilirsiniz.

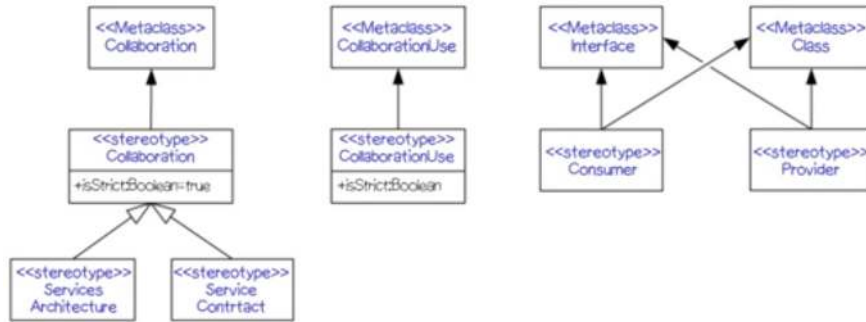
- Modeldeki belirli elemanlara yeni ikonlar, özel etiketler ekleyebilirsiniz.

Bu ilaveler bir UML profili içinde yapılır. Profil diyagramı ile bunu ifade edebilirsiniz.

Profil Diyagramı

<<metaclass>> ve <<stereotype>> şablonları

Standart UML'i belirli türdeki sistemleri tanımlamak için genişletiyoruz.



UML Yapı Diyagramları - Genel

Sınıf: Bileşenler ve yapısal özellikleri

Kompozit Yapı: İç yapı ve olası etkileşimler

Bileşen: Fiziki yazılım bileşenlerinin organizasyonu

Barındırma: Fiziki sistem kaynakları ve donanıma olan dağıtımları

Nesne: Belirli bir zamandaki statik yapı

Paket: Mantıksal grupta ve bağımlılıklar

Profil: UML meta modeline yapılan eklentiler

Diyagramlarda fazla sayıda örtüşmeler bulunuyor.

Sistem Mimarisini İfade Eden UML Yapı Diyagramları

Sınıf Diyagramı

Bileşen Diyagramı

Barındırma Diyagramı

Paket Diyagramı

Davranışsal Diyagramlar

Davranışsal Diyagramlar

Yapısal diyagramlar sistemi bir bütün olarak tanımlarken, davranışsal diyagramlar belirli bir davranış örneği ile ilgilidir.

Genel sistem davranışını göstermek için birden çok ardışıl veya işbirliği diyagramına ihtiyaç vardır.

Bir sistemde tüm diyagram tiplerini yine kullanmak zorunda değiliz.

Kullanım Senaryosu (Use Case) Diyagramı

UML'de, OMT'nin veri akış diyagramları yerine Jacobson'un kullanım senaryosu diyagramları yer alır.

Kullanım senaryosu: Sistem tepkilerini de içerecek şekilde kullanıcı tarafından gözlenebilir eylemler sırası.

Sistemin belirli bir kullanıcı etkileşimi ile nasıl başa çıktığının hikayesidir.

Kullanım senaryoları özellikle gereksinimleri ortaya çıkarmak için yararlıdır.

Sistemin nasıl kullanılacağına dair değişik hikayeleri ortaya çıkarıp, sonra dallanmaları somutlaştırırız.

- Birşeyler yanlış olursa ne olacak?
- Belirgin yapmadığımız ara adımlarda neler olacak?

Kullanım Senaryosu (Use Case) Diyagramı

İki ana parçası vardır

Çöp adamlar dış aktörleri ifade eder. Sistemin kullanıcıları oldukları gibi, başka sistemler ya da harici cihazlar da olabilirler.

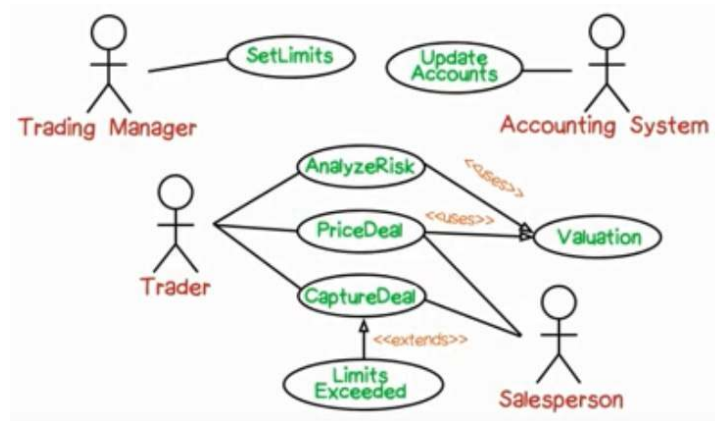
Elipsler senaryolardır. Kullanım senaryosu diyagramı, bir sistem hikayesi değildir. Sistem senaryoları kümesinin tanımıdır.

Açıklama taşımayan çizgiler ortaklık/katılım demektir. Bir uçtaki aktör, diğer uçtaki senaryo ile ilgilidir.

İki özel açıklama vardır.

- `<<extends>>` Bir hikayemiz var ve bunu zorunlu olmadan duruma göre değişen durumlarla genişletmek istiyoruz. *İkisi tek fiyatına.*
- `<<include>>` `<<uses>>` Alt yordam/fonksiyon çağırısı gibidir. Pek çok senaryoda ortak kullanılan davranış parçasıdır.

Kullanım Senaryosu Diyagramı



Kullanım Senaryosu

Diyagamda hepsi bir aradaydı.

Kullanım senaryosu bir hikayedir ve yapısız metin ya da tablo şeklinde yazılabilir.

Örneğin;

Ayşe Amazon'dan ürün almak ister.

Ayşe Amazon web sitesine girer.

Ayşe aradığı ürünü bulana kadar sitede gezer.

Ayşe ürünü sepete ekler.

Ayşe Satın alma sayfasına geçer.

Ayşe fatura bilgilerini verir ve ödemeyi tamamlar.

Kullanım Senaryosu Tablo Gösterimi

Aktör	Eylem	Nesne
Ayşe	gezer	Amazon websitesi
Ayşe	seçer	Kitap
Ayşe	ekler	Kitap, Sepet
Ayşe	Satın alır	
Amazon	ister	Fatura bilgisi
Ayşe	sağlar	Ad, adres, kart bilgisi
Amazon	doğrular	Kart bilgisi
Ayşe	onaylar	satış

Bağlam (Context) Diyagramı

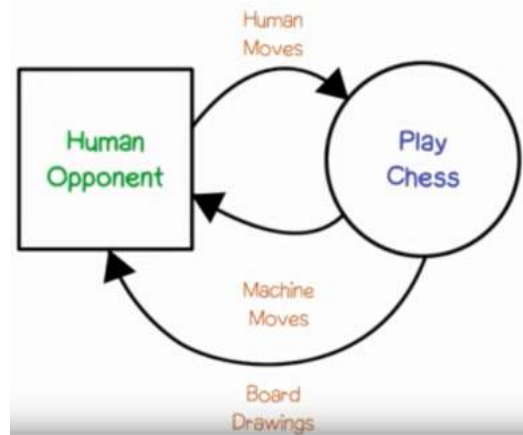
UML'in parçası değil ama ilginç kabiliyet sunar

En üst seviye veri akış diyagramıdır.

Tek bir elips içerir. Sistem bir bütün olarak yer alır.

Dikdörtgenler aktörleri gösterir.

Bağlantılar aralarındaki veri akışlarıdır.



Ardışıl (Sequence) Diyagram

Bir senaryoyu açıklar.

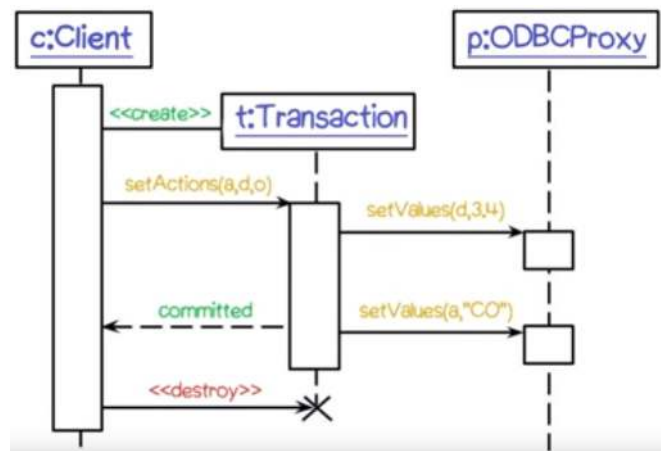
Her kolonda tekil bir katılımcı (genellikle nesneler) yer alır.

Zaman, aşağı doğru çizisel olarak devam eder.

Yatay çizgiler, nesneler arası mesajları gösterir.

Telefon sanayisindeki mesaj sırası kartlarından evrimleşmiştir.

İletişim diyagramlarıyla birebir aynı içeriğe sahiptir.



İletişim Diyagramı

Sınıf diyagramına benzer.

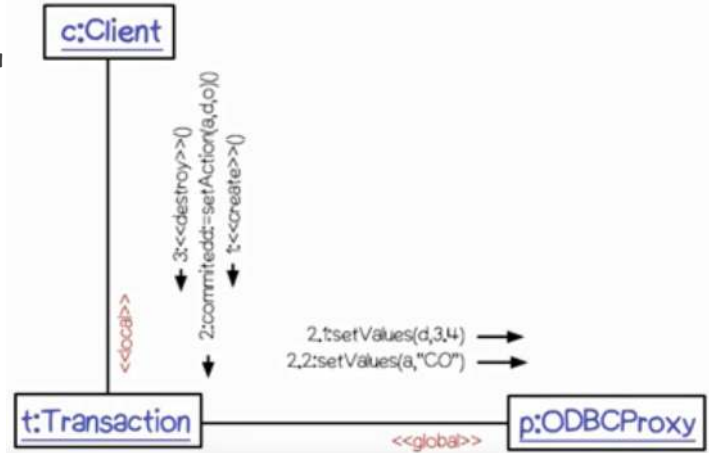
Fark olarak çizgiler işlem çağrılarını (haberleşmeleri) taşır.

Bir önceki örneğin aynısı:

Mesaj belirteçleri numaralıdır.
Sırayı gösterir.

Dewey ondalık gösterimini kullanır.

- 1, 2, 2.1, 2.2, 3, ...



Aktivite Diyagramı

Ardışıl ve işbirliği diyagramlarında senkronizasyon bilgisi yer almaz.

Birden çok durumun aktif olabileceği bir durum geçiş makinesi betimler.

- Hepsinin kendi kontrol parçacığı vardır.

Petri ağlarından türetilmiştir.

- Petri ağlarında durum geçişleri aktivitenin tamamlanması ile tetiklenir.
- Dış olaylar yerine, durumu bitirmeniz gerekir.

Aktivite diyagramları ile iş akışlarını, işlem senkronizasyonunu ve eşzamanlılığı modelleyebiliriz.

Aktivite Diyagramı

Diyagramda tüm durumların üzerinde durabilecek bir simgemiz olsun.

Dolu daire olan tepeden başlar, yatay çizgiden ilerleyerek ilk duruma geçer. Sonra dörtgene geçer. Burada ikiye ayrılır. Biri sağa, biri de aşağı ilerler.

Sağdaki gibebildiği kadar aşağı (alttaki dörtgene kadar) gider.

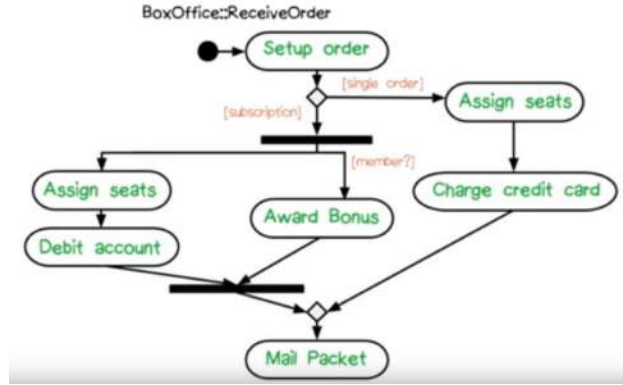
Aşağı giden, yatay siyah bar tarafından engellenir. Burası bir senkronizasyon noktasıdır.

Bu vakada senkronize edecek birşey yoktur ama altta iki çizgi çıkmaktadır. Herbirinde simgenin bir kopyası olur. Biri soldaki iki aktiviteye girer, ikincisi sağdan aşağı devam eder.

Sonunda bu ikisi ikinci yatay çizgide birleşir. Burası da bir senkronizasyon noktasıdır.

Kendi simgesine sahip iki yol bağımsız çalışır ve alttaki yatay çizgi sadece tepeden gelen iki simge de gelip birleştiğinde açılan bir kapı gibi davranır (Aktivitelere senkronize eder).

Birleşen simge dörtgene (birleşme noktasına) girer, oradan da son duruma ve bitişe geçer.

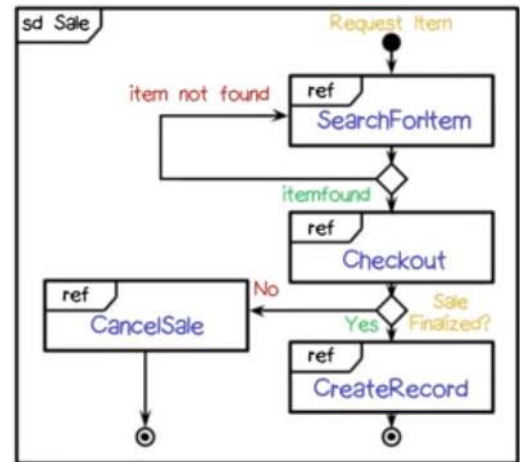


Genel Etkileşim Diyagramı

Bir çeşit aktivite diyagramıdır.

Düğümler alt seviyeli etkileşim diyagramlarını gösterir.

- Ardışıl, iletişim, genel etkileşim, zamanlama diyagramı olabilir.



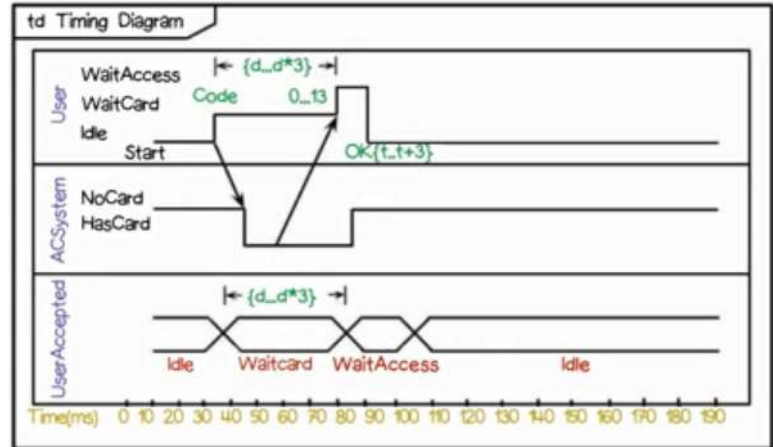
Zamanlama (Timing) Diyagramı

Dijital çip tasarımından miras

Sistemde durumların belirli zamanlamaları olması gerektiğinde kullanılır.

Zaman soldan sağa ilerler,

Oklar zamanlama senkronizasyon noktalarını gösterir.



Durum (state) Diyagramı

Davranışsal diyagramların en güçlü ve karmaşık olanı

Sonlu durum makinelerinin yeteneklerini, birleştirme (aggregation), eşzamanlılık, geçmiş, olayları tüme yayım gibi maddelerle genişletir.

Bunlarla ilgili bir dersimiz olacak.

Durum Diyagramı

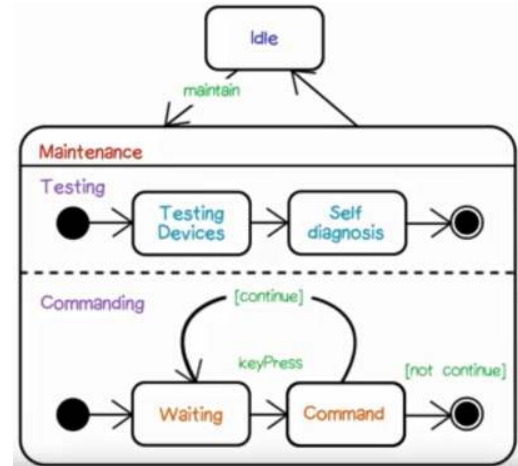
İki harici durum: Boş ve Bakım

Bakım durumunda alt durumlar var.

- Test
- Komut

Bunlar eşzamanlı çalışır.

- Test durumu soldan sağa çalışır
- Komut durumu, son duruma gelene kadar pekçok kez çalışabilir.



UML Davranış Diyagramları - Genel

Aktivite: Aktiviteler arası akış kontrolü

Ardışıl: Mesajlaşan sınıflar arası etkileşim

İletişim: Numaralandırılmış mesajlarla nesne etkileşimi

Genel Etkileşim: Daha alt seviyedeki aktivite diyagramlarının sentezi

Zamanlama: Döndürülmüş ardışıl diyagram

Kullanım Senaryosu: Harici aktörlere sağlanan sistem fonksiyonları

Durum: Uyarana tepki olarak dinamik davranış

Tekrar: Geliştireceğiniz bir sistemde hepsine ihtiyaç duymazsınız. En popüler olanlarını kullanırsınız.

İhtiyacınız olursa, hepsi ortada duruyor.

Nesne Kısıt Dili

UML'in diyagramsız kısmı ve diyagramların metin uzantısı

Diyagramlarda tanımlayamadığımız şeyler için kullanılır.

- Sınıf ve durum geçiş diyagramlarında metin uzantısı olur.

Sistem tanımlarında kesin olma gereği olduğunda kullanılır.

Nesne Kısıt Dili

Sınıf diyagramında bir hesap sınıfı var. Hesap sınıfının para yatırma işlemi olsun.

Bu işlem, "miktar" isimli bir gerçek sayı alsın.

Pre anahtar kelimesi miktarın sıfırdan büyük olmasını gösterir.

Post anahtar kelimesi son miktarın, mevcut + ilk miktar olmasını gösterir.

```
context Account::deposit(Real : amount)
pre: amount > 0
post: balance = balance@pre + amount
```

UML Meta Model

UML'in UML ile tanımı

UML meta modeli UML profilleri ile Profil diyagramı kullanarak genişletebilirsiniz.

Yapacağınız eklentiler ilave edeceğiniz stereotipler, etiket değerleri ve kısıtlardır.

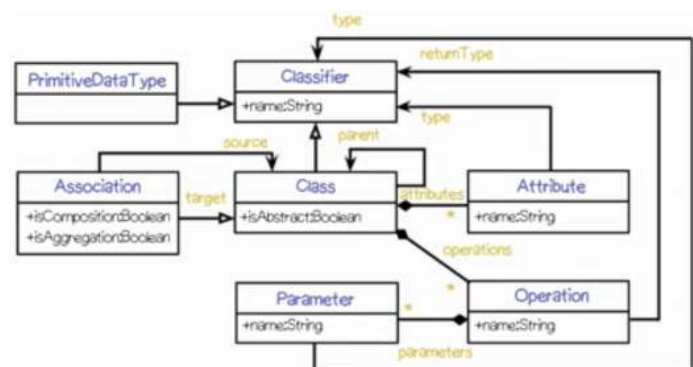
Kısıtlar modelde değil diyagramlardadır.

UML Meta Model

Kendi sınıfı olanlar:

- Sınıflar
- Sınıfların özellikleri
- Bir ilişki sınıfı
- Sınıflar arası ilişkiler

UML'i oluşturan çeşitli parçaların hepsi burada birarada bulunuyor.



Sonuç

Karmaşık bir sistemi ne yapacağını varsaymadan tasarlayamazsınız.

Diyagramlar problemi nasıl anladığınızı ve çözümünüzü nasıl ifade ettiğinizi temsil eder.

UML'de kullanabileceğiniz pek çok diyagram vardır, bunların yanında OCL ve meta model de mevcuttur.

Problemi ne kadar iyi anlarsanız, sistemin geliştirme geçmişinde o kadar az alt problemle karşılaşırsınız.

Nesneye Yönelik Analiz

Tasarıma Nasıl Başlarsınız?

Problemi çözmeniz için önce anlamanız gerekir. → Analiz

Nesneye Yönelik Analiz gerçekleştireceğiz.

Abbott ve Booch tarafından 1980'lerde geliştirildi.

Nesne analiz modeli yaratmak için, doğal dil ile tanımlanmış gerçek dünya nesnelerini modellemeye odaklanır.

UML sınıf diyagramları ile ifade edilir. Önemli olan, problem ile ilgili nesnelerdir.

Alternatifi fonksiyonel çözümlemedir. Çözümün sağlaması gereken fonksiyonlara odaklanır.

Neden Fonksiyon yerine Nesneler?

Yazılım sistemlerinin uzun yaşam ömürleri vardır ve bu süre boyunca evrimsel değişikliklere uğrarlar.

Özellikler eklendikçe, hatalar düzeltildikçe, kütüphaneler değiştikçe programın orijinal yapısı da kaybolur.

Bakım esnasında fonksiyonlar nesnelerden çok daha sık değişir.

Örnek: Bankacılık uygulaması.

- Faiz hesabı, İşlem ücreti hesaplama, vergi alma gibi FONKSİYONlar zaman içinde değişir.
- Hesaplar, Müşteriler, İşlemler gibi NESNELER ise değişmez.

Nesneler fonksiyonlardan daha kararlı yapılardır.

- Nesneler, daha sürdürülebilir tasarımlar sağlar.

Nesneye Yönelik Analiz ve Tasarım

Yapısal analiz ve tasarım, fonksiyon yönelimlidir.

Yapılması gereken hesaplamalar üzerine yoğunlaşırlar.

Hesaplamaların üzerinde işlediği veriler ise ikinci plandadır.

Nesneye yönelik analiz ve tasarım veri nesnelerini ilk planda ele alır.

- Özellikler ve veri tipleri önce modellenir.
- Sonra nesnelere davranışlar (fonksiyonlar) eklenir.

Nesneye Yönelik Analiz

Gerçekleştirilecek sistemin metin ifadesini (gereksinim dokümanları) alırsınız.

- Ya da oluşturursunuz.

İsim, fiil, sıfat gibi kelime tiplerini araştırırız (altlarını çizeriz)

- Sınıfları isimlerle
- Özellikleri sıfatlarla
- İşlemleri eylem bildiren fiillerle
- İlişkileri durum bildiren fiillerle oluştururuz.

Oluşan sınıf modeli müşteri ile gözden geçirilebilir.

Kolay gibi görünse de ancak düşünerek düzgün çözebileceğimiz tarafları da olacak.



Bir ağacın yapraklarını saymak

1. İsimlerden aday sınıflara

Ağaç: Döngü içermeyen çizge. Her düğümün sıfır ya da daha fazla alt düğümü ve en fazla bir üst düğümü olabilir.

Yöntem:

Sayılmamış ağaç parçalarını alıp boş yığına koyun.

Yaprakların sayacını sıfır yapın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.

Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.

İki tane alt ağaç varsa sol ve sağı ayırıp ikisini de yığına ekleyin.

Yığın boşaldığında, yaprak sayısını gösterin.

Bir ağacın yapraklarını saymak

1. İsimlerden aday sınıflara

Ağaç: Döngü içermeyen çizge. Her düğümün sıfır ya da daha fazla alt düğümü ve en fazla bir üst düğümü olabilir.

Yöntem:

Sayılmamış ağaç parçalarını alıp boş yığına koyun.

Yaprakların sayacını sıfır yapın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.

Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.

İki tane alt ağaç varsa sol ve sağı ayırıp ikisini de yığına ekleyin.

Yığın boşaldığında, yaprak sayısını gösterin.

Başımıza gelenler

Nesneye Yönelik Analize isimlerle başlayınca olanlar:

Bazı kelimeler tekrar eder.

- Ağaç, ağaç, ağaç

Bazı kelimelerin kökü ortaktır.

Bazı kelimeler anlamsal olarak birbirine çok yakındır ve aynı saklı konsepti içerir.

- Yaprak yapraklar

Bunları yoğunlaştırıp tek kopyasını ve kelime kökünü kullanacağız.

- Sayaç / Sayı ?

2. İsimlerimizden Aday Sınıflara

Yığın, Parçalar, Ağaç, AltAğaç, Sayı, Sayaç, Yaprak, Yapraklar, Sıfır

Temizleyerek aday sınıflarımızı elde edelim:

Yığın → Önemli bir kavram olarak görünüyor.

Parça ? Alt ağaç da olabilir?

Ağaç → bu da önemli

Sayaç → bu da önemli

Yaprak ? Özel bir ağaç isimlendirmesi olabilir mi?

Düğüm? Analist probleme orjinal tanımda olmayan, hiç düşünülmemiş kavram ilave edebilir. Problemlî, tufarsız ya da eksik kısımları ortaya çıkarabilir.

Sıfır? → Sayacın bir özelliğinin değeri olabilir. Tabii modelimizin bir parçası. Sonra unutmayın!

UML Kavramsal Sınıf Diyagramı

Aday sınıflardan gruplar oluşturulur ve analiz modeli Kavramsal sınıf diyagramı ile ifade edilir.

Sınıf: birbiri ile ilgili nesne grubunun tanımı



Analiz modeli için uyarılar

Tüm analizlerde olduğu gibi, vardığımız sonuçlar kesin değildir.

Süreçte ilerledikçe, problem hakkında daha fazla bilgi sahibi oldukça, modelimiz revizyona uğrar.

Analist, gereksinim belgelerindeki eksikleri gidererek doğru (müşterinin gerçekten istediği) modeli oluşturabilmelidir.

- Müşterilere geri dönmek gerekebilir.
- Çoğu zaman, sizin soru olarak getirdiğiniz şeyi düşünmemişlerdir bile.
- Sonunda size teşekkür ederler.

Süreç, analist ve müşteri arasında artımlı olarak devam eder ve ortak anlaşılmış bir noktada son bulur.

Bir ağacın yapraklarını saymak

3. Sıfatlardan özelliklere

Sıfatları, etkiledikleri isimlerle birlikte işaretleyelim:

Sayılmamış ağaç parçalarını alıp boş yığına koyun.

Yaprakları sayacını sıfır yapın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.

Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.

İki tane alt ağaç varsa sol ve sağ ayırıp ikisini de yığına ekleyin.

Yığın boşaldığında, yaprak sayısını gösterin.

Bir ağacın yapraklarını saymak

3. Sıfatlardan özelliklere

Sıfatları, etkiledikleri isimlerle birlikte işaretleyelim:

Sayılmamış ağaç parçalarını alıp boş yığına koyun.

Yaprakların sayacını sıfır yapın.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp inceleyin.

Ağaçta tek yaprak varsa, yaprakların sayacını arttırın ve ağacı atın.

İki tane alt ağaç varsa sol ve sağ ayırıp ikisini de yığına ekleyin.

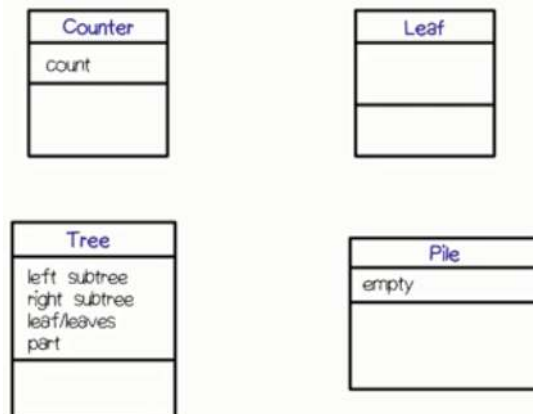
Yığın boşaldığında, yaprak sayısını gösterin.

Bir ağacın yapraklarını saymak

3. Sıfatlardan özelliklere

Yığın: boş	Her sıfat özellik olacak diye bir şart yok. İlgisizleri elemeliyiz.
Ağaç: iki, sol, sağ	Edatlardan da özellikleri elde edebiliriz.
Sayaç: yaprak	- Ağacın parçaları - Yaprakların sayısı
Yaprak: tek	Tek yaprak, yaprağın alabileceği sayı değerlerinden sadece biridir. Öyleyse, değeri bir olan bir sayı özelliğimiz olmalı.
Parça:	İki alt ağaç, bir ağaçta olabilecek alt ağaçların sayısı özelliğinin bir değeri olmalı.
	Parça, ağacın bir parçası olduğu için özellik olmalı.

Güncellenmiş Sınıf modelimiz



Bir ağacın yapraklarını saymak

4. Fiillerden işlemlere

İşlem: Nesnenin sağladığı bilişimsel servis.

- Eylem fiilleri ile bulunabilirler. Eylemleri ilgili olduğu sınıflara atarız.
 - İlgili fiilleri özellik belirtir. Ör: büyük görünen ev
 - Durum fiilleri nesne değil durum bildirir, nesneler arası ilişkileri oluşturur

Sayılmamış ağaç parçalarını boş yığına **koyun**.

Yaprakları sayacını sıfır **yapın**.

Yığın boş olmadığı sürece, tekrar tekrar ağaç alıp **inceleyin**.

Ağaçta tek yaprak **varsa**, yaprakların sayacını **arttırın** ve ağacı **atın**.

İki tane alt ağaç varsa sol ve sağı **ayırıp** ikisini de yığına **ekleyin**.

Yığın boşaldığında, yaprak sayısını **gösterin**.

İşlemlerin ilgilileri kim?

Örneğimizde yığını kim tutacak?

- Tasarladığınız her sistemde, sistem de bir sınıf olabilir. Onun da sorumlulukları var.

Sayma işini kim yapacak? Neyi sayacak? * "**Sayılmamış**" kelimesi, aslında eylem değil durum belirtiyor

- İşlem ve argümanları
- Savaş, yaprakları.

Ağacı (parçaları) yığına kim koyacak/yığından kim alacak?

- Yığının sorumluluğu, ağaç da argümanı

Kim Sıfır yapacak? Kim arttıracak?

- Savaş

Kim inceleyecek?

- Sistem mi? Yığın mı?
- Şu aşamada statik yapıyı, sistemi anlamaya çalışıyoruz, davranış çözümlemesi daha sonra.

Ağacı kim atacak?

- Savaş mı? Yığın mı?

Ağacı kim bölecek?

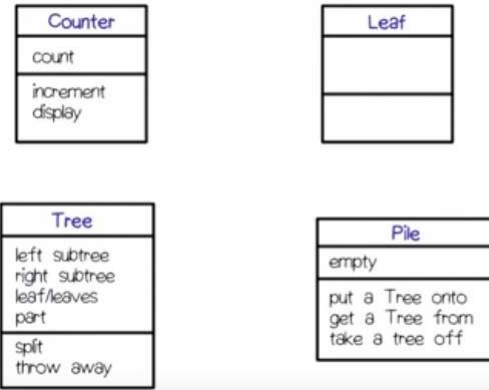
Sınıf Modelimiz

İşlemlerimizin parametrelerini, veri tiplerini, dönüş değerlerini de istersek belirtebilirdik.

Ancak esas amacımız,

- Problemin parçaları nelerdir ve
- Bunlar hangi özelliklerle temsil edilir
- Hangi servisleri sağlayacaktır

cevaplarını bulmaktır.



5. İlişkiler/Bağlantılar

Sınıf modelinin sınıflardan geriye kalan yarısı

Genelleme (Generalization[Specialization])

- Alt sınıfın nesneleri, üst sınıfın türündedir.
- Alt sınıf, üst sınıfın alt kümesidir.
- Aslında içindeydi, biz onu dışarı çıkarıp ayırdık
- Ör: Araba / Araç, Ağaç / Yaprak

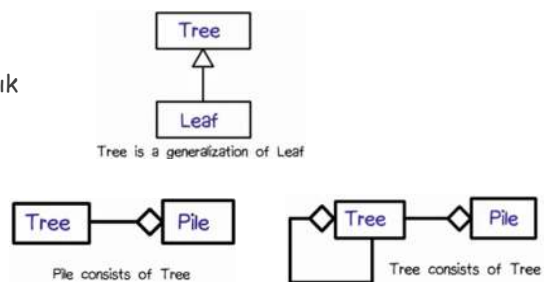
Birleşme (Aggregation) (Parça-bütün)

- Bir çeşit "X koleksiyonu", "X grubu"
- İçerir, aittir, kapsar, vardır, sahibidir, ...

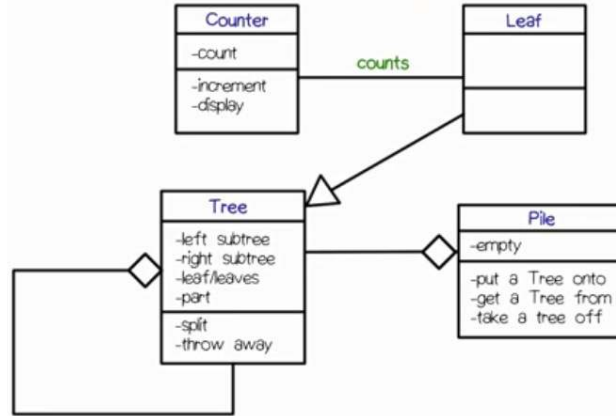
İşbirliği (Association)

- Durum bildiren fiillerle ortaya çıkar
- Bağlantının üzerine ismi yazılır
- Ör: Öğrenci üniversitede okur.
- Sayaç-Yaprak

- Generalizations
- Aggregations
- Associations



Sınıf Modelimiz



Analiz modeli için uyarılar

Tüm sınıflarımız aslında herşeyi kapsayan AğaçYaprakSaymaSistemi sınıfındadır. Genellikle diyagramlarda sistem sınıfı gösterilmez.

Normalde tipik gereksinim dokümanları yüzlerce sayfa olabilir ve pek çok özel kelime içerir.

Örneğimiz çok kısaydı ve aslında algoritma ve gerçekleştirme detayları içeriyordu.

Gereksinim dokümanlarında çözüm yolları yer almaz, sadece problemi içerirler.

UML Sınıf Modelleri

UML ile problemi modelleme

UML'in en popüler tarafı. Analiz modelini nasıl ifade edeceğinizi tanımlar.

Aslında en karmaşık olanı

- Modellemenin başında tüm detaylarını kullanmak zorunda değilsiniz.

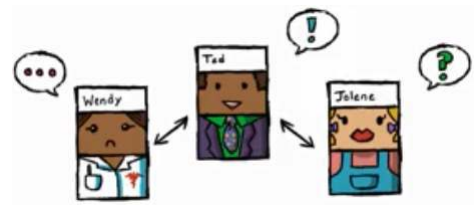
Bu derste tüm yönlerine bakacağız.

- İhtiyacınız olduğunda kullanabilmeniz için bilmeniz gerekli

UML Sınıf diyagramları Statik Yapı diyagramı olarak da bilinir

Sınıflar dışında arayüzler, nesneler, ilişkiler de yer alabilir.

UML Elkitabı bu konuda temel referans kaynağıdır.



Sınıf

Benzer örneklerin kümesinin tanımıdır.

Neler sınıf olabilir?

- Etki alanı nesneleri
- Roller
- Olaylar
- Etkileşimler

İsimlerle aday sınıflar bulunabilir.

Genellikle üç parçalı dikdörtgen ile gösterilir.

- Ad kesindir, genellikle özellik ve işlemler de yer alır
- Sorumluluklar, aykırı durumlar, ... da ayrı parçalarda dörtgen içinde gösterilebilir.



Ad Alanı

Sınıfın adı bir isim olmalıdır !!

Eğik yazılırsa (veya <<abstract>>) soyut sınıf anlamı taşır

- Soyut sınıf: alt sınıfların özelliklerini tanımlayan, kendilerinden asla örnek türetilmeyen kontratlardır.
- Ortak özellikleri olan alakalı sınıfları, bir soyut sınıf ile gruplayabilirsiniz.

Bunun dışında, stereotipler (kışeler) ile temel UML modelleme dilini genişletebilirsiniz.



Nitelikler

Sınıfların nitelikleri vardır.

- Özellikler
- Davranışlar

Sınıflar gerçek hayatta var olurlar.

Nitelikler, aslında bilgisayardaki karşılıklarıdır.

- Özellikler: Örnek değişkenleri
- Davranışlar: Metotlar.

Özellikler Alanı

Tüm özelliklerin adları, tipleri, ve erişim türleri vardır.

Erişim türleri görünürlüklerini (diğer sınıflardan erişilebilirliklerini) tanımlar ve başta opsiyoneldir, sonra ekleyebilirsiniz.

+: Genel görünür.

-: Özel görünür

#: Korumalı görünür (sadece alt sınıflar)

~: Paketlerde, paket içinde görünür.

Çoğullamalar da belirtilir.

Özelliklerin tipleri tanımlanmalıdır.

İlk değer atamaları istenirse yapılabilir. Türetildiği (hesaplanarak bulunduğu, atanmadığı) da belirtilebilir.

Özel tanımlamalar {...} ile yapılabilir.

- {frozen}: özelliğin değeri değiştirilemez.

İşlemler Alanı

İsimleri tanımlanmalıdır.

Metotların görünürlüğü özelliklerdeki gibi tanımlanabilir.

Değer döndüren işlemlerin dönüş tipi belirlenmelidir.

Varsa Parametreleri, tipleri tanımlanmalıdır.

- istenirse ilk değer atamaları ve içeri/dışarı tanımları yapılabilir.

Özel tanımlamalar {..} ile yapılabilir.

- {query}: bu bazı özelliklerin bilgisi veren sorgu işlemidir,
- {concurrency}
- {abstract}

Sınıf Kapsamı bu işlem sınıf kapsamındadır, tekil nesnelere ait değildir. Ör. Bu sınıftan kaç nesne türettik?

Örnek Sınıflar

Rectangle
pt:Point p2:Point
<pre> <<constructor>> Rectangle(pt:Point, p2:Point) <<query>> area(): Real aspect(): Real ... <<update>> move(delta: Point) scale(ratio: Real) ... </pre>

Reservation
<pre> operations guarantee() cancel() change(newDate: Date) </pre>
<pre> responsibilities bill no-shows match to available rooms </pre>
<pre> exceptions invalid credit card </pre>

İleri Sınıf Modeli

Arayüzler

- Bir arayüz yaratıcısı ile programınızda tip tanımlayabilirsiniz.
- Arayüz sisteme neler sunuyor, sistemden neler bekliyor

Parametrik sınıflar

- Java generics ya da C++ templates olarak gerçekleşir. Koleksiyonda yer alan sınıfların türünün ne olduğunu belirten bir parametre tanımlanmasını sağlarlar.

İççe (nested) sınıflar

- Sınıf içinde sınıf tanımı.
- UML bu durumları tanımlamak için bir özellik sunar.

Kompozit Nesneler

- İçinde başka nesneler barındıran nesneler

İleri İlişkiler/Bağlantılar

İsimlerle sınıfları, fiillerle işlemleri ve bağlantıları bulduk

İşbirliği (Association)

- İnsan, araç → insanlar araçları kullanır.

Genelleme (Generalization)

- Araba bir araçtır

Bağımlılık (Dependency)

- Arabalar ve Emisyon Kanunları
- Emisyon kuralı değişirse, arabaların yeni bir cihaz takılması gibi adapte olmaları gerekir

İleri ilişkiler

Poligon **içerir** noktalar

- İfadenin sonundaki üçgen okuma yönünü gösterir
- İlişkilerin erişim yönlerini çizginin ucunu ok yaparak gösterebilirsiniz (navigability)

İkinci ilişkinin adı yok

- Role göre koyabiliriz.
- İlişki sınıf da tanımlayabiliriz

İçerme ilişkileri

- Açık dörtgen birleşme (aggregation)
 - Poligon noktalardan yapılmıştır
- Kapalı dörtgen bütünleşme (composition)

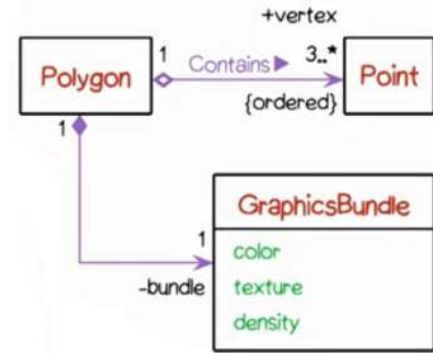
Çoğullamaları tanımlarız. Ör: 3..*

Kısıtlamaları tanımlarız.

- {ordered} stereotipi ile belirli bir sırada olmalarını belirtiyoruz.

Rolleri tanımlarız. Ör: bundle: bu ilişkide Grafik sınıfının rolü "bundle"

- Her iki uçta da tanımlayabiliriz.



İlişki Sınıfları

Sınıf nitelikleri taşıyan ilişkileri ayrı bir sınıf olarak modelleyebiliriz.

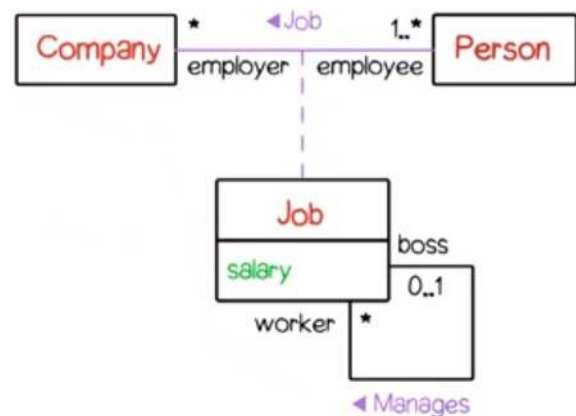
Kişi **çalışır** şirkette ilişkisinde, maaş özelliği tanımlayabiliriz.

- Maaş kişinin değil, çalışma ilişkisinin özelliğidir.
- Maaş şirketin özelliği de olamaz, pek çok çalışsını var.
- Kişinin birden fazla görevi için ayrı maaşları olabilir.

İlişki sınıfları, ilişki çizgisine dayanan kesikli çizgilerle gösterilir.

Burada İş sınıfının kendine ilişkisi vardır. Ozyinelemeli ilişki adı verilir.

- Rol tanımlarını iki uçta da yapmak faydalıdır.
- Patron **yönetir** işçiyi.



Birleşme ve Bütünleşme

Birleşme: Bir sınıfın bir örneği, diğer sınıfın pek çok örneğinden yapılmıştır.

Yine bir ilişkidir. Birleşmeyi göstermek için içi boş dörtgen kullanılır.

Birleşme ilişkisi, ilişkinin anlamı ile ilgili bilgi vermez. Örneğin nenelerin yaşam ömürleri hakkında bilgi vermez.

Ev ve oda sınıflarımız olsun.

- Evin odaları vardır, burada birleşme ilişkisi vardır.
- Ancak, evi yıkarsak, tüm odaları da yıkmış oluruz.
- Buna bütünleşme ilişkisi diyoruz ve içi dolu dörtgen ile ifade ediyoruz.

Bütünleşmede, belirli bir öge sadece tek bir bütünleşmede yer alır.

- Ögenin yaşam süresini yönetme sorumluluğu da vardır.

Bütünleşmeler genellikle geçişken yapıdadır. Evin odaları, odanın gömme dolapları, ...

Odada masa olsun. Bu bir birleşmedir. Odayı yıkmadan önce masayı alırız. Bu nesnelerin birbirine bağlı olmayan ayrı yaşam ömürleri vardır.

Hangi ilişki?

Dersler ve Öğrenciler

- Birleşme
- Öğrencileri yok etmeden dersi kapatabilirsiniz

Gelin ve Damat

- İlişki
- Bağımsız yaşam ömürleri var

Banka Hesabı ve Mudi

- Birleşme
- Mudinin banka hesabı olabilir

Yazı tipleri ve Gölgeler

- Bütünleşme
- Yazı tipi giderse tüm süslemeleri de gider

Niteleyiciler (Qualifiers)

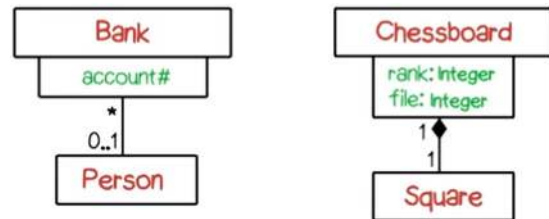
Sınıf dörtgenlerinin kenarında yer alan küçük dörtgenlerdir.

Sınıfın belirli bir özelliğinin ismini taşır.

- Bu özellik sınıfın nesnelere erişim sağlar
- Veritabanındaki birincil anahtar gibi düşünebiliriz.

Ör: insanlar bankaya hesap numaraları ile erişir

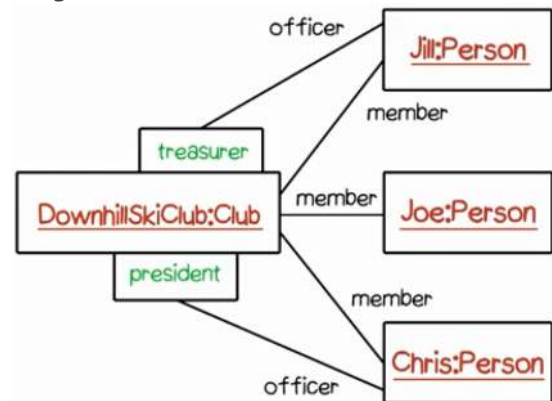
Ör: Satranç tahtasında kareler koordinat ile tanımlanır. (Burada bütünleşme de var)



Bağlantılar (Link)

Sınıfların örnekleri olabildiği gibi, ilişkilerin de bağlantıları olabilir.

Bir nesneyi tekrar göstermek yerine her rol için bir bağlantı çizilir.



Genelleme (Generalization)

Bağlantının ucunda üçgen yer alır.

Üçgen tarafındaki üst sınıf / ana sınıftır.

Diğer uçtaki(ler) alt sınıf / çocuk sınıftır.

Anlamı: alt sınıfın örnekleri aynı zamanda üst sınıfın da örneğidir.

Nesneye dayalı programlamadaki kalıtım ile birebir aynı kavram değildir!

- Kalıtım gerçekleştirme tekniğidir, genelleme is modellemedir.

UML'e çoklu üst sınıf ve çoklu alt sınıflar tanımlanabilir.

Ayrıştırıcılar (discriminator) ile alt sınıflar gruplanabilir.

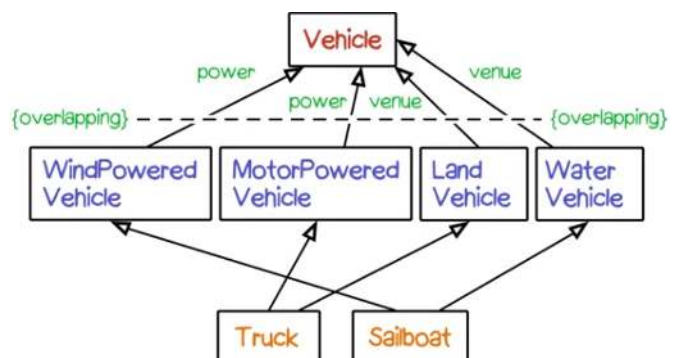
Genelleme (Generalization)

Bir üst sınıftan türeyen alt alt sınıflar birden fazla alt sınıfa aitse **{overlapping}** kullanılır.

Sadece tek bir üst sınıfa ait olursa alt alt sınıflar **{disjoint}**dir

Çocuk sınıfın durumu, kendisinden türeyen tüm nesneleri kapsıyorsa **complete**, kapsamıyorsa **incomplete**

- Örneğin hibrit bir aracınız var ve çocukların hiçbirine uymuyor.



Genelleme ve Kısıtlarına örnekler

Atlet: VoleybolOyuncusu, BasketbolOyuncusu, FutbolOyuncusu

- Overlapping: Bir oyuncu hem voleybol hem futbol oynayabilir.
- Incomplete: Bunları oynamayan atletler de var

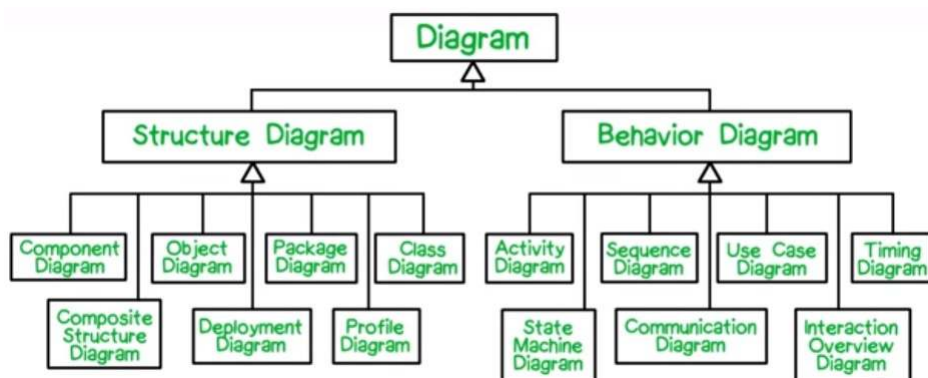
Kitaplar: KağıtCiltli, ÇizgiRoman, BezCiltli

- Disjoint: Tekil bir kitap aynı anda ikisi birden olamaz
- InComplete: Başka kitap türleri de var.

Omnivorlar mı üst sınıftır, Vejetaryenler mi?

- Vejetaryenler

UML Diyagramlarının Hiyerarşisi



Tasarım Çalışmaları

Tasarım

Tasarım, kararlar vermektir.

- Genellikle işlevsel olmayan kriterlerden hangisinden ödün verileceği ile ilgilidir.

Müşteri, son kullanıcı, teknik beklentiler, rakip ürünler bu kararları etkiler.

Bazen de daha detaylı analiz gerekir.

- Simülasyonlar
- Prototipler
- Tasarım Çalışmaları

Tasarım Çalışmaları

Bir mimar bir binayı tasarladığında, yaptığı işlerin ilki tasarım çalışmasıdır.

Tasarım uzayını daha iyi hissetmek üzere bir takım ölçekli modeller üzerinde çözüm oluşturabilecek değişik yaklaşımları keşfetmeye çalışır.

Benzer yaklaşım pek çok alanda uygulanır:

- Araba tasarımı
- Uçak tasarımı
- Giysi tasarımı



Tasarım çalışması, bir tasarımı etkileyen faktörlerin titiz ve sistematik bir değerlendirmesidir.

- İlgili kriterler, nasıl ölçülecekleri ve hangi ölçüm sonuçlarının kabul edilebileceği belirlenir.
- Değişik yaklaşımlar bu kriterlere göre ölçülerek değerlendirilir.

Tasarım Uzayları

Tasarım çalışmaları, tasarımcının bir seçenekler uzayını daha iyi keşfetmesini sağlar.

Ör: Binalarda estetik önemli rol oynasa da daha nesnel faktörler de göz önüne alınmalıdır:

- İnşaatın maliyeti
- İnşaat malzemelerinin bulunabilmesi
- İnşaat yasalarına ve bölge kısıtlarına uygunluk
- Trafiğe olan etki

Biz, bilgisayar yazılımlarının tasarımlarını tasarım çalışmaları ile değerlendireceğiz.

- Performans,
- Kullandığı bellek
- Üretim zamanı

Gibi işlevsel olmayan faktörler önemlidir.

Tipik bir Tasarım Çalışması Raporu

1. Bağlam: Geçmiş, motivasyon, anahtar kelimeler ve tanımlar
2. Araştırma soruları: Bağımlı değişkenler cinsinden karşılaştırılan işlevsel olmayan gereksinimler ve etkilenen faktörler
 - (ör: ön işleme hesapları değişince zaman/bellek kullanımları nasıl değişiyor?)
3. Konuların tanımları: Programımız. Çözüm yaklaşımlarımız neler?
4. Deneye özel koşul ve durumlar: Kullandığınız konfigürasyon özellikleri (tekrar edilebilirlik için gerekli)
5. Değişkenler (Bağımlı ve bağımsız): Değiştirdiğiniz durumlar bağımsız değişkenleri, bunlarla elde ettiğimiz sonuçlar bağımlı değişkenleri oluşturur. Durumların adı, tanımı, ölçü birimi (ör saniye) tanımlı olmalıdır.
6. Metot: Denemeleriniz, ölçü alet ve araçlarınız, çalışan konular ve kullanılan argümanlar (ör grid boyutu)
7. Deney Sonuçları: Elde edilen sonuçlar.
8. Tartışma: Sonuçların yorumlanması ve değerlendirilmesi. Ör. Anormal sonuçlar, neler yapılabilir.
9. Sonuç: Araştırma sorularınıza açık cevaplar verebilmeli.

Kütüphane Örneği

KÜTÜPHANECİ ALİ, ARKADAŞI VELİ'DEN ÇALIŞTIĞI
KÜTÜPHANE İÇİN BİR BİLGİ SİSTEMİ
GELİŞTİRMESİNİ RİCA EDER.

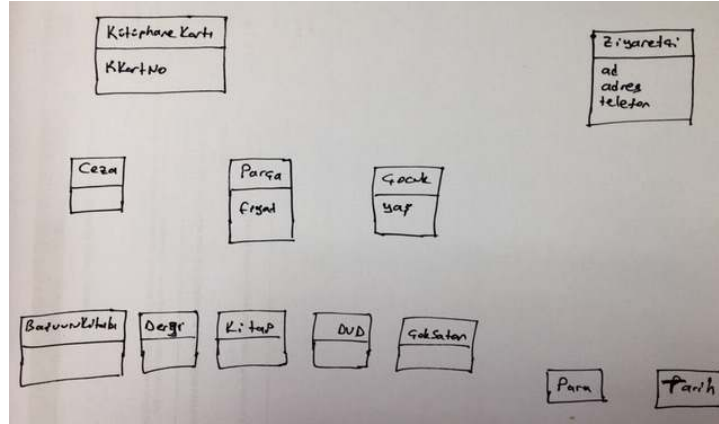
Gereksinimleri Tanımlar:

1. Her ziyaretçi sistemde olduğu sürece bir tekil kütüphane kartına sahiptir.
2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
4. 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.
5. Ziyaretçi kitap veya DVD ödünç alabilir.
6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.
7. DVD'ler iki hafta için ödünç alınabilir.
8. Gecikme cezası her parça için günlük on TL'dir. Gecikme cezası parçanın fiyatından daha yüksek olamaz.
9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
10. Ziyaretçi, kütüphanede olmayan bir kitabı ya da DVD'yi talep edebilir.
11. Ziyaretçi bir parça için sadece bir kez süre uzatma yapabilir. Parça için çok talep varsa, parçayı geri getirmelidir.

Gereksinimleri Tanımlar:

1. Her ziyaretçi sistemde olduğu sürece bir tekil kütüphane kartına sahiptir.
2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
3. Kütüphane herhangi bir zamanda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
4. 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.
5. Ziyaretçi kitap veya DVD ödünç alabilir.
6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.
7. DVD'ler iki hafta için ödünç alınabilir.
8. Gecikme cezası her parça için günlük on TL'dir. Gecikme cezası parçanın fiyatından daha yüksek olamaz.
9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
10. Ziyaretçi, kütüphanede olmayan bir kitabı ya da DVD'yi talep edebilir.
11. Ziyaretçi bir parça için sadece bir kez süre uzatma yapabilir. Parça için çok talep varsa, parçayı geri getirmelidir.

İlk sınıflarımız



Sınıflarımız Fazla. Gereksizler?

Kart sadece bir numara tutuyor.

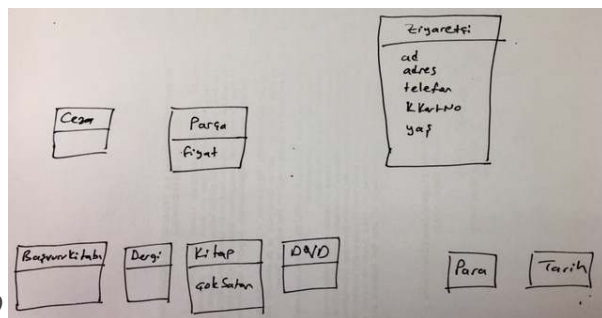
- Ziyaretçinin benzersiz özelliği olabilir.

Çocuk aslında bir ziyaretçi.

- Tek farkı yaşı ve bu nedenle 5 kitap alabilmesi.
- 13 yaşına gelince normal ziyaretçi olacak. O zaman tipini nasıl değiştirebiliriz??

ÇokSatın aslında bir kitap.

- Bir süre sonra normal kitap olacak. O zaman tipini nasıl değiştirebiliriz??

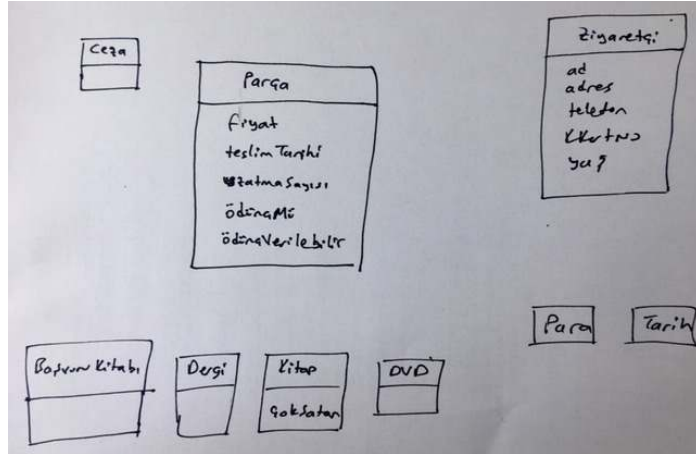


Özelliklerimizi ekleyelim

Teslim tarihi-gecikme cezası ?

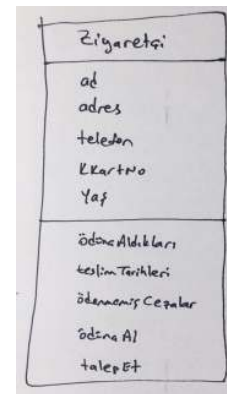
Süre uzatma sayısı?
İleride değişebilir.

Parçalarımızın ödünç alınıp alınmadığı?



Davranışları Ekleyelim

1. Her **ziyaretçi** **sistemde** olduğu sürece bir tekil **kütüphane kartı**na sahiptir.
2. **Kütüphane** her ziyaretçi için en az **ad, adres, telefon, kütüphane kartı numarasını** bilmelidir.
3. Kütüphane herhangi bir **zamanda** bir ziyaretçinin **ödünç aldığı parçaları, teslim tarihini, ödemediği gecikme cezalarını** **bilebilmeli** veya hesaplayabilmelidir.
4. **12 yaş altındaki çocuklar** için özel bir **kısıtlama** vardır. Bir kerede en fazla 5 parça ödünç alabilirler.
5. Ziyaretçi **kitap** veya **DVD** **ödünç alabilir**.
6. Kitaplar üç **hafta** için ödünç alınabilir. **Çok satan kitaplar** için **limit** iki haftadır.
7. DVD'ler iki hafta için ödünç alınabilir.
8. Gecikme cezası her parça için günlük on **TL**'dir. Gecikme cezası parçanın **fiyatından** daha yüksek olamaz.
9. Kütüphanede ödünç alınamayan **başvuru kitapları** ve **dergiler** de vardır.
10. Ziyaretçi, kütüphanede olmayan bir kitabı ya da DVD'yi **talep edebilir**.
11. Ziyaretçi bir parça için sadece bir kez **süre uzatma** yapabilir. Parça için çok talep varsa, parçayı geri getirmelidir.



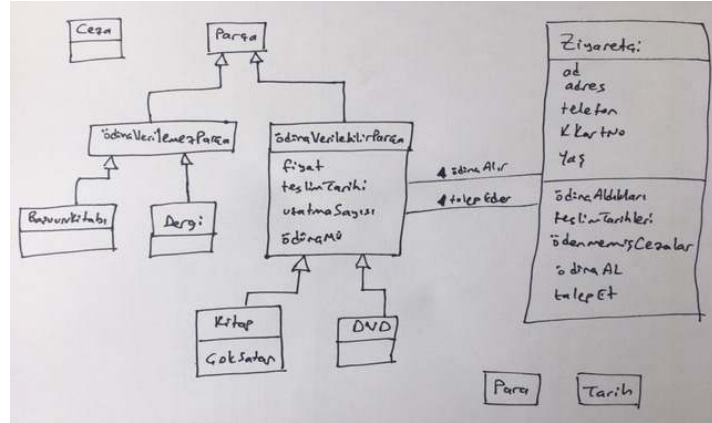
İlişkileri Ekleyelim

Ziyaretçi ile Parça arasında «ödünç Alma» ve «talep Etme» ilişkileri var.

Başvuru Kitabı, Dergi, Kitap, DVD hepsi özel birer parçadır.

- Başvuru Kitabı ve Dergi, ödünç verilemez parçalardır.
- Kitap ve DVD ödünç verilebilir parçalardır.
- Bunları hiyerarşik genişletirebilir ve «ödünç Verilebilir» özelliğini kaldırabiliriz.

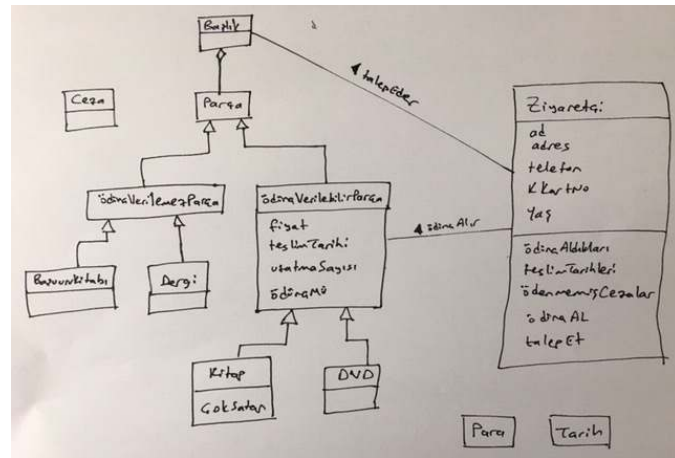
Dikkat! Bu soyutlama ile, «ödünç Alma» ve «talep Etme» ilişkileri artık «ödünç Verilebilir» sınıfına olmalıdır!



İlişkileri iyileştirelim

Ya aynı isimli beş kitap varsa ve birisi talep ederse?

İlla ki üçüncü kopyayı istiyorum diyemez.



Diyagramı iyileştirelim

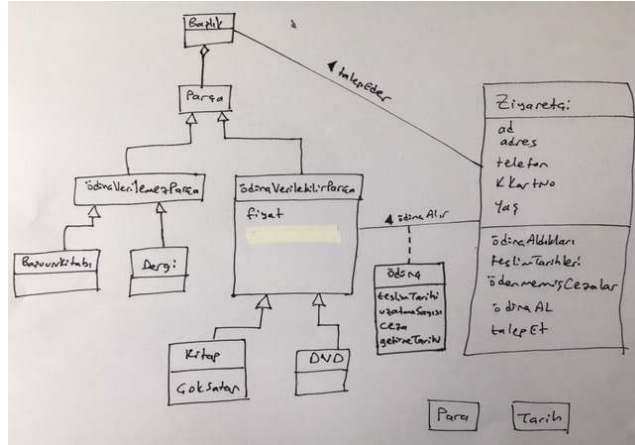
uzatmaSayısı ve ödünçMü özellikleri, aslında parçanın değil, ilişkinin özellikleri.

- Bir kitabı hayatı boyunca bir kez uzatmayız.

«ödünç» isimli bir ilişki sınıfı yaratıp özellikleri ona atarız.

Ceza da ödünç'ün bir özelliği olur.

Ziyaretçi kitabı geri getirecek, ya kitabı bırakır da cezayı ödemezse??



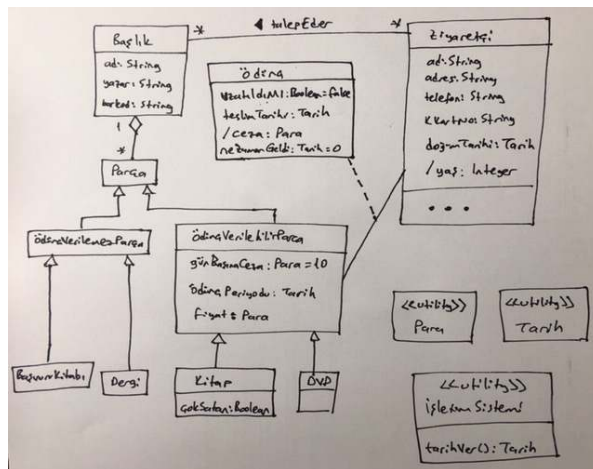
Son dokunuşlar

Yaş ve ceza, hesaplanan özelliklerimiz

Özelliklerin tiplerini ekleyelim.

Çoğullamaları ekleyelim.

Stereotipleri ekleyelim.



Biçimsel Belirtim

(FORMAL SPECIFICATION)

Belirtim (Specification)

Bir program tasarlarken ilk bilmek isteyeceğiniz şey programın ne yapacağıdır.

Bu tanımlara spesifikasyon adı verilir.

- Müşteriden
- Son kullanıcıdan
- Analiz takımından gelebilir.

Gayriresmi yazı, yapılı belge ya da biçimsel matematiksel ifadelerle tanımlı olabilir.

Bu derste matematik notasyonu ile kesin belirtme bakacağız.

İki aracımız var: FOL ve OCL

Matematiksel Mantık

Tanımlarımızda kullandığımız akıl yürütme ve ispatları nasıl resmileştirebiliriz?

Önerme Mantığı (Propositional Logic)

- Temel mantıksal bağlaçlar
- Doğruluk tabloları
- Mantıksal denklikler

İlk-Sıra Mantık (First-Order Logic)

- Çoklu nesnelerin özellikleri hakkında akıl yürütme

Önerme (Proposition)

Doğru ya da yanlış olan ifade

- Köpek yavruları kedi yavrularından daha sevimlidir.
- Kedi yavruları köpek yavrularından daha sevimlidir.
- Bu listedeki son maddedir.

Önerme olmayanlar:

- Komutlar
- Sorular
- Anlamsız/Muğlak ifadeler

Önerme mantığında her ifade, önerme bağlaçları (connectives) ile birleştirilen önerme değişkenlerinden (variables) oluşur.

- Genellikle küçük harfler kullanılır, her değişken doğru ya da yanlıştır.

Önerme Bağlaçları

Mantıksal Değil (NOT): $\neg p$

- $\neg p$ is true if and only if p is false.
- Negation

Mantıksal Ve (AND): $p \wedge q$

- $p \wedge q$ is true if both p and q are true.
- Conjunction

Mantıksal Veya (OR): $p \vee q$

- $p \vee q$ is true if at least one of p or q are true (inclusive OR).
- Disjunction

Çıkarım (Implication): $\rightarrow$

- $p \rightarrow q$ if p is true, q is true as well.
- $\neg(p \wedge \neg q)$

İki koşullu (Biconditional): $\leftrightarrow$

- $p \leftrightarrow q$ if and only if q
- p implies q and q implies p

Doğru (True): T

Yanlış (False): $\perp$

Önerme doğruluk tabloları (FOL)

($\neg P$) // Not P

($| P Q$) // P or Q

($\& P Q$) // P and Q

($\Rightarrow P Q$) // if P then Q or P implies Q

($\Leftrightarrow P Q$) // P if and only if Q; P is equivalent to Q

P Q | ($| P Q$) ($\& P Q$) ($\neg P$) ($\Rightarrow P Q$) ($\Leftrightarrow P Q$)

P	Q	($ P Q$)	($\& P Q$)	($\neg P$)	($\Rightarrow P Q$)	($\Leftrightarrow P Q$)
T	T	T	T	F	T	T
T	F	T	F	T	F	F
F	T	T	F	T	T	F
F	F	F	F	T	T	T

Operatör Öncelikleri

$\neg \wedge \vee \rightarrow$ 

$\neg$ takip eden değişkene yapışır.

$\wedge$ ve $\vee$, $\rightarrow$ 'dan daha sıkı bağlıdır.

$\neg x \rightarrow y \vee z \rightarrow x \vee y \wedge z$

$(\neg x) \rightarrow ((y \vee z) \rightarrow (x \vee (y \wedge z)))$

Örnek Önermeler

a: Bu akşam uyanık olacağım.

b: Bu akşam ay tutulmasını göreceğim.

«Eğer bu akşam uyanık olmazsam ay tutulmasını göremem.»

$\neg a \rightarrow \neg b$

a: Bu akşam uyanık olacağım.

b: Ay tutulması göreceğim.

c: Bu akşam ay tutulması var.

«Eğer bu akşam uyanık olursam, fakat ay tutulması yoksa, ay tutulmasını göremem.»

$a \wedge \neg c \rightarrow \neg b$

De Morgan Kanunu

$$\neg(p \wedge q) \equiv \neg p \vee \neg q$$

$$\neg(p \vee q) \equiv \neg p \wedge \neg q$$

if (!p() && q()) { /* ... */ } yerine,
if (!p() || !q()) { /* ... */ } daha hızlı olur.

$$p \rightarrow q \equiv \neg(p \wedge \neg q)$$

$$\neg(p \rightarrow q) \equiv p \wedge \neg q$$

$$p \rightarrow q \equiv \neg q \rightarrow \neg p$$

Birinci Derece Mantık (First-Order Logic, FOL)

Önerme mantığındaki bağlaçları canlandırarak nesnelerin özellikleri hakkında önermeler yapmamızı sağlar.

- ifadeler (predicates) ile nesnelerin özellikleri tanımlanır.
- fonksiyonlar (functions) ile nesneler eşlenir
- niceleyiciler (quantifiers) ile çoklu nesneler için önerme kurulur.

Likes(You, Eggs) $\wedge$ Likes(You, Tomato) $\rightarrow$ Likes(You, Menemen)

- you, eggs, tomato, menemen: sabitlerdir. önerme değil, nesnelerdir
- likes: ifadedir. nesneleri argüman olarak alıp, doğru/yanlış üretirler
- $\wedge$, $\rightarrow$ bağlaçlardır.

Eşitlik

FOL'de özel bir ifade vardır: =

- İki nesne birbirine eşittir.
- Sadece nesnelere uygulanır.
- Önermeler için $\leftrightarrow$

$\text{StarOf}(\text{FavoriteMovieOf}(\text{You})) = \text{StarOf}(\text{FavoriteMovieOf}(\text{Her}))$

- StarOf ve FavoriteMovieOf fonksiyondur. Nesneleri argüman olarak alır, **sonuçta nesne üretirler**.
- $\text{ColorOf}(\text{Money})$
- $\text{MedianOf}(x, y, z)$
- $x + y$

Nesneler ve ifadeler

Nesneler (gerçek şeyler) ve ifadeleri (doğru/yanlış) ayrı tutmaya dikkat etmemiz gerekir.

Nesnelere bağlaç uygulayamayız

- $\text{Venus} \rightarrow \text{The Sun}$ ✗

Fonksiyonlara önerme uygulayamayız

- $\text{StarOf}(\text{IsRed}(\text{Sun}) \wedge \text{IsGreen}(\text{Mars}))$ ✗

Niceleyiciler (Quantifiers)

Evrensel niceleyici (Universal quantifier)

- $\forall$ «Tüm değişkenler için bu ifade doğrudur»
- $\forall n. (n \in \mathbb{N} \rightarrow (\text{Even}(n) \leftrightarrow \text{Even}(n^2)))$
- For any natural number n , n is even if n^2 is even

Varoluşsal niceleyici (Existential quantifier)

- $\exists$ «Bazı değişkenler için bu ifade doğrudur»
- $\exists x. (\text{Even}(x) \wedge \text{Prime}(x))$
- $\exists x. (\text{TallerThan}(x, \text{me}) \wedge \text{LighterThan}(x, \text{me}))$

Değişkenler ve Niceleyiciler

Her niceleyicide tanıtilan değişken ve nicelenen ifade vardır.

Değişken sadece nicelenen ifade kapsamında geçerlidir.

$(\forall x. \text{Loves}(\text{You}, x)) \rightarrow (\forall y. \text{Loves}(y, \text{You}))$

--x burada yaşar-----y burada yaşar---

$(\forall x. \text{Loves}(\text{You}, x)) \rightarrow (\forall x. \text{Loves}(x, \text{You}))$

--x burada yaşar---- başka x burada yaşar--

Değişkenler ve Niceleyiciler

$\forall x. P(x) \vee R(x) \rightarrow Q(x)$ parantezlersek;

$((\forall x. P(x)) \vee R(x)) \rightarrow Q(x)$

- Sözdizimsel olarak yanlış! x , ifadenin yarısında geçerli ve tanımlı değil!
- Parantezleri düzgün kurgulamalıyız:

$\forall x. (P(x) \vee R(x) \rightarrow Q(x))$

İfadeleri Kurgulamak

All puppies are cute.

- $\forall x. (Puppy(x) \wedge Cute(x))$
 - Elimizde puppy dışında x 'ler için de doğru olması gerekir. İngilizce'si doğru olsa bile FOL ifadesi yanlıştır.
- $\forall x. (Puppy(x) \rightarrow Cute(x))$

All P's are Q's: $\forall x. (P(x) \rightarrow Q(x))$

Some blobfish is cute.

- $\exists x. (Blobfish(x) \rightarrow Cute(x))$
 - En azından bir örnek için doğru olmalıdır. İngilizce'si doğru olsa bile FOL ifadesi yanlıştır.
- $\exists x. (Blobfish(x) \wedge Cute(x))$

Some P is a Q: $\exists x. (P(x) \wedge Q(x))$

İfadeleri Kurgulamak

Person (p), Loves(x,y)

«Everybody loves someone else.»

$\forall p. (Person(p) \rightarrow \exists q. (Person(q) \wedge p \neq q \wedge Loves(p, q)))$

For every person there is some person who isn't them that they love.

«There is a person that everyone else loves.»

$\exists p. (Person(p) \wedge \forall q. (Person(q) \wedge p \neq q \rightarrow Loves(q, p)))$

There is some person who everyone who isn't them loves.

Niceleyici Sıralama

$\forall x. \exists y. P(x, y)$

For any choice x, there's some y where P(x, y) is true.

$\exists x. \forall y. P(x, y)$

There is some x where for any choice of y, we get that P(x, y) is true.

Niceleyicileri «Değillemek» (Negation)

	Ne zaman doğru?	Ne zaman yanlış?
$\forall x. P(x)$	Tüm x'ler için $P(x)$	$\exists x. \neg P(x)$
$\exists x. P(x)$	Bazı x'ler için $P(x)$	$\forall x. \neg P(x)$
$\forall x. \neg P(x)$	Tüm x'ler için $\neg P(x)$	$\exists x. P(x)$
$\exists x. \neg P(x)$	Bazı x'ler için $\neg P(x)$	$\forall x. P(x)$

- Değil'i niceleyicinin içine itin.
- Niceleyiciyi diğeri ile değiştirin.

Niceleyicileri «Değillemek» (Negation)

$\forall x. \exists y. \text{Loves}(x, y)$

"Everyone loves someone."

$\neg \forall x. \exists y. \text{Loves}(x, y)$

$\exists x. \neg \exists y. \text{Loves}(x, y)$

$\exists x. \forall y. \neg \text{Loves}(x, y)$

"There's someone who doesn't love anyone."

Küme İşlemleri

$\text{Set}(s)$: s bir kümedir.

$x \in y$: x , y 'nin bir elemanıdır.

«Empty set exists»

$\exists S. (\text{Set}(S) \wedge \neg \exists x. x \in S)$

$\exists S. (\text{Set}(S) \wedge \forall x. \neg(x \in S))$

«Two sets are equal if and only if they contain the same elements»

$\forall S. (\text{Set}(S) \rightarrow \forall T. (\text{Set}(T) \rightarrow (S = T \iff \forall x. (x \in S \iff x \in T))))$

Küme İşlemleri

$\forall x \in S. P(x)$

"for any element x of set S , $P(x)$ holds."

$\exists x \in S. P(x)$

"there is an element x of set S where $P(x)$ holds."

Birinci Derece Mantık (First Order Logic, FOL)

Matematiksel mantık ifadeleri

- Birinci derece mantık (FOL)
- İfadesel hesap (Predicate Calculus)

Gibi isimlerle bilinir.

Önergeleri mantıksal bağlaçlarla (ve, veya, değil) kesin olarak tanımlayabilmemizi sağlar.

Niceleyicilerin (değişkenlerin) belirli tanım alanları (domain) vardır. Mantık modeli bu alan için doğru ya da yanlış olur.

$Ali(a) \Rightarrow bakkal(a)$ öyle bir a vardır ki bu a 'nın adı Alidir ve bu a bakkaldır. Bu a nasıl bir a 'dır? Bütün a 'lar için geçerli midir? Hayır. Etki alanında geçerlidir. Bütün Ali'ler bakkal olurdu.

FOL - Sözdizimi

Cümleler (önergeler) basit ve karmaşık olabilir.

İfadelerin (predicates) ismi ve terim listesi vardır. Terimler virgül ile ayrılır

- evli(Ali, Ayşe)

Terimler değişken veya sabitleri ifade eder. Değişkenler için küçük harfler kullanılır.

Karmaşık ifadeler birli, ikili ya da çoklu olabilir.

- Birli: ! ve önerme : ! sıcak(bugün)
- İkili: &, |, $\Rightarrow$, $\Leftrightarrow$ ile bağlanan iki önerme. Önergeler parantezlenir.
- Çoklu: Çoklayıcı, değişken, önerme. «all(her)» $\forall$, «exists(öyle bazı)» $\exists$. $\forall(x, \text{yaşlıdır}(x, 50))$

FOL - Sözdizimi

$\forall(x, \forall(y, bla))$ ifadesi $\forall((x,y), bla)$ olarak kısaltılabilir.

Karmaşıklık yoksa parantez kullanılmayabilir.

Önergeler için tanımlar == ile yapılabilir.

- $P == \text{yaşlı}(\text{ali}, \text{veli}) \ \& \ \text{mutlu}(\text{veli}, \text{ali})$ (P önermesi ali veliden yaşlıdır ve veli aliden mutludur)

Tanımlar değişken isimleriyle parametrik yapılabilir.

- $Q(x, y) == \forall(z, (\text{yaşlı}(x, z) \ \& \ \text{yaşlı}(z, y)) \Rightarrow \text{yaşlı}(x, y))$

$P(V \% E) : V$ isim, E önerme ise, her V yerine E 'yi yerleştirir. (önergelerdeki isimlere sistematik değişiklik)

FOL - Yorumlama

İfadeler (predicates) «T» ve «F» üreten fonksiyonlardır.

Tekli ve ikili operatörler kendi anlamlarını taşır.

- ! not
- & and
- | or
- $\Rightarrow$ implies
- $\Leftrightarrow$ denklik

Çoklu ifadeler, nitelenen değişkenin tanımlandığı ve sabitlendiği bir etki alanı tanımlar. Bu etki alanlarını içiçe uzatabilirsiniz.

- Değişkenlerin ismi farklı olduğu sürece etki alanı tüm iç bölgeyi kapsar.
- Aynı isimli değişkenler var ise, dıştaki içeride etki etmez, iç etki alanı bittiğinde tekrar devam eder.

OCL

UML'in bir parçası

FOL için sözdizimi sağlar.

Bu sayede UML diyagramlarına açıklama ekleyebiliriz.

Örnek: Sıralama

Kesin spesifikasyonlarımızı nasıl yaptığımızı anlayabilmek için basit bir örnek olarak sıralamayı kullanacağız.

Herkes bir listeyi nasıl sıralayabileceğini bilir.

- Bilgisayar bilmez.
- Ona sıralama programı sağlamalısınız.

Örnek: Tamsayı vektörlerini sıralayabilecek bir program için spesifikasyon yazalım.

- Girdi argümanı X, tamsayı vektörü
- Çıktı argümanı Y, X'in artan sıralanmış hali

İlk olarak FOL kullanacağız

Sıralama

Y'nin, X'in sıralı hali olması için beklenen davranışı Türkçe tanımlayalım.

- X, tamsayı dizisi
- Y, tamsayı dizisi
- Sıralama artan şekilde

X tamsayı dizisi verildiğinde, Y tamsayı dizisini döndürürüz ve Y'deki her eleman bir sonrakinden daha küçüktür.

- Ya eşit elemanlar varsa? → küçük veya eşittir.
- Ya son eleman? Onun sonrası yok?

Girdiler/Çıktılar

İlk düşünmemiz gereken, spesifikasyon girdiyi tanımlıyor mu?

Bizde X isimli bir tamsayı vektörü denmişti.

- Böyle olmasaydı, programımızın vektör, elma, armut, kestane, muz hepsini anlamlı bir şekilde sıralaması gerekirdi.

Çıktımız da Y isimli bir tamsayı vektörü.

Sıralamanın artan yönde olmasını istiyoruz. Az önceki tanımda belirli konumlardaki değerlerin sonrakinden küçük veya eşit olmasını söyledik.

- Son eleman özel durum çünkü ondan sonra eleman yok.

Girdiye olan Duyarlılık

Önemli bir özelliğimiz daha var.

Çıktılarımızın içeriği, girdilerimizin içeriği ile birebir aynı ve tamsayı olmalıdır!

- 3, 2, 1 girdi, 4, 5, 6 çıktı diyelim. Çıktı önceki tanımımıza uyuyor?

"Çıktı Y vektöründeki her eleman, girdi X vektöründe de yer almalıdır."

- Peki ya 3, 2, 1 girdiğinde, 1, 1, 1, 1, 2, 3 çıkarsa?

"Çıktı Y vektöründeki her eleman, girdi X vektöründe aynı sayıda yer almalıdır."

- Peki ya 3, 2, 1 girdiğinde 1, 2, 3, 4 çıkarsa?

"Çıktı Y vektöründeki her eleman, girdi X vektöründe aynı sayıda yer almalıdır ve yeni eleman yer almamalıdır."

Hani kısa ve kesin tanım yapacaktık? Sıralama çok basit bir problem. Ya askeri bir yazılımda güvenlik için program yazıyorsak? Kesin ve net tanım yapabilmeliyiz.

Döngüsellik (Circularity)

Bir kavramı belirlerken, kavramın tanımda tanımlanmasına çalışırız.

Sıralamayı sıralama terimleri ile ifade etmeye çalıştık.

Bu duruma döngüsellik denir ve bizi problemi anlama konusunda ileriye taşımaz.

Tanımımıza tekrar bakalım:

Bir X tamsayı vektörü verildiğinde, Y tamsayı vektörü üretilir. Y vektörü sıralı olmalıdır (her eleman sonrakinden [küçüktür|büyüktür]). Y vektörünün içeriği, X vektörünün içeriği ile aynı olmalıdır (Y vektöründeki her eleman X'ten gelmelidir). Y'deki her bir elemanın tekrar sayısı, X'teki tekrar sayısıyla aynı olmalıdır.

Şimdi bu problemi kesin matematik dili ile modelleyelim.

Biçimsel Belirtim

Matematiksel tanım kurgusunda üç aşama vardır.

1. İmza (Signature)
2. Önkoşul (Precondition)
3. Sonkoşul (Postcondition)

İmza (Signature)

Programın adı, girdi argümanlarının adları ve tipleri, çıktılarının adları ve tipleri tanımlanır.

```
Vector<int> Y = SORT(Vector<int> X)
```

- Programın tek girdisi X, tipi tamsayı vektörü, tek çıktısı Y tamsayı vektörü.

Değişkenlere (X ve Y) gibi kesin isimler verilir. Bunlara ön ve son koşullarda atıf yapacağız. (Java'da değişken tanımlamanın aynısı)

Sıralama programımızı, "sıralamaya uygun basit veri tipi olması koşulu ile", başka veri türlerine de uygulayabiliriz.

- Sıralamaya uygunluk: >, <, >=, <=, = gibi sıralama ifadelerinde yer alabilme
- Tamsayılar, gerçek sayılar, stringler, ...

ÖnKoşullar

Önkoşullar, fonksiyonun girdi argümanları hakkındaki savlardır.

Parametreler alan bir fonksiyonu kodlarken ilk yapılan, girdilerin beklenen girdiler olup olmadığının kontrolüdür.

Biçimsel belirtimimiz ile bu kontrol koşullarının neler olduğunu belirleriz.

Başka bir örnek: `Real Y = SQRT (Real X)`

Önkoşul, X'in negatif olmamasıdır. $x \geq 0$

- Kompleks sayılarımız olsaydı, bu önkoşul olmayacaktı.
- X negatif ise olanları tanımlamıyoruz. Sadece fonksiyonun davranışını tanımlıyoruz.
- Herhangi bir girdiyi alıp, uygun olmayanlarda hata/exception üreten bir versiyon da tanımlayabilirdik.

Sıralama için Önkoşullar

Ön/son koşullarda veri tipleri ile ilgilenmeyiz. Onu imza kısmında hallettik.

X vektörü boş ise?

- Y vektörü boş döner.

$X > 0$ yapabiliriz. Ama daha genel olması için boş kümeyi de kapsamalı.

Öyleyse?

Sıralama için ön koşul yok. Ya da "True" (her koşulda çalışır)

Sonkoşullar

Fonksiyon tarafından üretilen çıktı hakkında neyin mutlaka doğru olması gerektiğini söyler.

Tipik olarak, çıktının girdi ile nasıl ilişkili olduğunu anlatır.

SQRT fonksiyonumuz için son koşul ne olabilir?

- $Y = \text{SQRT}(X)$ dersek, döngüsel tanım yaratmış oluruz.

Aralarındaki sayısal ilişkiyi tanımlamalıyız:

- $Y * Y = X$
- Bu, karekök çalıştıktan sonra doğru olması gereken ifadedir. Buna son koşul diyoruz.

Sonkoşullar

Şimdiye kadar saf fonksiyonlar ile ilgilendik.

- Saf fonksiyonlarda çıktı tamamen girdi tarafından belirlenir.

Programlama dillerinde, fonksiyon, yordam, metod gibi hesap birimleri saf olmayabilir.

- Son koşulları yazarken, çıktının girdiye nasıl bağlı olduğunun yanında, olası yan etkileri (side effect) de belirtmeliyiz.
- Yan etkiler:
 - Global değişkenlerde yapılan değişiklikler
 - Fonksiyonun girdi ya da çıktısında sonuçları gösterilmeyen işlemler
 - Nesneye yönelik dillerde çalıştığımız nesnenin bir örneğinde olan herhangi bir değişiklik

Sıralama için Sonkoşullar

Doğal dil ile nasıl tanımlamıştık, onu FOL ile nasıl gösterebiliriz?

Veri tipleri ile ilgili ifadeleri kaldırdığımızda:

Çıktı vektörü Y sıralı olmalıdır.

Y 'nin içeriği, X 'in içeriği ile aynı olmalıdır.

- X 'teki herşey Y 'de olmalıdır.
- Y 'deki herşey X 'ten gelmelidir.
- Y 'deki her bir elemanın tekrar sayısı X 'teki tekrar sayısı ile aynı olmalıdır.

Sıralama için Sonkoşullar

İki parçada halletmemiz gerekir.

İlk kısım, sıralı olması. Y tamsayı vektörünün sıralı olması ne demektir?

« Y 'deki **her bir** eleman için, **eğer** kendisinden sonra gelen bir eleman varsa, o eleman baktığımız elemandan daha büyük ya da eşit olmalıdır»

- Son eleman için hiçbir şey söylemedik.
- İfade tüm elemanlar için doğru ise, son eleman zaten en büyük eleman olmalıdır.
- Her bir tanımlayıcısı için, bir indeks değişkeni de kullanacağız. i , $i+1$

Sıralama için Sonkoşullar

İkinci kısım, eleman sayısı. Y tamsayı vektöründe kaç eleman var?

- n gibi bir değişken ile belirleyebiliriz. !! N negatif olmamalı !! i , $1..n$ arasında değer alır.
- Tanımımıza göre, $1..n-1$ arasındaki değerler için, $i+1$ 'deki elemanın büyük ya da eşit olmasını söylüyoruz.
- $N=0$ ise?? $N-1$ negatiftir?? $i:=1..n-1$, n 0 olduğu için aslında hiç «i» olmayacak. Saçma ama ifade doğru ☺ Son koşulun doğruluğunu değiştirmiyor.

$\forall i: 1 \leq i \leq (n-1), y[i] \leq y[i+1]$

Bu y 'ler nelerdir ve girdiye nasıl bağlıdır söylemedik. Sonra bunları da belirteceğiz.

OCL de aslında FOL üzerinde yer alan farklı bir sözdizimidir. Burada çok az değinip, sonra FOL gibi bir ders yapacağız.

Sıralı için önkoşul?

Bool $Y = \text{ORDERED}(\text{Vector} \langle \text{int} \rangle X)$

Tip kontrolü yapmıyoruz, bu nedenle hangi X gelirse gelsin sıralı mı diye sorabiliriz.

X boş bir vektörse?

- Amacımız kodlamaya yol göstermek. Kullanıcı karşılaşmadan durumu ele alabilmek.
- Doğal olarak sıralıdır diyebiliriz. İfademiz de boş vektör için geçerlidir.

Son koşul nedir?

Sıralı için Sonkoşul

$\forall i: 1 \leq i < |X| \bullet X[i] \leq X[i+1]$

Y'deki baştan sondan bir öncekine kadar tüm pozisyonlar için, vektörün o pozisyonunda saklanan değer, sonraki pozisyonda saklanandan büyük değildir.

$\forall$: belirteç

i: indeks değişkeni

|...|: vektörün boyutu

•: Durum mutlaka böyle olmalı (it must be the case)

TersSıralı için İmza, ilkkoşul, Sonkoşul?

Bool Y = TersSıralı (Vector <int> X)

Önkoşul: True

Sonkoşul: $\forall i: 1 \leq i < |X| \bullet X[i] \geq X[i+1]$

Sıralama için Sonkoşullar

Sıralı olması kısmını hallettik.

Şimdi, ikinci kısım: Y'nin X ile aynı elemanlardan oluşması (Ve tabii ki aynı sayıda tekrar içermesi).

İmza: $\text{Bool } Z = \text{AynıElemanlardanOluşur}(\text{Vector } \langle \text{int} \rangle X, \text{Vector } \langle \text{int} \rangle Y)$

Önkoşul: İki vektörün aynı uzunlukta olması $|X| = |Y|$

Sonkoşul: Üç ifademizi de FOL ile tanımlamalıyız:

- X'teki herşey Y'de olmalıdır.
- Y'deki herşey X'ten gelmelidir.
- Y'deki her bir elemanın tekrar sayısı X'teki tekrar sayısı ile aynı olmalıdır.

Ya da, ...

Permütasyon

Permütasyonun tanımı: X'in elemanları, Y'nin elemanlarının bir permütasyonu ise; X, Y ile aynı elemanlara sahiptir.

Permütasyon iyi tanımlanmış bir matematiksel özellik olduğu için, kendimiz yazmak zorunda kalmayız.

$\text{Bool } Z = \text{Permütasyon}(\text{Vector } \langle \text{int} \rangle X, \text{Vector } \langle \text{int} \rangle Y)$

Son koşulları biraz hileli. Özel durumlara bölüp her durumu ayrı ele alırız.

- 1. durum: iki vektör de boş: $|X|=0 \Rightarrow \text{Permütasyon}(X, Y)$
- 2. durum: Vektörler boş değil. İlk elemanları aynı mı değil mi?
- 2.1: Aynı: X ve Y'nin kalan kısımları permütasyonu mu? $|X| > 0 \wedge X[1] = Y[1] \wedge \text{Permütasyon}(\text{kalan}(X), \text{kalan}(Y)) \Rightarrow \text{Permütasyon}(X, Y)$ (özyinelemeli. Kalan her zaman daha küçük ve sıfır uzunluk durumunu ele aldık.)

Permütasyon

- 2.2: İlk elemanları Farklı: Y vektörünü, $X[1]$ 'in geçtiği yere göre üç parçaya böleriz.
- Ör. $X[1]=5$ ise, Y'nin bir yerinde 5 olmalı. Yoksa, permütasyonu değildir.
- J. Pozisyonda olsun, $j > 1$ ve $j \leq |Y|$ olmalı.
- Y'nin birinci parçası, 1.den j.ye (j dahil değil),
- Y'nin ikinci parçası, j. Eleman
- Y'nin üçüncü parçası, j+1'de sona kadar kalan elemanlar

Üçüne de bakarak permütasyon kararını veririz.

$\exists j: 1 < j \leq |Y| \bullet (X[1] = Y[j])$

Kalan iki (1. ve 3.) parça için ifadeleri yapıştırırız: $Y[1..j-1] \circ Y[j+1..|Y|]$ (concat)

Permütasyon(kalan(X), $(Y[1..j-1] \circ Y[j+1..|Y|])$)

X'in kalan kısmı ile, Y'nin j.pozisyon harici kısmı birbirinin permütasyonu mu? Yine rekürsif. Boyutları azalttığımız için, yine tüme varabiliriz.

Permütasyon için Sonkoşul

Parça parça ilerledik, şimdi:

Permütasyon için sonkoşul:

Permütasyon(X,Y) $\Leftrightarrow$

$|X|=0 \vee$

$(|X| > 0 \Rightarrow$

$(X[1] = Y[1] \wedge \text{Permütasyon(kalan(X), kalan(Y)) }) \vee$

$(X[1] \neq Y[1] \wedge$

$(\exists j: 1 < j \leq |Y| \bullet (X[1] = Y[j]) \wedge$

$\text{Permütasyon(kalan(X), (Y[1..j-1] \circ Y[j+1..|Y|])) }) \vee$

Kontrol edelim

X boş vektör ise belirtim ne söylüyor?

- Y de boş vektördür, ikisi aynı elemanlara sahiptir.

X'in ilk elemanı, Y'de birden fazla yerde geçerse?

- Tanımımız ile bunu da karşılayabiliriz.
- Geçtiği yerleri bulabiliyoruz. Rekürsif olarak kalan parçalara bakıyoruz. Her bulduğumuzda yeni özyineleme yaratırız. Boş vektör karşılaştırmasına kadar ineriz.

OCL

Buraya kadar FOL ile belirtim yaptık.

UML'de FOL'un bir başka hali olan OCL bulunur.

Sıralı() için OCL belirtimi:

context Vector::Sıralı() : Boolean

- Metodun argümanı yok. Tüm nesneye dayalı dillerdeki gibi mesajın yollandığı nesne belirtilen argüman olarak hizmet eder.

pre: true

- True ise tanımlamaya gerek yok.

post: self -> forAll (i:Integer | i > 0

and i < self.length implies

self.at[i] <= self.at[i+1]

OCL - FOL farkları:

| sembolü değişkeni önermeden ayırmak için kullanılır.

«i» değişkeni için kısıtlar önermenin parçası olarak yer alır.

«i» değişkeni için kısıtları «implies» anahtar kelimesi ile belirtir.

OCL'de vector yoktur, sequence vardır, ama aslında ikisi de aynı şeydir.

Sonuç

Bazen, bir sistemin gerektirdiği işlevselliği kesin olarak tanımlamamız gerekir.

Bu tanımlamaları yapmak için çeşitli biçimsel diller ve araçlar bulunmaktadır.

Bunları kullanabilmek için, tam olarak ne söylemeye çalıştığınız hakkında çok sıkı düşünmeniz gereklidir.

Bu efor, sizi anlaşılmayan gereksinimler yüzünden çokça tekrar çalışmadan kurtarabilir.

OCL

OCL: UML'in resmi bir parçası

Şimdiye kadar hep diyagramlara baktık.

- Ama diyagramlar herşeyi anlatamaz.
- Tanımlarda, tasarımda daha detay gerektiren noktalar vardır --> OCL

OCL bir belirtim (spesifikasyon) dilidir. Sistem özelliklerinin işlevsel detaylarını tanımlamanızı sağlar.

- Bildiren (declarative: Yapılacak kodu ve adımları açıkça belirtmeden sonuçları tanımlar)
- Güçlü tipli (Strongly typed: tüm değişkenler belli ve bir tipi diğeri yerine kullanamazsınız)
- Kısıtlar, koleksiyon sınıfları, diyagramlardaki sınıflar/ilişkiler arası gezinim mekanizması

Pek çok araç OCL'i destekler

- Rational Rose, ArgoUML, Eclipse, Poseidon/Octopus, Enterprise Architect, ...

OCL'e neden ihtiyaç duyarız?

Diyagramlarla yapısal ilişkileri ve davranış tanımlarını çok iyi tanımlayabiliriz.

Kesin anlamsal detayları tanımlayacak bir mekanizma diyagramlarda yoktur.

- Anlamsal: Bu bileşenin bu görevi yapması ne demektir?

OCL, UML'i şunlarla genişletir:

- Değişmeyen sınıf yapısı
- İşlemlerin ön ve son koşullarının kesin tanımları
- Durum diyagramlarında geçişlerin kontrolü

OCL: Genel Bakış

Declarative (bildiren): Prosedürel değil, programlama dili değil.

Tamamen ifade dili. Aktiviteleri değil, değerleri tanımlar.

Atama ifadesi yok. = simgesi ile bildiri, kısıt, özellik ifade edilir.

Strongly-typed (güçlü tipli): Kendi veri tipleri ve UML diyagramlarındaki veri tipleri vardır.

Sadece bir anahtar konsept vardır: Kısıt (constraint) = Sistem özelliklerinin biçimsel bildirimi.

Yapabilecekleriniz:

- Sınıflarda değişmeyecekleri tanımlayabilirsiniz.
- İşlemlerde ön ve son koşulları tanımlayabilirsiniz.
- Türetilen özellikler için türetme kurallarını tanımlayabilirsiniz.
- Durum diyagramlarında geçişler için kontrol (guard) tanımlayabilirsiniz.
- Mesajlar için hedef tanımlayabilirsiniz.
- Stereotipler için tip sabitleri (type invariant) tanımlayabilirsiniz.
- Bir sorgu dili olarak, sınıf diyagramında türetilen nesneleri belirli kriterlere göre sorgulayabilirsiniz.

OCL Sözdizimi

Dilde sadece tek bir ifade vardır: Kısıt

```
context <identifier> <constraintType>: <Boolean expression>
```

Kısıtta anahtar kelimeler ve ifadeler yer alır.

- İlk anahtar kelime: «context» Context, diyagramda nerede olduğumuzu gösterir.
- Sonra tanımlayıcı <identifier>: context'e ismini verir. Genellikle bir sınıfın adıdır. Belirli bir metodun adı da olabilir.
- İkinci anahtar kelime: Kısıtın tipi: (Invariance, Precondition, Postcondition)
- Sonra mantıksal ifade: İfadenin anlattığı kısıtın ne olduğu.

Kısıtlar UML sınıf modeli ile birebir ilişkilidir. Modeli oluşturmuş olmamız ve detay bilgileri OCL ile vermemiz beklenir.

OCL Değişmezleri (Invariant)

Bir özelliğin her zaman doğru olan ifadesi.

Sistemin anahtar bir gereksiniminin tanımı olarak düşünebiliriz.

- Ör. Sistemdeki nesnelerin değerleri arasında gerekli bir ilişki.

Anahtar kelime: «inv»

```
context BüyükŞirket
  inv: çalışanSayısı > 50
```

- BüyükŞirket sınıfında çalışanSayısı özelliği 50'den büyük olmalıdır.

Veritabanındaki veri bütünlüğü kısıtlarına (integrity constraints) benzer.

- İşçi şirkette çalışır. --> Şirket iflas etti, veritabanından silindi. İşçi nerede çalışır??
- Programlamadaki avare işaretçi (dangling pointer) durumu oluştu. Veritabanı yönetim sistemi bu durumu engeller.
- OCL değişmez kısıtları da sistem tanımlamalarında aynı rolü üstlenir.

Ön ve Son Koşullar

UML'de işlemlerin anlamını tanımlar. Bir sınıfa ait işlemi kullanmak ne anlama geliyor?

pre: İşlemin anlamlı bir şekilde çalışması için ne doğru olmalı?

- Belirli bir işlemin gerçekleşmesine izin verilen koşulları belirtir

post: İşlem tamamlandıktan sonra ne garanti ediliyor?

- Belirli bir işlemin çalışmasının sonuçlarını belirtir.
- Dönüş değeri ile girdi parametreleri arasındaki ilişkiyi tanımlar.
- İşlemin tetiklenmesi ile sınıfta oluşacak özellik değişimlerini tanımlar.

Bir kısıtta sadece birini ya da ikisini birden tanımlayabiliriz.

Ayrı kısıtların birinde pre, birinde post da tanımlayabiliriz.

Ön ve Son Koşullar: Karekök

Ön koşul: Karekök rutininin argümanı negatif sayı olmamalıdır.

Son koşul: Hesaplanan sonucun karesi, argümana eşit olmalıdır.

Biraz tersine düşünme gibi olsa da, eşitliğin doğru ifadesidir. Karekökün böyle davranması gerekir.

OCL:

```
context Real::squareRoot() : Real
pre: self >= 0
post: self = result * result
      and result >= 0
```

Özellik Değerlerinde Değişiklik

Karekök örneğinde bir fonksiyonun hesaplanması ile sonuçların özellikleri tanımlanmıştı.

Belirli bir işlemin etkisi olarak özellik değerlerinde değişiklik de olabilir. Son koşullar ile bunu da tanımlayabiliriz.

Örnek: Banka Hesabı. Bakiye özelliği var. Para yatırma ve çekme işlemleri var. Para yattığında bakiye güncellenmeli.

```
context Account::deposit(Real : amount)
```

```
pre: amount > 0
```

```
post: balance = balance + amount
```

Dikkat: OCL bildirimsel bir dildir. Buradaki = işareti atama değil, eşitliktir!! İfade yanlış olur. Değer değiştiriyorsak, @pre kullanmalıyız. → işlem çalışmadan önceki değer.

```
post: balance = balance@pre + amount
```

Özellik Değerlerinde Değişiklik

a ve b isimli Real tipinde iki özelliği olan bir sınıfımız olsun.

Sınıfta swap isimli bir işlem var, a ve b'nin değerlerini takas ediyor.

Son koşul ile nasıl ifade ederiz?

```
post : _____ = _____ and _____ = _____
```

```
post : a = b@pre and b = a@pre
```

Bir programlama dilinde ihtiyacımız olan geçici değişkene burada gerek yok ☺

OCL Mevcut Basit Veri Tipleri

Tip	Değerler	İşlemler
Boolean	True, false	and, or, xor, not, implies, if-then-else
Integer	1, -5, 2, 34, 26524, ...	*, +, -, /, abs(), ...
Real	1.2, 3.14, -45.678, ...	*, +, -, /, floor(), ...
String	'merhaba dünya'	toUpper(), concat()

OCL Anahtar Kelimeleri

OCL dilinde küçük bir anahtar kelime kümesi vardır	Anahtar Kelime	Tanım
Bunları gördük	inv, pre, post	Kısıtları tanımlar
Koşullu ifadeler için	if-then-else-endif	Koşul ifadesi
Mantıksal operatörler	not, or, and, xor, implies	Mantıksal operatörler
UML'in birimleri bölütleme yeteneğini gösteren paketleme	package, endpackage	Paketler
Bunu da gördük	context	Kısıtları tanımlar
Tanım yapmamızı sağlayan çeşitli kelimeler	def	Global tanım
Bunlarla kısa hallerini kullanabiliriz	let, in	Lokal tanım
hesaplanan özellik değerini tanımlayabiliriz	derive	Özellik türetme
ilk değer atamasını init ile yapabiliriz	init	İlk değer tanımı
Bunu da gördük	result	İşlemin sonuç değeri
Bunu da gördük	self	Kısıtlanan nesnenin kendisi

"Let" İfadesi

Yerel kısaltma/tanım yapabilmemizi sağlar.

Kısıtlarınızın birinde, birden fazla defa kullanacağınız karmaşık bir hesaplama olsun.

- Her seferinde tekrar yazabiliriz ama hata olasılığı yüksektir.

Let ifadesi ile ifadenin değerini taşıyan yeni bir tanımlayıcı türetebilir ve bunu kısıtlarımızda kullanabiliriz.

Ör: income, işlerimizdeki tüm maaşlarımızın toplamı olsun.

```
let income: Integer = self.job.salary->sum() in
    if isUnemployed then income < 100
    else income >= 100
endif
```

income ifadesini koşulun içinde tekrar tekrar yazabilirdik.

Gezinti

En başta ne demiştik?

"Kısıtlar, koleksiyon sınıfları, diyagramlardaki sınıflar/ilişkiler arası gezinim mekanizması"

OCL'in tipik olarak belirli bir sınıf diyagramı ile ilişkili olduğunu söylemiştik.

- Her bir kısıt ifadesi belirli bir sınıf veya işlem ile başlıyordu.

Kısıtlarımızda sadece context sınıfını değil, ilişkili başka sınıfların da özelliklerini değiştirebilmeliyiz.

- Bir OCL değişiminde non-context sınıfın özelliklerine referans vermek için nitelikli isim kullanmalıyız

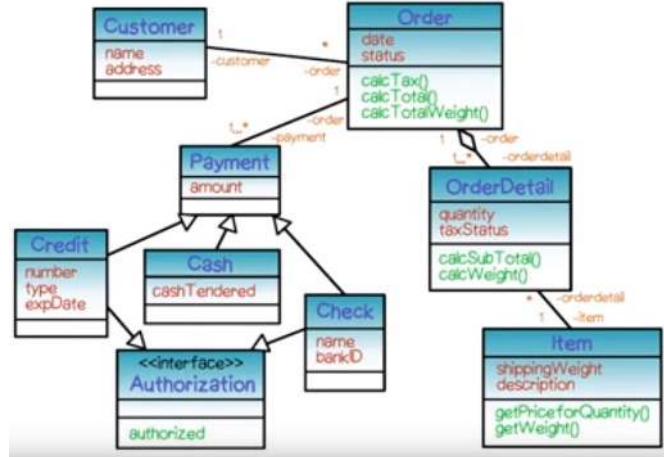
OCL'deki Gezinti (navigation) mekanizması ile diyagramda yürüyebiliriz ve her adımda, nokta ile ilişkideki takip eden sınıfı ya da ilişkiyi ekleyebiliriz.

Gezinti Örneği

Customer sınıfı bizim contextimiz olsun.

Bir siparişin hangi tarihte verildiği ile ilgili bir kısıt yazalım.

`self.order.date` ile erişebiliriz.



Gezinti Çoğullama

Sınıf diyagramlarında sayı ve * ile ilişkilerin çoğullamalarını (bir sınıftan kaç örnek diğer sınıftan kaç örnek ile ilişkiye geçecek) verebiliyorduk.

1 - 1: gelin/damat 1 - n: anne/çocuk m - n: öğrenci/ders

UML'de çoğullama kullanıldığında, gezintinin sonucu tekil bir nesne değil, bir koleksiyondur. Küme, yığın, sıralı dizi olabilir.

OCL'de bu koleksiyonlar üzerinde işlem yapabiliriz.

-> sembolü ile koleksiyonun adını işlemin adından ayırırız.

Önceki örnekte ödeme birden fazla bankanın çeki ile yapılsın. Sayı nasıl bulunur? (koleksiyonda `size()` işlemi vardır)

Customer nesnesindeyiz:

```
self.Order.Check.bankID->size()
```

Koleksiyonlar

Kısıtlara ve gezintiye baktık. OCL'deki üçüncü ana elemanı koleksiyon sınıfları.

OCL'de 1-n ve n-m tipinde ilişkileri destekleyen dört çeşit koleksiyon mevcuttur:

Koleksiyon (Collection)

- Küme (set)
- Yığın (bag)
- Sıralı dizi (sequence)

Koleksiyon sınıfının işlemleri:

- Size, includes, count, includesAll, isEmpty, notEmpty, sum, exists, forAll, iterate
- Alt sınıflar bunlar miras alır. Bunun dışında özelleşmiş işlemleri de olabilir.
- OCL referans manuelinde hepsini bulabilirsiniz.

Diğer OCL Özellikleri

Kısıtlar, gezinti ve koleksiyonlar dışında OCL'in daha az kullanılan özellikleri

Tuple: (Değişkenler grubu) Programlama dillerindeki struct/record tarzı yapılar.

Enumeration: (Sayım) Java ve diğer dillerdekiler gibi.

Message Expression: (Mesaj ifadesi) bir işaretin gönderildiğinin belirtilmesi ya da bir işlemin tetiklenmesi

UML Metamodel Erişimi

Automatic Flattening: (otomatik düzleştirme): Gezintide, üzerinde pek çok bileşeni olan iki ilişki olsun (kümeler kümesi, kümeler yığını gibi).

- OCL'de iki seviyeyi de sözdizimimizde belirtmemize gerek yoktur.

OCL - Son sözler

Görece kolay bir dil

CASE araçları destek veriyor

Sistemimizin özelliklerini kesin olarak tanımlayabilmemize olanak sağlıyor

Yapısal ve davranışsal diyagramların tamamlayıcısıdır.

OCL ile istediğiniz kadar kesin tanım yapabilirsiniz.

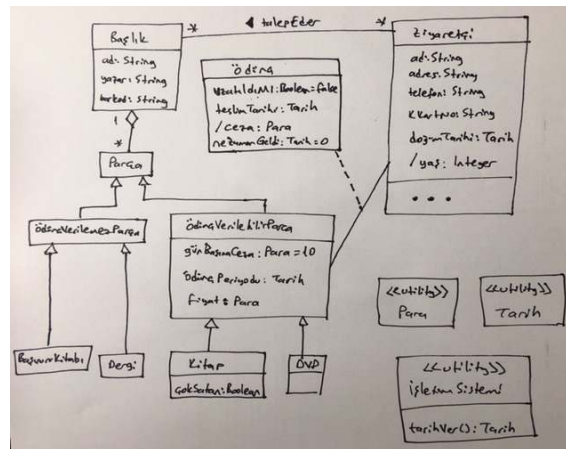
OCL - Kütüphane Örneğimiz

UML DİYAGRAMI İLE AÇIKLAYAMADIĞIMIZ KISIMLAR
İÇİN OCL'İ KULLANALIM

Ali'nin Gereksinimleri

1. Her ziyaretçi sistemde olduğu sürece bir tekil kütüphane kartına sahiptir.
2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
4. 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.
5. Ziyaretçi kitap veya DVD ödünç alabilir.
6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.
7. DVD'ler iki hafta için ödünç alınabilir.
8. Gecikme cezası her parça için günlük on TL'dir. Gecikme cezası parçanın fiyatından daha yüksek olamaz.
9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
10. Ziyaretçi, kütüphanede olmayan bir kitabı ya da DVD'yi talep edebilir.
11. Ziyaretçi bir parça için sadece bir kez süre uzatma yapabilir. Parça için çok talep varsa, parçayı geri getirmelidir.

Sınıf Modelimiz



Gereksinimlerimiz yeterince aydınlatıldı mı?

2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
 - Gereksinimler bilgilerin varlığı gibi ise, bunlar diyagramda (özelliklerde) yeterince anlatılır.
 3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
 - Burada bilgiyi üretmek için sorgular yapılmalı, Bir işlem varsa, diyagramda bunun imzasını verebiliriz. Hesaplamayı OCL ile tanımlarız.
 9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
 - Burada da bilgi var ve genelleme ile tüm bilgiyi sağladık.
 5. Ziyaretçi kitap veya DVD ödünç alabilir.
 - Burada da bilgi var ama nasıl ödünç alınamacağını anlatamıyoruz. Ödünç alma işleminin detaylarını vermedik. Örneğin çocuk ise limit var vs.
- Sınırlama: Ya diyagramdaki tanım desteklenmeli, ya da yeni bir tanım getirilmelidir.

6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.

- İki hafta - Üç hafta → kesin sayılar. Belirgin sayılar varsa, bir kontrol olmalıdır.
- Bir kontrol varsa, bunu tanımlamalıyız.
- OCL ile diyagramda tanımladığımız elemanlara bu gibi gereksinimleri ekleyebiliriz.
 - FOL ile güçleri denk olduğu için, sistemin işlevsel gereksinimlerini istenilen detaya kadar tanımlayabilmemizi sağlar.
- İlk adım: diyagramdaki hangi nesne OCL ifadesini yerleştirmek için uygundur?
- Context: OCL'i tanımlayacağımız sınıf ya da metodun adı
 - Bu gereksinim için context: Kitap
- İkinci adım: Kısıt sistemle mi, belirli bir sınıfla mı ya da belirli bir işlemle mi ilgili?
- Sistem veya sınıf ile ilgili ise invariant (sınıf/sistemle ilgili ne doğru olmalı),
 - İşlemle ilgili ise ön ve son koşullar.
 - Bu gereksinim için kısıt: invariant (diyagram gibi tek kesin bir cevabı yoktur. ödünçAl işlemine sonkoşul olarak da yazabilirdik. Limit hep olduğu için bizimki daha mantıklı)

Gereksinim 6: OCL

Kullanacağımız sınıfı ve kısıt tipini belirledikten sonra, kısıtı OCL ile yazabiliriz:

```
context Kitap inv:
    if cokSatan then
        oduncPeriyodu = 2 --hafta
    else
        oduncPeriyodu = 3
    endif
```

Koşullu ifade, programlama dillerindeki deyim gibi değildir, Durum değişikliği yerine, değer üretir. İki değer olasılığı var ve ikili ifadenin (çokSatan) değerine göre, oduncPeriyodu ya 2, ya 3'tür.

Gereksinim 6: OCL

Yazdığımız ifade tek bir değişmez (invariant) kısıt (sistemin bir özelliğinin hep alması gereken bir değer) tanımlar.

Burada 2 ve 3 rakamlarını açıkça kullandık tarih olduklarını söylemedik. Ancak diyagramdan Tarih tipinde olduklarını biliyoruz.

- Türlerin uyummasını sağlamalıyız.
- Tarih sınıfımız var, düzgün şekilde kullanmalıyız.

Her OCL kısıtı belirli bir sınıfın kapsamında yorumlanır.

- Noktasız gördüğümüz tüm ifadeler sınıfın kapsamındadır (özellikler, işlemler)
- Diyagramda başka kapsamdaki sınıfları açıkça isimleriyle kullanabiliriz.
-

Gereksinim 7: DVD'ler iki hafta için ödünç alınabilir.

context DVD

inv: oduncPeriyodu = 2

Burada şartlı ifadeye de gerek yok.

Değişmezler, özelliklerin alacakları değerleri belirlemek için faydalıdır.

Gereksinim 3: Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.

Burada sorgu işlemleri var. Hiçbir şeyi değiştirmeden özelliklerin değerlerini sorgulayan işlemler.

OCL'de işlemleri ilk ve son koşul kısıtları ile tanımlayabiliriz.

İlk kısmına odaklanalım. Ziyaretçinin ödünç aldığı parçalar nelerdir?

Diyagramda Ziyaretçi'de ödünç Aldıkları davranışı eklemiştik.

- Bu davranışın geri dönüşü olarak sadece belirli ziyaretçinin ödünç aldığı parçaları listeleyebilmeliyiz.
- İşlem, ilişkiyi sorguluyor.

context Ziyaretci::oduncAldiklari() :

Set (OduncVerilebilirParca)

Şu anda görünür argümanımız yok. Nesneye yönelik programlamada görünmez argüman var mıdır?

Ama geri dönüş değerimiz var. Sıralı olmak zorunda değil,

Bir kitabın pek çok kopyasını da ödünç alabiliriz. Ama bir kitabı aynı anda iki kere ödünç alamayız.

Gereksinim 3: Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.

context Ziyaretci::oduncAldiklari() :

Set (OduncVerilebilirParca)

Bir kitabı ödünç alsak, teslim zamanını geçirsek, borcumuz olsa, sonra geri getirsek, sistem bunu yine de hatırlayabilmeli. Kayıt kalmalı. Bu yüzden küme koleksiyonu uygun.

Sonra ön koşulları sorgularız.

- İşlemin çalışmasının anlam ifade ettiği durum nedir?
- Bu problemde yok. Durumda değişikliğe sebebiyet vermeden özelliklerin değerini sorgulayan işlemlerde ön koşul genellikle olmaz.
- Ya pre: True, ya da hiç yazmayız.

Gereksinim 3: Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.

Sonraki adımda işlemten dönen değeri belirleriz.

Ödünç ilişkisi üzerinden ilgili (bu ziyaretçinin aldığı) ödünçAlınabilirParça'lara erişiriz.

- Hem ödünç alıp henüz getirmediklerimiz,
- Hem de ödünç alıp, zamanını geçirip, geç getirdiklerimiz

context Ziyaretci::oduncAldiklari() :

Set (OduncVerilebilirParca)

post:

result = Odunc.oduncVerilebilirParca

Gereksinim 4: 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.

Tüm ziyaretçiler için kısıt yok ve tüm çocuklar için kısıt var.

Değişmez mi yoksa önkoşul mu?

- Çocuktaki kontrol her ödünç alma işleminde yapılmalı!

context Ziyaretci::oduncAl(i: oduncVerilebilirParca)

pre: yas <= 12 **implies** oduncAldiklari() -> **size**() < 5

Ön koşul doğru olmazsa işlem çalışmayacak.

Çocuk ise önkoşul var (FOL ==>)

Koşul: alınanlar kümesinin boyutunun 5'den küçük olması DİKKAT!! <= değil.

Bu kısıt çocukların kaç tane ödünç aldıklarını açıkça göstermez. Asla 5'ten fazla olmamasını garanti altına alır.

Post kısmı yok, şu anda tam bir tanım değil.

Gereksinim 5: Ziyaretçi kitap veya DVD ödünç alabilir.

Şimdiye kadar özelliklerin değerine bakan sorgulama işlemlerine baktık.

İşlemlerimiz, elemanlarımızın durumlarını da değiştirebilir.

- Veritabanında kayıt eklemek, güncellemek gibi işlemler yapmalıyız.

Her zaman aynı çıktıyı üreten işlemler matematiksel fonksiyonlar gibidir ve saf fonksiyonlardır.

Saf olmayan fonksiyonların yan etkileri vardır. Ör. VT'da değer değişikliği, I/O işlemi, ekrana çıktı üretmek, ...

Context Ziyaretci::oduncAl(i: oduncVerilebilirParca)

Bir context'te birden fazla kısıt yazarsak hepsini and'leyebiliriz ya da her birini ayrı ayrı tanımlayabiliriz.

Gereksinim 5: Ziyaretçi kitap veya DVD ödünç alabilir.

Ön koşullar:

- ÖdünçAlınabilirParça var olmalı → yeni işlem tanımlarız (ödünçAlınabilirParça'da)
- Ziyaretçi, istek listesindeki ilk kişi olmalı → yeni işlem tanımlarız (ödünçAlınabilirParça'da)
- Bir de çocuklar için olan ön koşulumuz vardı
- Adım adım, yeni işlemler ekleriz. Algoritmayı parçalamak gibi.

context Ziyaretci::oduncAl(i: oduncVerilebilirParca)

pre:

```
i.mevcutMu() and
i.baskasiDahaOnceIstemediMi() and
(yas <=12 implies oduncAldiklari() -> size() < 5)
```

oduncAl Son koşulları-1

Genellikle ön koşullar daha kolaydır.

oduncAl işlemi sonrasında bilgi sistemimizde ne etkiler olacak?

1. oduncAl ilişkisinde yeni bir bağlantı (yeni bir nesne örneği yaratılacak) ve bize döndürülecek.
2. İstek listesini güncellememiz gerek (eğer biz talep etmiştiysek ve ödünç aldıysak) (almamız için listenin en başında olmamız gerekli)

post: exists (

o: Odunc |

o.oduncVerilebilirParca = i **and**

o.teslimTarihi = os.getDate() + o.oduncVerilebilirParca.oduncPeriyodu **and**

Odunc = Odunc@pre -> **including**(o))

oduncAl Son koşulları-1

```
post: exists(
  o: Odunc |
  o.oduncVerilebilirParca = i and
  o.teslimTarihi=os.getDate()+ o.oduncVerilebilirParca.oduncPeriyodu and
  Odunc = Odunc@pre -> including(o) )
```

Exists: "there must exist" $\exists \rightarrow$ o bizim ilişkideki yeni oluşan bağlantımız. İlişkinin iki tarafını da tanımlıyoruz:

- İlk cümlecik: oduncAl işlemi tamamlanınca mutlaka ilgili oduncVerilebilirParca'sı oduncAl(i)'nin argümanı olan bir "o" örneği yaratılmalı
- İkinci cümlecik: teslim tarihi (önceki OCL'den) ayarlanmalı
- Üçüncü cümlecik: işlem bitince nesnemiz ilişki kümesine eklenmeli

oduncAl Son koşulları-2

2. İstek listesini güncellememiz gerek (eğer biz talep etmişiysek ve ödünç aldıysak) (almamız için listenin en başında olmamız gerekli)

Let ifadesi ile mevcut parçanın başlığını b olarak kısaltıp kullanıyoruz:

```
context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
post: let b : Baslik = i.baslik in
  if b.talepler.ziyaretci -> includes(self)
  then talepler=talepler@pre->reject(baslik=b and ziyaretci=self)
  else
    true
  endif
```

Eğer bu başlığa ait talebi yapan ziyaretçi biz isek, talep listesinden kendimizi çıkarıyoruz.

Implies ile de yapabildik.

Ziyaretçi'nin Yaşı OCL

Veri türetmek için de OCL'i kullanabiliriz.

Bu sefer contextimiz bir özellik.

context Ziyaretci::yas: Date

derive:

`OperatingSystem.getDate() - doğumTarihi`

Eksiklerimiz

1, 3, 8, 10, 11. gereksinimlere hiç bakmadık.

İşlevsel olmayan gereksinimlere hiç bakmadık.

Türettiğimiz işlemlere (ör. mevcutMu()) hiç eğilmedik.

iadeEt, cezaÖde, talepİptali işlemlerine hiç bakmadık.

Tarih, Para gibi sınıfları varsaydık.

Parçaların ve Ziyaretçilerin nasıl yaratılacaklarını çalışmadık.

Özetle,

Tek doğru yok.

Analiz safhasında yeni gereksinimler doğabilir.

Açık olmayan noktalar ve hatalar olabilir.

Müşteriyle tekrar tekrar temasa geçmek zorunda kalabiliriz.

OCL ile sıkı düşünüp bu tip noktaları minimuma indirebiliriz.

Davranış Modellemesi

Davranış Modellemesi

Bu ana kadar incelediğimiz yapısal modeller, sistemin her zaman doğru olan özelliklerini tanımladı.

Sistem dış uyarılara nasıl tepki verecek?

- UML'de sistem davranışlarını göstermek için de diyagramlar bulunur.
- Sistem özelliklerini kesin tanımlamak için durum diyagramlarına da bakacağız.

Durumlar (States)

Durum: Verilen bir anda sistem değerleri kümesinin soyut bir tanımı.

Örnek: Dışarıda yağmur yağıyor.

- Zamanda seçilen bir noktada havadaki yağış miktarı için bir tahmin ya da soyutlama.

Daha karışık örnek: Dik kesişen 10 sokak ve 8 caddesi olan hayali bir kasabamız olsun. İçinde tek bir araba olan kasabanın durumu nedir?

- Örneğin, arabamız 3.sokak ve 5. caddede olsun.
- Bunun gibi kaç olası durum vardır? $10 \times 8 = 80$
- Kasabada iki araba olursa? $80 \times 80 = 6400$

Bir sistemin durum uzayı, göz önüne aldığımız değişken sayısı (ve değişkenlerin alabileceği değerlere) katında artar.

- Buna durum patlaması (state explosion) problemi adı verilir.

Olaylar (Events)

Tekil, ani, fark edilebilir bir oluştumdur. Sistem tepki vereceği için, bir uyarı olarak da düşünülebilir.

Sistemdeki olaylar senkron ya da asenkron olabilir.

- **Asenkron:** Rasgele oluşanlar. Ani yığılma da olabilir, zamana da yayılabilir.
- **Senkron:** Periyodik aralıklarla olanlar. Sistemde yoğunlukla bu olayları kontrol eden bir saat ya da olay döngüsü bulunur.

Olaylar, sistemde bir durum değişikliğine neden olurlar. Buna **durum geçişi** (state transition) adı verilir.

- Örneğin, ÖSYM sonuç mektubunu aldığınızda, hayatınızda önemli bir değişiklik olur.

Olaylar kullanıcı eylemlerinin sonucu olarak doğabilir (ör. Ekranda bir düğmeye basmak).

Verideki değişiklikler ile oluşabilir (ör. Sıcaklığın 100 dereceyi geçmesi)

Olaylar zaman geçtikçe biriktirilebilir.

UML Olay Tasnifi

UML'de değişik olay tipleri mevcuttur.

Sinyaller (Signal): Asenkron bildirimler

Metot çağrılarları (Method call): Senkron işlem yürütme/başlatma

Veri durumları (Data conditions): Veride durum değişimi .

Zaman akışı

Durum mu Olay mı?

Jesse Owens'ın Berlin'de 1936'da 100 metre olimpiyat şampiyonu olması

- Durum

Mor renk

- Hiçbiri. Sadece bir özelliğin değeri

Binanın yangın söndürme sisteminin bir yangın nedeniyle devreye girmesi

- Olay

Telefonun çalması

- Olay

Yağmurlu bir gün

- Durum

Saatın zilinin 12'yi vurması

- Olay

Uluslararası Uzay İstasyonu

- Hiçbiri. Sadece çok karmaşık bir nesne

Bilgisayarınızın ekran koruyucusu açılıyor.

- Olay

Modelleme Teknikleri

Olaylara tepki veren sistemlere reaktif sistem adı verilir.

Geleneksel sistemlerde işlerin sırasını kontrol etme sorumluluğu sistemdedir.

- Burada ise sistemde birşeylerin olmasına yol açan uyarılar dış dünyadan gelir.

İlk olarak en basiti olan kombinatoryal sistemlere,

- Sadece durumlar, olay yok

Sonra sıralı (sequential) sistemlere,

- Her durum doğrusal olarak bir diğerinden sonra olaylarla sıralıdır.

En son da en karmaşık olan eşzamanlı (concurrent) sistemlere bakacağız.

- Asankron eşzamanlı sistemlerde pek çok durum ve olay vardır, olaylar zamanda kestirilemeyen anlarda oluşabilir.

Kombinatoryal Modelleme

Yalnızca basit kombinatoryal sistemlerin mantığı tanımlanır.

Sonraki durumlar sadece girdiler tarafından belirlenir.

Önceki durumların geçmişi, sonraki durumu etkilemez.

İki denk şekil: Karar ağacı ve karar tablosu

Karar Tablosu

Sistemin davranışını etkileyebilecek pek çok durum olduğunda kullanılır.

Girdi koşulları ve bu girdilere göre çıktı tepkileri halinde tanımlanır.

- Girdi kombinasyonları, sonucu oluşturma şeklini belirler. (Kombinatoryal)

Tablonun sütunlarında girdiler ve çıktılar yer alır.

Her satırda değişik bir girdi kombinasyonu yer alır.

Ör: Bir atölyemiz ve üç şalterimiz olsun. İki çıktıyı kontrol etsinler.

Girdi			Çıktı	
Ana şalter	Işık şalteri	Güç şalteri	Işık	Motor
+	+	+	+	+
+	+	-	+	-
+	-	+	-	+
+	-	-	-	-
-	+	+	-	-
-	+	-	-	-
-	-	+	-	-
-	-	-	-	-

Karar Ağacı

Karar tablosunun grafik gösterimi

Kararlar sıralı olarak alınıp çıktısı ağaçta bir yol olarak izlenebilir.

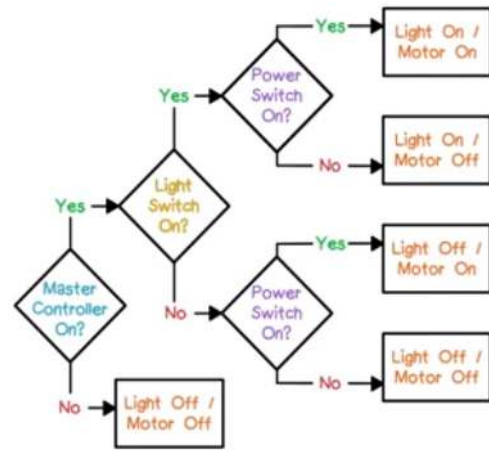
Tamamen aynı bilgi, farklı gösterim.

Eşkenar dörtgenler kararları, dikdörtgenler eylemleri gösterir.

Diyagramdaki bağlantılar olumlu ya da olumsuz karar cevabını gösterir.

Bazı düğümler tekrar edebilir (karar tablosunda da benzer durum vardı)

Girdi olasılıkları arttıkça, ağaç yönetilemez bir hale gelir.



Sıralı (Sequential) Sistemler

Kombinatoryal'den farklı olarak sıralı ve eşzamanlı sistemlerde önceden olanların tutulduğu bir tarihçe/hafıza vardır.

Önceden bir durumdaydık, bu duruma bağlı olarak olaylar yeni bir duruma bizi taşıdı.

Bu gibi sistemlere sonlu durum sistemleri de denir. Çünkü bulunabilecekleri durumları belirli bir sayıya kısıtlarız.

Sonlu durum makineleri ile modelleme yapabiliriz.

Durum Geçiş Tablosu

FSM'leri farklı şekillerde gösterebiliriz. Tablo şekli ile başlayalım.

Satırlarda durumlarımız var.

- 1. sütun durumun adı
- 2.sütun geçişe sebep olan girdi olayı,
- 3.sütun geçiş ile alınacak çıktı aksiyonu
- 4.sütun sonraki durum.

Belirli bir durumda olan sistem için, belirli bir uyarı verildiğinde (oluştığında), oluşacak tepki ve sistemin bulunacağı (yeni) durum tanımlanır.

Durum Geçiş Tablosu

Garaj kapısı örneği: Motor kapıyı yukarı itebilir, aşağı çekebilir, durdurabilir. Motorun çalışıp durmasını sağlayan bir düğme olsun. Kapı tamamen açılabilir, tamamen kapanabilir, ya da arada bir yerde durdurulabilir.

- Düğmeye bastığınızda olacaklar, mevcut durumunuza göre değişecektir.
- Durmuşsa ve bastıysak, aksi yönde hareket edecektir. Yukarı çıkıyorsa ve bastıysak, kapı duracaktır.
- İniyorsa ve bastıysak, sadece durmayıp, güvenlik için açılacaktır.
- Sensörler ile tam açık/tam kapalı durumunu anlayabiliriz.

Kaç durum vardır?

- Kapı açık, motor kapalı
- Kapı kapalı, motor kapalı
- Kapı yarım açık, motor kapalı
- Kapı aşağı, motor açık
- Kapı yukarı, motor açık
- Kapı yarım açık, motor (geçici) kapalı

Kaç olay vardır?

- Düğme
- Sensör açık
- Sensör kapalı

Garaj Kapısı Durum Geçiş Tablosu

Mevcut Durum	Girdi	Eylem	Sonraki Durum
Kapı kapalı motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor
Motor yukarı çalışıyor	Kapı açıldı sensörü	Motor durdur	Kapı açık motor duruyor
Motor yukarı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Kapı açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Motor aşağı çalışıyor	Kapı kapandı sensörü	Motor durdur	Kapı kapalı motor duruyor
Motor aşağı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor

Durum ve olay sayısı arttıkça tablo anlaşılmas bir hal alır.

Garaj Kapısı Durum Geçiş Diyagramı

Sonlu Durum Makinesinin grafik gösterimi

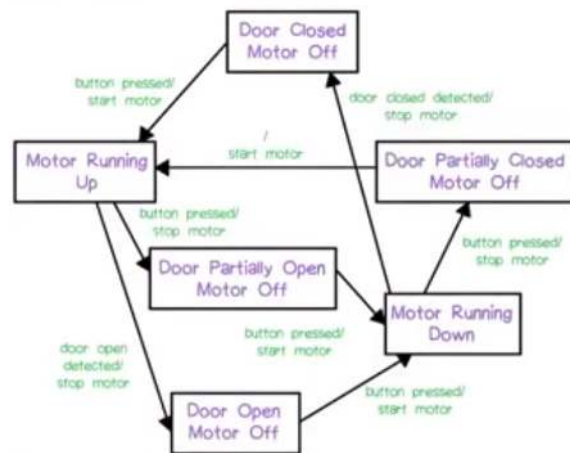
Düğüm oval ya da dikdörtgen ile gösterilir

Bağlantılar durum geçişleridir. Eylem ve genellikle **geçiş** ile isimlendirilebilir.

Düğümün yerlerinin anlamsal bir manası yoktur.

Zamanda bir anda, belirli bir durumda olduğunuzu, bir olay olana dek o durumda kalacağınızı, olaya göre uygun geçişi yapacağınızı düşünerek hazırlayabilirsiniz.

Dikkat! Kapı yarım kapalı-motor duruyor iken, güvenlik nedeniyle eylemsiz olarak motor yukarı durumuna geçiş yapıyoruz. (Epsilon geçiş)

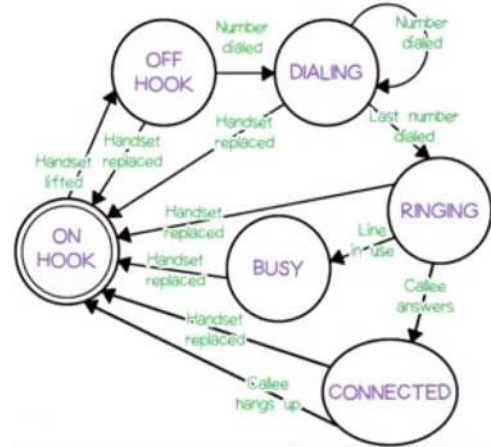


Telefon örneği

Durumlar:

- Telefon kapalı (Çift çizgili, sistemin başlangıç durumu)
- Telefon açık
- Tuşlanıyor (Kendine geçişli, 1. 2. ... Numara çevriliyor. Her aşama ayrı durum)
- Çalıyor
- Meşgul
- Bağlı

Diyagramda gereksiz etiketli geçişler var. Eşzamanlı sistemleri modellerken basitleştirerek iyileştireceğiz.



Durum Geçiş Diyagramı Problemleri

Çok fazla geçiş (ok) var

N durum ve M olay var ise, $N \times M$ olası geçişiniz var.

Çok fazla durum var.

Olabilecek şeylere göre, durumlar da üstel olarak artıyor.

İç içe bulunma çözümü yok.

Çevrilecek telefon numaraları kaç tane olmalı? Kaç kere dönecek?

Durum Diyagramı (Statechart)

Bahsettiğimiz problemleri kısmen çözmek üzere durum diyagramlarını kullanırız.

- Her zaman bu diyagramın da yeterli gelemeyeceği karmaşıklıkta sistemlerle karşılaşabiliriz.

David Harel tarafından geliştirilmiştir.

- Durum geçiş diyagramlarına yaptığı iyileştirmelere durum diyagramı adını vermiştir.
- Karmaşıklığı yönetebilmek için çeşitli mekanizmalar geliştirmiştir.

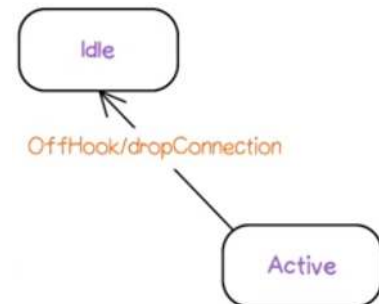
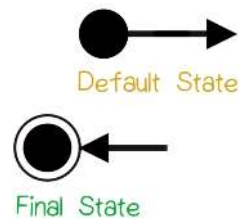
Durum diyagramları UML'in bir parçasıdır ve araçlar tarafından desteklenir.

Durum Diyagramı Sembolleri

Oval ya da dikdörtgen yerine kenarları yuvarlatılmış dörtgen kullanır. İçinde durumun adı yazılır.

Durumlar oklar ile birbirine bağlanır. Olay / aksiyon

Başlangıç / varsayılan durum içi dolu büyük nokta, son durum da çerçeveli hali ile belirtilir.

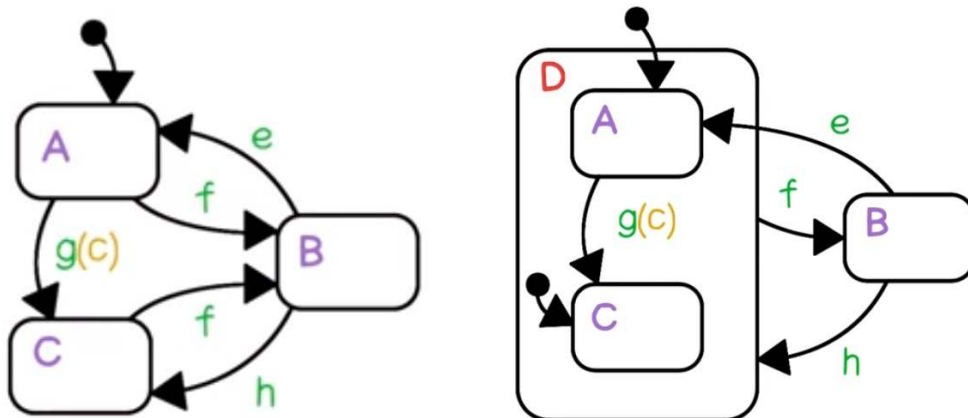


Durum Diyagramı Eklentileri

Durum diyagramları ile yeni özellikler de eklenir.

1. Derinlik (iç içe gruplama): Belirli bir durumun kendi durum makinesi de olabilir. Oraya züm yapabiliriz.
2. Eşzamanlılık (Concurrency): Aynı anda devam eden iki süreç var, bunlarda farklı sayıda durum olabilir. Durum geçiş diyagramlarında olasılıkların çarpımı kadar durum sayısı ile halledabiliyorduk. Burada ayrı ayrı ele alabiliyoruz.
 - Senkronizasyonu da yayım (broadcast) olayları ile halledabiliyoruz.
3. Koşullu Geçişler
4. Giriş/Çıkış eylemleri / aktiviteleri
5. Olay parametreleri
6. Durum geçmişi
7. Varsayılan durumlar

Durum Diyagramı İç içe Gruplama

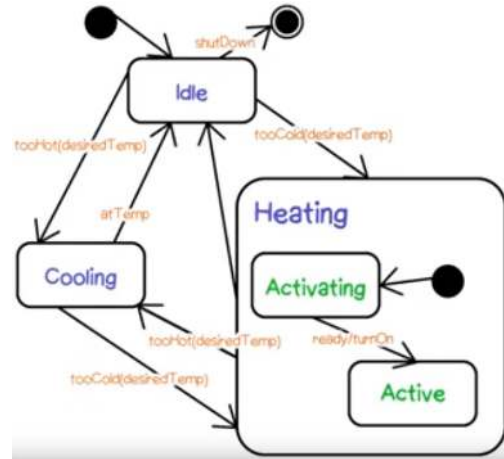


İç içe Gruplama Örneği

Örnek: İklimlendirme Aleti

Isıtma, iki durumlu bir alt sistem

- Varsayılan durumu "activating"
- İç durumların ikisinden de çıkabilecek ikiye ok yerine, gruptan ikiye ok çıkıyor.



İç içe Gruplama Örneği

Örnek: Printer Sistemi

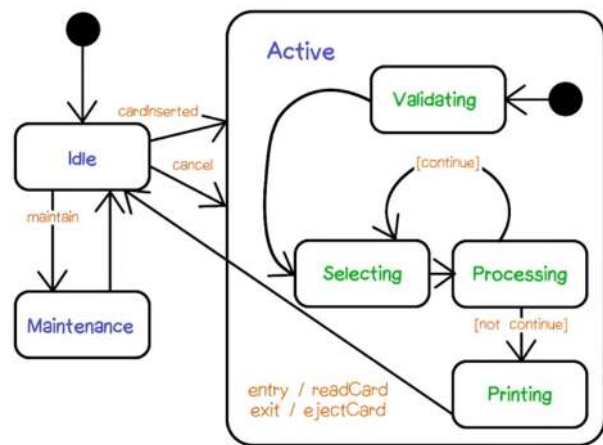
Üç tane üst durum var, "idle", "maintenance", "active"

Active içinde 4 durum var. Idle'dan gelen geçişler, varsayılan "validating"e bağlanıyor.

[continue] ve [not continue] koşullu geçiş

- Bu durum makinesinde temsil edilen nesnenin özellikleri üzerinde mantıksal test ile sağlanır.

Entry ve exit ile belirlenen eylemler, "idle" durumundan gelen ve giden geçişlerde alınacak aksiyonları ifade eder. Hiçbir şey yapılmadan önce, "readCard" yapılacak, "active" durumundan çıkılırken ejectCard yapılacak.



Eşzamanlılık

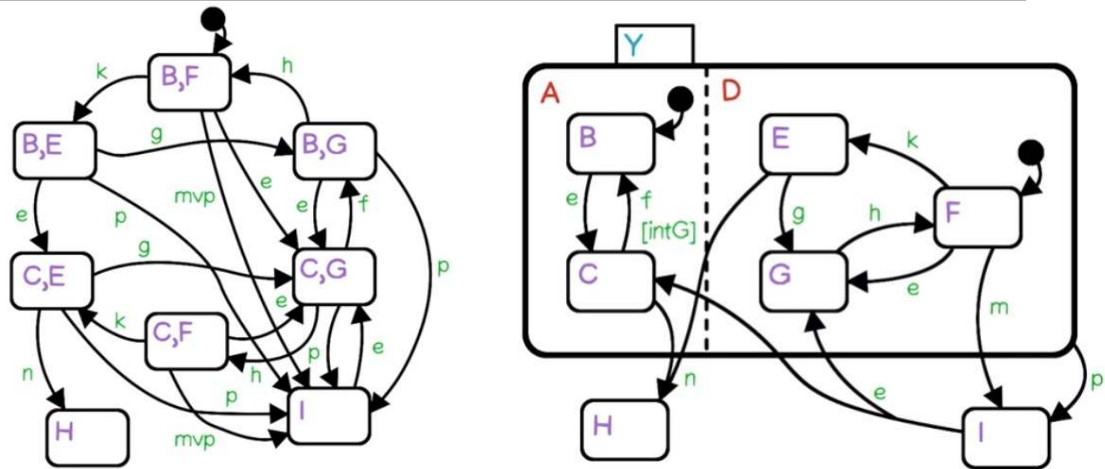
Durumun içinde kesikli çizgi ile ayrılan alanlar ile temsil edilir.

Her makinenin mevcut durumları olabilir, geçişlere tepki verebilirler ve her biri geçişlerde aksiyon gerçekleştirebilirler.

Çarpımlı kombinasyon yerine toplamı kombinasyona olanak verir.

Bir sonraki slaytta Spagetti stilinde eşzamanlılıkta hangi durumdayız takip etmek zordur. Y, H ve I ile eşzamanlı A ve D durumları ile karmaşa azaltılmıştır.

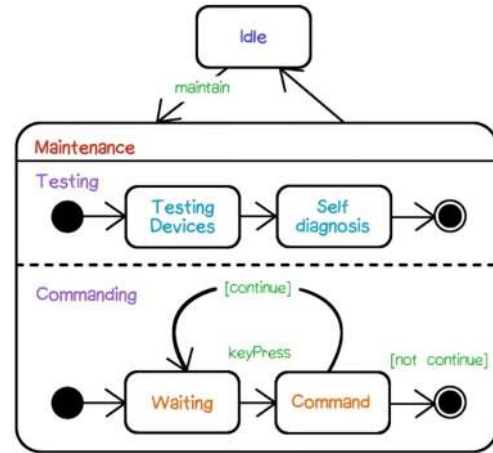
Eşzamanlılık



Eşzamanlılık - UML Gösterimi

"idle" ve "maintenance" durumları.

"maintenance" içinde eş zamanlı iki makine (testing ve commanding)



Senkronizasyon

Eşzamanlı çalışan makineleriniz varsa, bir şekilde işbirliği yapmaları beklenir.

- Yoksa neden diyagrama koyduk ki?

Bu işbirliği için koordinasyon veya senkronizasyon da gereklidir.

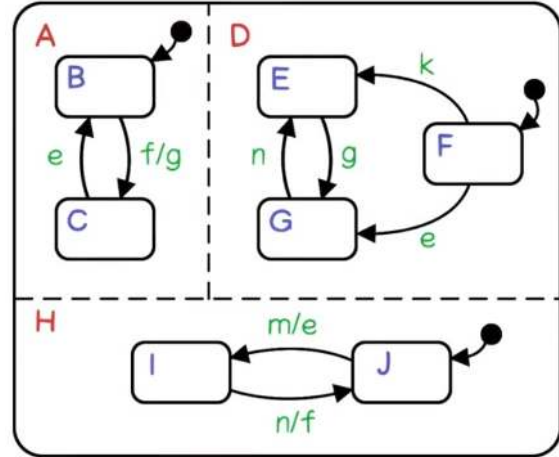
- Yayım [ardarda] Olayları (broadcast [cascade] events)
- Koşullu geçişler (Veri koşulları)

Senkronizasyon- Yayım (ardarda) olaylar

A, D ve H 3 eşzamanlı makine.

Aralarında geçişler var. Bazılarında aksiyonlar (nesneye yönelik sınıf diyagramlarımızda metod çağrıları gibi) var. Fakat, başka bir olayın başlaması da olabilir.

- A'da B'den C'ye F geçişi var. Bu oluyorsa, aynı zamanda G olayı da oluşur diyoruz. Olayları ardarda koyduk.
- Olaylar genel görünür, bu yüzden eşzamanlı makineler de bilir.



Senkronizasyon - Koşullu Geçişler

Köşeli parantezlerle verinin olabileceği durumlara göre koşulları belirtiriz.

- Boolean özellik değerleri
- Değerler sürekli izlenir ve bir değer doğru olduğunda, ilgili geçiş gerçekleşir.

In ve Not In anahtar kelimeleri de desteklenir.

- In: başka bir eşzamanlı makinede x durumunda ise, bu geçişi yaparız.

Koşullu geçiş değişkenleri tüm eşzamanlı makinelere global olarak görünürdür.

Özel Geçişler

UML anahtar kelimelerle belirtilen özel geçişlere olanak verir.

"Active" durumundan "idle" durumuna geçiş etiketi "2 saniye". Sanki bir geri sayaç var ve "active" durumunda 2 saniye kalınca, "idle" duruma geri dönüyor.

"idle" durumunda kendine geçiş var. Tam belirli 11.49 saatinde geçiş yapıyor.

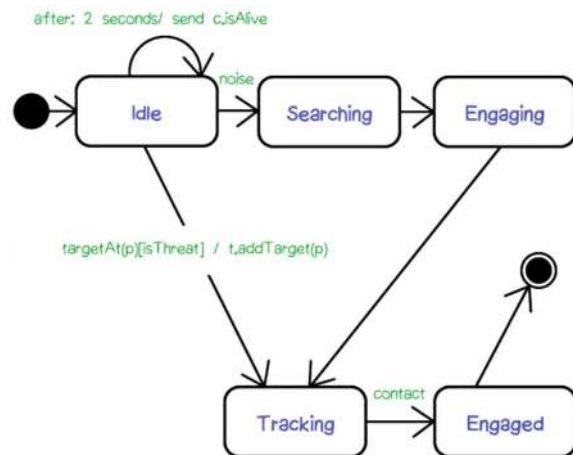


Özel Geçişler

"Idle" durumunda kendine geçiş, diğer durumlarda normal geçişler var.

"idle" ve "tracking" arasındaki geçişte bir eylem var. Bir metodu tetikliyor ve metodun "p" parametresi var.

- Eylem çağrılarında bilgi aktarabiliriz.
- Geçişe neden olan olayda da "p" parametresi vardı. Bilgiyi aktardık.



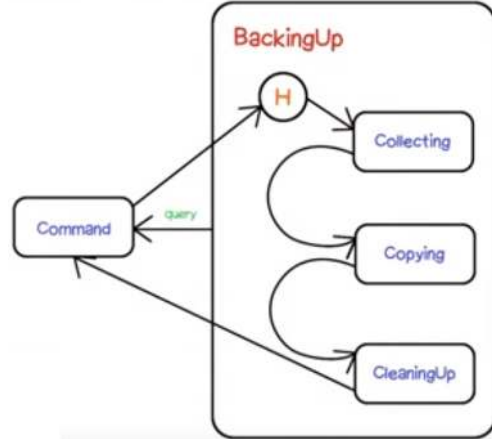
Geçmiş Durumlar

"Command" ve "backing up" harici durumlarımız var.

"backing up"tan "command"a iki geçiş var.

"command"tan çıkan geçiş içiçe durumda H etiketli çembere giriyor.

- Geçmiş (history)
- BackingUp makinesine en son geldiğimizde çıkarken hangi durumda olduğumuzu hatırlar. Son kaldığımız durumdan başlar.
- Geçmiş durum * ile yazılırsa ,iç içe makinelerdeki geçmiş durumlara da dönebiliriz. Güçlüdür ancak, okunurluğu ve takibi çok zorlaştırır.



UML Durum Tanımı

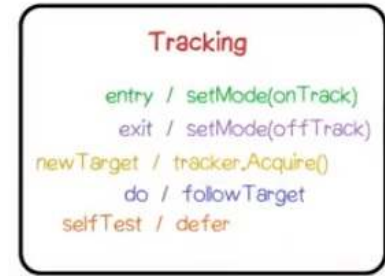
En üstte durumun adı.

Girdi ve çıktıda varsa alınacak aksiyonlar: entry, exit

Girdi-çıkıtlı işlemleri olmadan kendine geçiş: newTarget

Aksiyonlar aç/kapa gibi zaman almayan eylemler, aktiviteler ise zaman alan eylemler: do

Kuyruğa eklenip daha sonra işletilen eylemler: defer



UML Geçiş Tanımı

Geçişin geldiği Kaynak durumu

Geçişin gittiği Hedef durumu

Geçiş tetikleyen olay.

Geçiş üzerinde kontrol (guard).

Geçiş olurken alınabilecek aksiyon

Geçişte Ayrılma ve Birleşmeler olabilir.

Sınıf ve Durum Diyagramı İlişkisi

Geçiş diyagramı oldukça güçlü bir yapıdır.

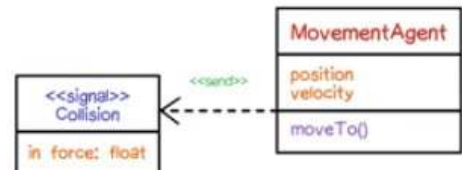
- Tüm özelliklerini her diyagram çiziminizde kullanmanıza gerek yoktur.

Sınıflarımızın özellikleri bir durum uzayı (state space) yaratır.

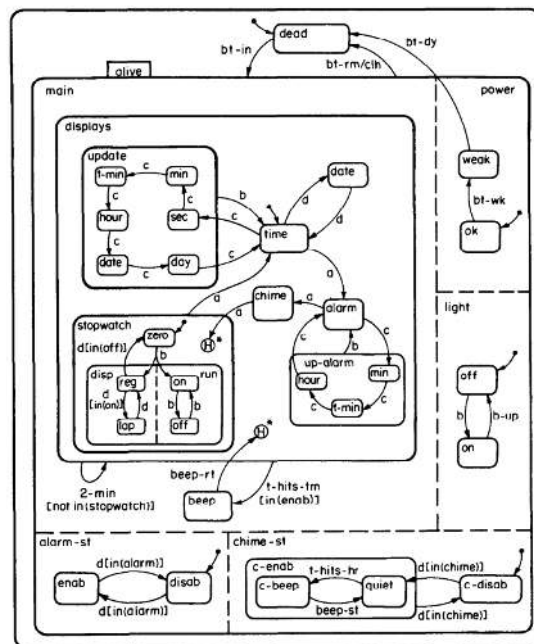
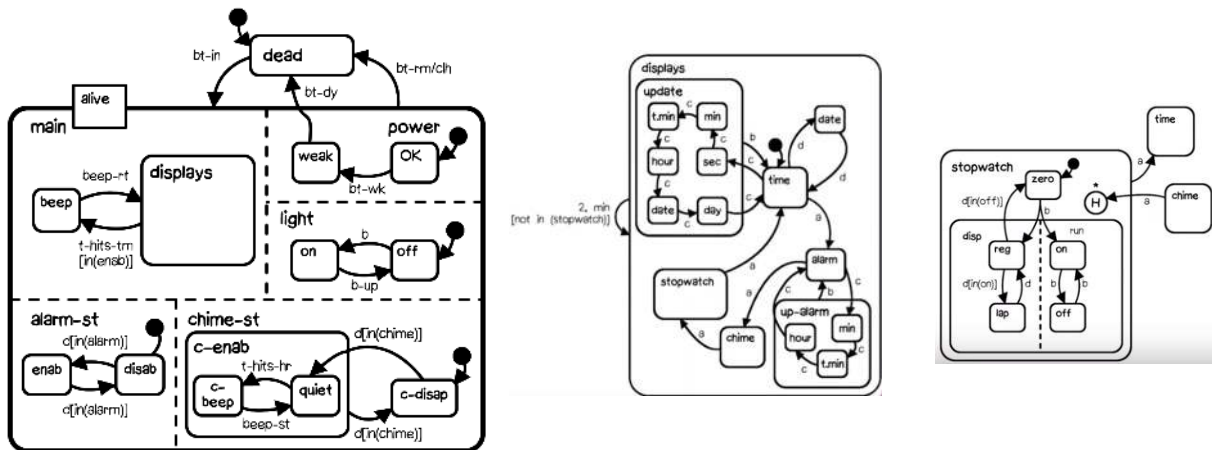
Her sınıfın ayrı bir durum diyagramı olabilir.

- Ancak genellikle sınıfların çoğunun basit durumları vardır ve metotlar yeterlidir.
- Tüm sınıflarınız için durum diyagramı çizmenize gerek yoktur.

Durum diyagramındaki özellikler sınıf özelliklerini, aksiyonlar metotları, olaylar da sinyalleri (bağımlılıkları) gösterir.



Dijital Saat - Üst Seviye Durum Diyagramı



Özet

Reaktif sistemleri inşa etmek zordur.

Karmaşık sistemlerin kilit durumlarına girdiğinin pek çok örneği ile karşılaşmışsınızdır.

Durum diyagramları bu durumları yönetebilmeniz için bir araçtır.

UML'de davranış modellemesi için aktivite, ardışıl, işbirliği, kullanım senaryosu, haberleşme, zamanlama ve genel etkileşim diyagramlarımız da mevcuttur.

UML dışında zamansal mantık (temporal logic) ve işlem cebiri (process algebra) gibi yöntemler bulunmaktadır.



Radıyo Saat Örneđi

Tanımlar

Radyo elektrikle çalışmaktadır. Pili yoktur.

Bir ses ve bir frekans ayar düğmesi ile kontrol edilmektedir..

- Seçilen frekans, küçük dikey beyaz bir çubuk ile gösterilmektedir.

Ekranda 12 saatlik zaman ve iki küçük lamba vardır.

- Birincisi (üstteki öğleden önce/sonra (AM/PM)
- İkincisi (alttaki) alarm kurulu/değil.



Tanımlar

İki şalter ve 5 düğmesi vardır.

- İlk şalter AM/FM bant seçici
- İkinci şalter 4 pozisyonlu: Soldan sağa: Radyo açık, radyo kapalı, radyo ile alarm, bipleme ile alarm
- Soldaki 4 düğme, saatteki çeşitli zamanlayıcıları ayarlamak için. (saat, dakika, uyanma, uyuma) Uyanma, alarmın ne zaman çalışacağı, uyuma, ne kadar süre sonra otomatik kapanacağı. İkisine de basılmadığında zaman ayarı. Saat ve dakika ile her bir sayaç ayarlanıyor.
- Yarın, şimdiki zamandan 15 dakika sonra uyandırılmak istiyorsanız, uyanma butonuna basılı tutup dakikayı 15 kere basmalısınız.
- 5. düğme, uyuklama düğmesi. Her basışınızda alarm 10 dakika ertelenir.
- Alarm, unutma durumlarına karşı her halde 1 saat sonra otomatik olarak susar.



Durum Diyagramı ile Radyo Saatin Dış Davranış Modellemesi

İlk olarak kullanım senaryolarına bakarız.

- Zamanı ayarlama (alarmı ve uyumayı da ayarlamak için ayrı senaryolar)
- Frekansı değiştirme
- Uyuklamaya basma
- ... Daha pek çok senaryo. Ör. Radyo dinleme

Durum diyagramımız olası tüm senaryoları kapsamalı. Hata durumları da dahil (olmasını istediğiniz şey gerçekleşmediğinde radyo ne yapacak?)

.

Görünen özellikler (Percepts)

Uygulamaya ait özelliklerden, kullanıcının görebildikleri, etkileşebildikleri.

Radyo saattekiler:

- Zaman göstergesi
- Ve iki Işık
- Zamanlayıcı düğmeleri. Ancak bunlar bir duruma sahip değil. Bu yüzden bir cisim olarak değil, bir olay olarak (düğmeye basmak/çekmek) modellemeliyiz.
- Hareket edebilen beyaz çubuk
- Frekans ayar düğmesi
- Ses ayar düğmesi
- AM/FM şalteri
- Mod şalteri
- Uyuklama düğmesi
- Hoparlör

Zaman göstergesinde kaç farklı durum olabilir?

İki ışığı düşünmeyelim.

Saatler 01..12 arası

Dakikalar 00..59 arası

$$12 \times 60 = 720$$

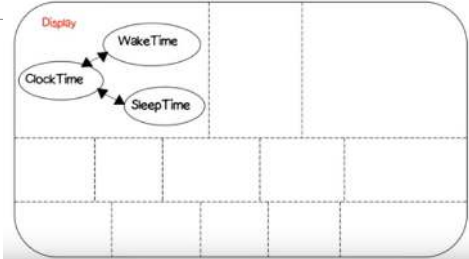
720 durumu olan bir sonlu durum makinesi ile modelleyebiliriz.

Durum diyagramı ile iki eşzamanlı makineyle $12+60 = 72$ duruma anıatabırız.

Bu bile fazla. Saat isimli tek bir düğüme soyutlarız. Beklendiği şekilde 720 durumun oluştuğunu varsayalım.

Gösterge eşzamanlı bir alt makine olarak çalışır. Saati, Uyanmak istediğiniz saati, Kapanma (uyku) saatini gösteren üç durumu vardır.

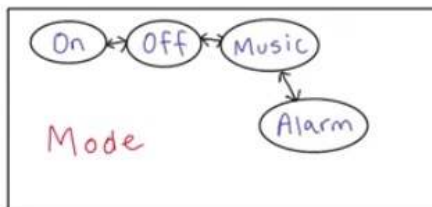
Oklar iki yönlü ama aslında iki ayrı geçiş !



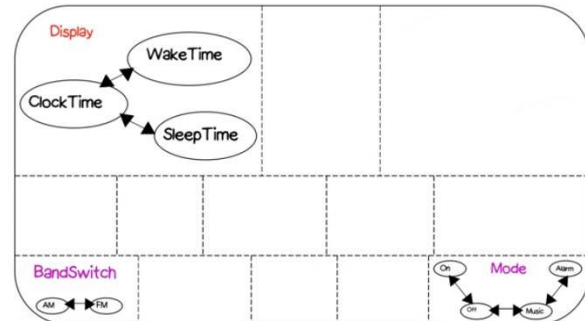
Mod Şalteri Durumları

Kayan "mod şalterinde" cihaz kapalı, radyo, alarm saatinde radyo ve alarm saatinde sadece alarm seçenekleri var.

Bunu FSM ile nasıl modelleriz?



Bunlar da gösterge ile eş zamanlı alt makineler →



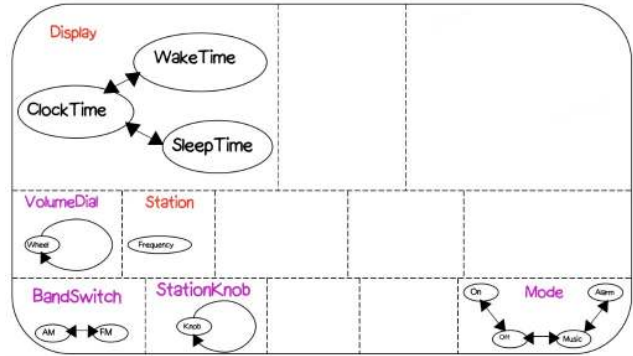
İstasyon Göstergesi

Hareket edebilen beyaz çubuk

Frekans ayar düğmesi ile direk olarak kontrol ediliyor.

Sonsuz sayıda konumda olabilir.

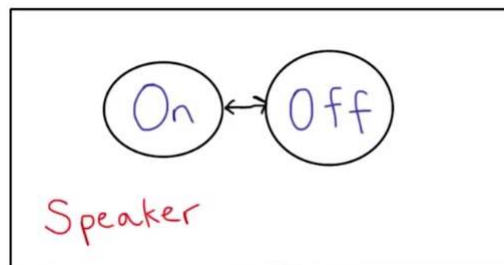
- Geçişsiz tek bir durumda gösterebiliriz.
 - Kullanıcı kendi başına göstergesi değiştiremez
 - Ayar düğmesini çevirir, bu sayede gösterge değişir
- Frekans ayar düğmesi döndükçe istasyon göstergesinin de hareket ettiğini tanımlamalıyız.
 - Biraz ileride.



Hoparlör

Elektronik detaylarına girersek, voltaj seviyeleri vs. pek çok durumumuz olabilir.

Yazılım açısından, sadece hoparlör açık ya da kapalı durumları bizim için yeterli.



Neredeyiz?

Radyo saatin davranışını modellemek için Durum diyagramı geliştiriyoruz.

Radyonun görünen özelliklerine ait davranışlar için şimdiye kadar 7 eşzamanlı makine kullandık.

- Boş yerlere diğer makineleri ekleyeceğiz.

Durum geçişlerini şimdilik gözardı ettik.

- Durum geçişlerine neden olan olaylara göz atalım.

Harici Kontroller ve Tepkiler

Davranış, cihazın bu olaylara nasıl tepki verdiği.

- Her bir harici kontrol, cihaza hangi tepkileri verdiriyor/olayı sağlıyor?

Mod şalteri: sola kaydır, sağa kaydır.

- İki bağımsız olay olabilir, ya da bir parametresi olan kaydır olayı olabilir.

Frekans ayar düğmesi: döndür (parametresi açı)

AM/FM şalteri: sola kaydır, sağa kaydır.

Uyuklama düğmesi: bas

- Kullanıcı açısından olaylara bakıyoruz. Alarm çalmazken de basabilir. Bu durumda ne olacağını bize durum diyagramımız söyleyecek.

Uyanma düğmesi: bas, çek

- Saat ve dakika düğmeleri: bas



Eylemlerden Olaylara

Kullanıcı aksiyonlarını olaylara (tepki vermemiz gereken anlık oluşum) çeviririz.

Tüm olayları ve varsa bunların kombinasyonlarını ele almalıyız.

1. Ses ayar düğmesini çevir
2. AM bandını seç
3. FM bandını seç
4. İstasyon seç
5. Fişten çıkar
6. Fişe tak
7. Uyku saati düğmesine bas
8. Uyku saati düğmesini bırak

Analizde hepsini listelemek ve parametrelerini belirlemek gerekir.

Sistemin bunlara ne tepki vereceği belirlenir.

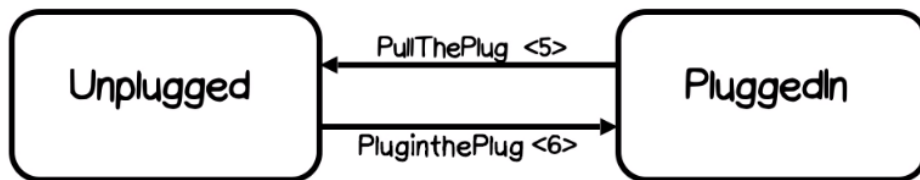
9. Uyanma saati düğmesine bas
10. Uyanma saati düğmesini bırak
11. Mod seçim düğmesini sağa kaydır
12. Mod seçim düğmesini sola kaydır
13. Uyuklama düğmesine bas
14. Saat düğmesine bas
15. Saat düğmesini bırak
16. Dakika düğmesine bas
17. Dakika düğmesini bırak

En Dış Durumlarımız nelerdir?

Fişe takılı olabiliriz yada fişimiz çekilmiş olabilir.

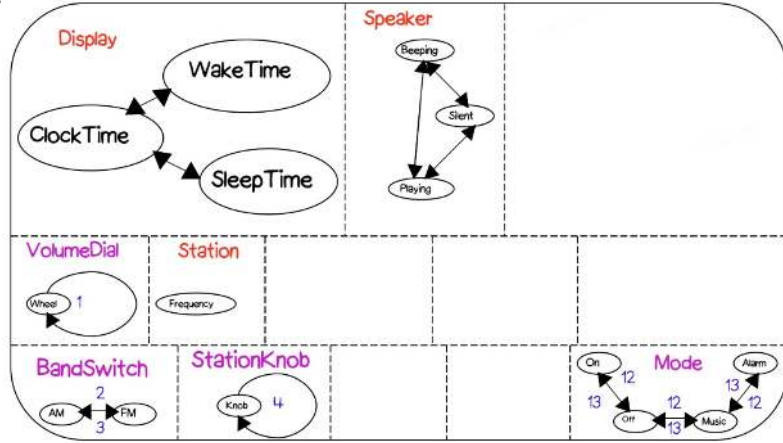
Bu iki durum arasında 5. ve 6. olaylarla geçiş yaparız.

Bundan sonrasında, hep fişe takılı durumunun içini modelleyeceğiz.



Olayların Durum makinesine yerleşimi

1. Ses ayar düğmesini çev
2. AM bandını seç
3. FM bandını seç
4. İstasyon seç
12. Mod seçim düğmesini sağa kaydır
13. Mod seçim düğmesini sola kaydır



Yeni alt makineler

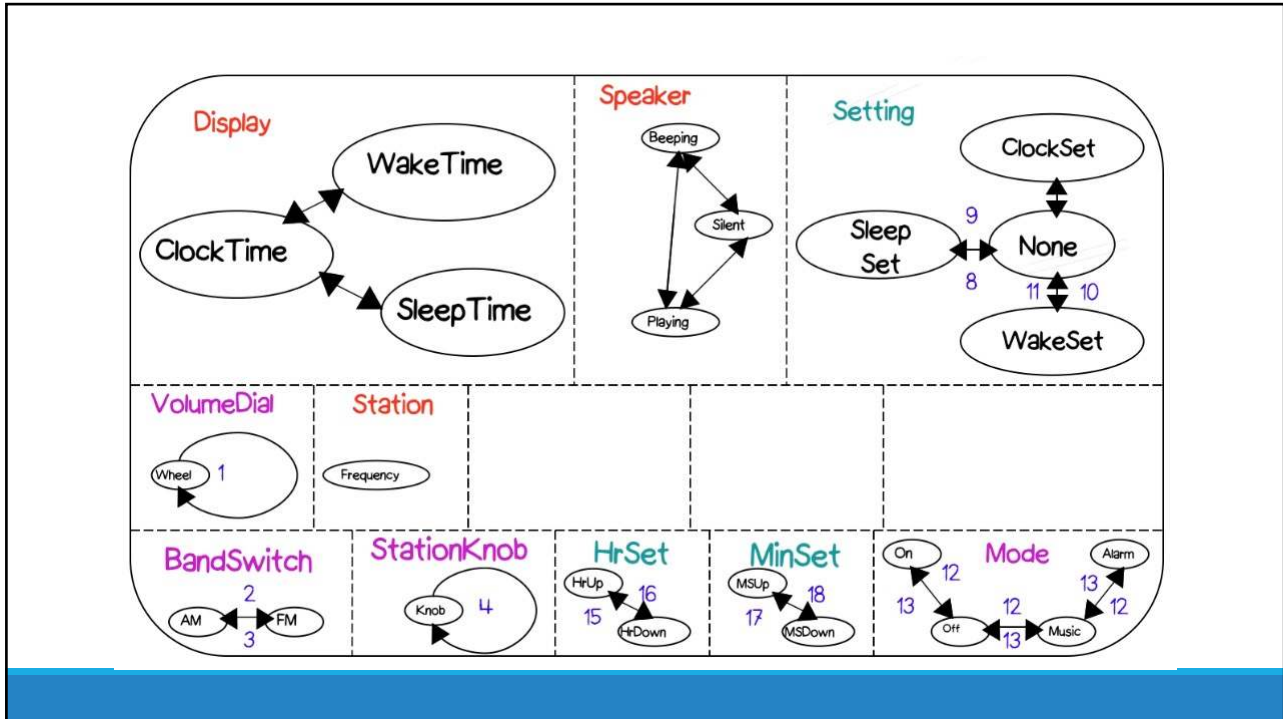
Olaylarımız, yeni eşzamanlı alt makinelere ihtiyaç duyduğumuzu gösteriyor.

Örneğin uyuklama düğmesine basınca, 10 dakika sonra tekrar alarm devreye girmeli.

- Kullanıcıya görünür değil ama kullanıcıyı etkileyen bir iç sayaç var gibi.
- Bunları da modellemeliyiz.

Uyku ve uyanma düğmelerine bastığımızda, bir duruma gireriz.

- Bu durumda saat ve dakika düğmelerine basarsak da ayar durumuna gireriz.



Olaylara verilen Tepkiler

Radyomuzun kullanıcıya sunduğu gerçek değeri, bu olaylara verdiği tepkilerdir.

Listelediğimiz tüm olayların radyonun durumunda bir etkisi olması gerekir.

Ör. Ses ayar düğmesini çevirdiğimizde, hoparlörden gelen sesin artmasını bekleriz.

- Ayrıca, düğmenin açılma konumu da değişir.
- Bunu da tepki olarak göstermeliyiz. (Gece karanlıkta sesi ayarlıyorsa?)

#	Uyarı	Tepki
11	Mod seçim düğmesini sağa kaydır	Açık → Kapalı Kapalı → Müzik Müzik → Alarm
12	Mod seçim düğmesini sola kaydır	Alarm → Müzik Müzik → Kapalı Kapalı → Açık

Uyaran Tepki Tablosu

#	Uyarı	Tepki
1	Ses ayar düğmesini çevir	Düğmenin pozisyonunu değiştir Hoparlörün sesini arttır
2	AM bandını seç	Düğmenin konumunu değiştir Bandı değiştir
3	FM bandını seç	Düğmenin konumunu değiştir Bandı değiştir
4	İstasyon seç	Düğmenin pozisyonunu değiştir Çalan istasyonu değiştir İstasyon göstergesini değiştir
5	Fişten çıkar	Fişten çıkarılmış durumuna geç
6	Fişe tak	Fişe takılı durumuna geç, tüm eşzamanlı makinelerin varsayılan durumlarına geç

Zamanlı olaylar

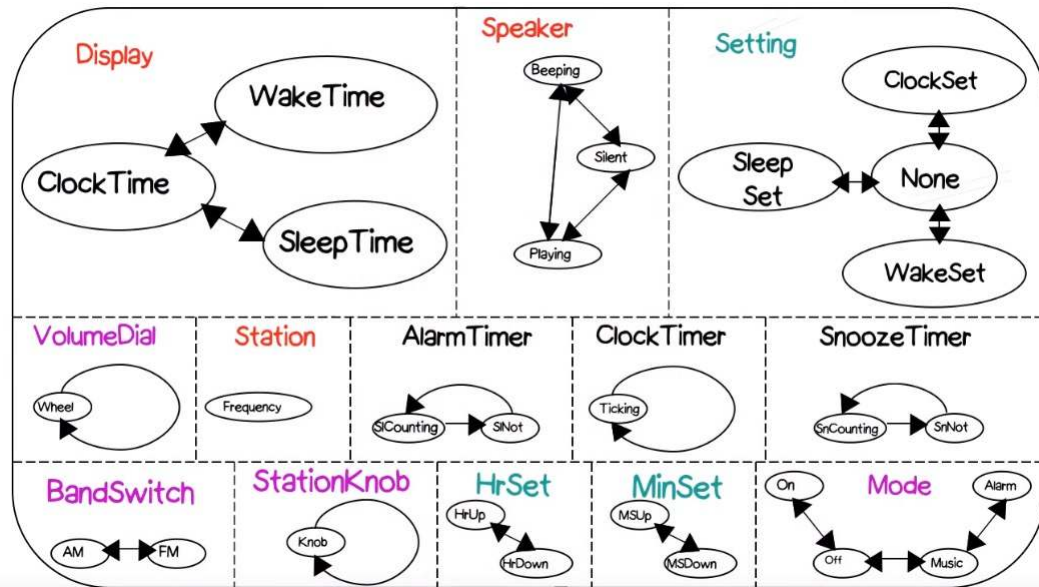
Radyonun saati, alarmin ayarlandığı saate geldiğinde ne tepki verilmesi gerekir?

1. Bip/radyo çal
2. Alarm susma zamanlayıcısını başlat
3. Hoparlörü aç

İç Durumlar

Her ne kadar dış uyaranlara karşı durumları modellesek de, sistemin tam olarak çalışması için kullanıcılara görünmeyen (zamanlayıcılar gibi) iç durumlara da ihtiyaç duyarız ve bunları da modellememiz gerekir.

- Uyuklama → başla, 10 dakikaya kadar say
- Alarm → başla, 60 dakikaya kadar say
- Uyku → başla, ayarlanan zamana kadar (max 60 dakika)
- Saat zamanı → her 1 dakikada değiştir.



Diğer İç Olaylar

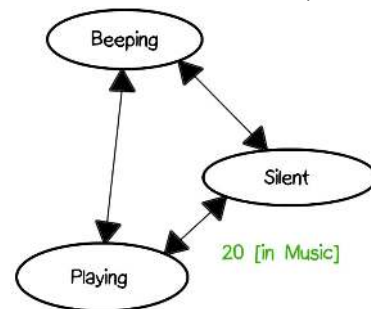
#	Uyarı	Tepki
19	Alarm sayacı bitti	If not in Hoparlör:Sessiz go to Hoparlör:Sessiz
20	Zaman, alarm saatine geldi	If in Mod:Muzik go to Hoparlör:Çalıyor If in Mod:alarm go to Hoparlör:Bipliyor
21	Uyuklama sayacı bitti	If in Mod:Muzik go to Hoparlör:Çalıyor If in Mod:alarm go to Hoparlör:Bipliyor
22	Zaman, uyanma saati +1 saate geldi	If in Hoparlör:Çalıyor go to Hoparlör:Sessiz If in Hoparlör:Bipliyor go to Hoparlör:Sessiz

Kontrollü (Guarded) Geçişler

Alt makinenin belirli bir durumda olması ile kontrol edilecek şekilde bir olaya tepki verilen durumlar

If in Mod:Muzik go to Hoparlör:Çalıyor

- Sessiz durumu ile Çalıyor durumu arasındaki geçişin kontrolü, Müzik durumunda olmaya bağlı
- Köşeli parantez ile gösteririz.



Yayım/Ardarda (Cascaded) Olaylar

Aktiviteleri koordine etmenin bir diğer yolu.

Bir iç olay yayımlanır. Tüm durumlar tüm olayları dinlediği için, eşzamanlı çalışan alt makinelerde senkronizasyon için kullanılabilir.

Frekans ayar düğmesi çevrildiğinde, üç tepki gerekli:

- Düğme fiziksel olarak yeni pozisyona gelmeli
- Radyo kanalı değişmeli
- O anki istasyonu gösteren beyaz çubuk hareket ettirilmeli → bu ayrı bir eş zamanlı makinede! Haberdar edilmeli.
 - Yeni bir iç olay yaratırız. İstasyon makinesi buna tepki verir.

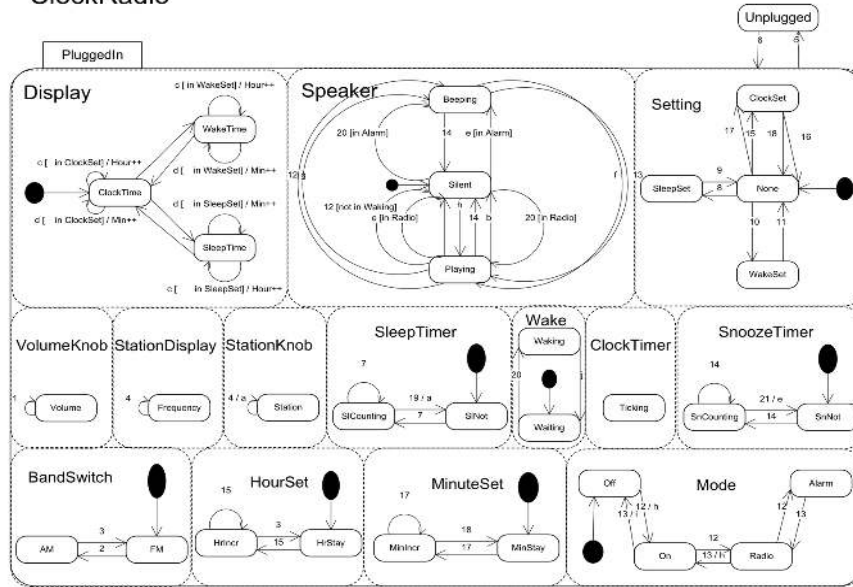
Yapabileceklerimiz

Bu şekilde yeni olaylar ekleyerek, sistemimizin istediğimiz gibi davranmasını sağlamayı garanti edebiliriz.

Tam bir model için şunları da tamamlamamız gerekli:

- Her eşzamanlı makinenin varsayılan durumları belirlemeliyiz.
- Geçişlere gerekli olan kontrolleri [guard] yerleştirmeliyiz.
- Geçişleri sağlamak için gerekli olan iç olayları yaratmalıyız.

ClockRadio



Geçerleme

Durum diyagramımızı tamamladıktan sonra geçerleme/validasyonunu da yapmamız gerekir.

Bir ekip ile gözden geçirme yapabiliriz. Tüm kullanım senaryolarının üzerinden geçerek, sistemin tam olarak beklediklerinizi yaptığını doğrulayabiliriz.

Model kontrolü yapabiliriz. Diyagramı gerçekleyip üzerinde testler koşturabiliriz ve beklediğimiz şekilde olayların ve geçişlerin oluşup oluşmadığını test edebiliriz.

Bir simülatör yaratıp sistemin çalışmasını simüle edebiliriz.

Yeni sorular oluşursa yine kullanıcıya döneriz.

Durum Diyagramı Modelleme Yöntemi

1. Kullanım senaryolarını hazırla
2. Görünen (harici) özellikleri (percepts) belirle
3. Görünen özellikleri durumlarla (eşzamanlı makinelerle) modelle
4. Harici kontrolleri ve uyarıları belirle
5. İlave durumlar ve geçişler/olaylar ile modelle
6. Uyarı/tepki ikililerini listele
7. Gerekli (ör. Zamanlayıcılar) iç durumları belirle
8. koordinasyon mekanizmalarını oluştur
9. Eylemleri/aktiviteleri ekle
10. Senaryolar ile sonuç makinesini geçlerle



Yazılım Mimarisi

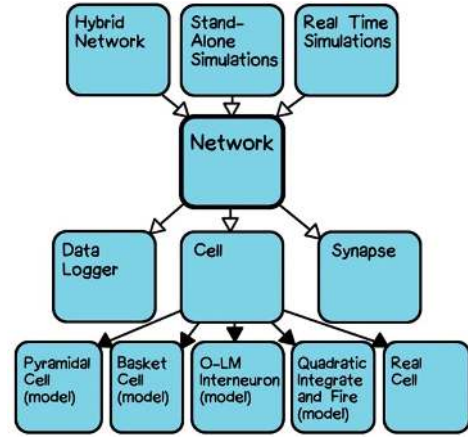
Yazılım Mimarisi - Genel Bakış

Bir tasarım probleminin en üst seviyeli ifadesi

Pratikte; kutular ve oklarla bir sistemin temel bileşenlerinin neler olduğunu ve birbirlerine nasıl bağlı olduklarını tasvir eden çizimlerdir.

- Kutular bileşenleri, oklar aralarındaki bağlantıları gösterir. Kontrol akışı ya da veri akışı olabilir.

Evrensel olarak kabul gören anlamsal bir uzlaş sağlanmamıştır.



Gayresmi Tanım

Sistemin bileşen altsistemleri ya da modüller halinde organizasyonu ya da bölümlenmesi

Evrensel olarak mimari, katmanlar halinde gerçekleştirilir. İteratif olarak çoklu katmanlar iyileştirilir.

- En soyut halini oluşturan bileşenler
- Bu bileşenlerin alt bileşenleri / alt sistemleri
- Parçaların hakikaten gerçekleştirilebileceği kadar düşük bir seviyeye kadar devam ederiz

Genellikle klişe mimari stiller de kullanılır.

Analizin Bileşenlere Etkisi

Sadece analiz modeline bakarak sistemin bileşenlerini belirlemeye çalışalım.

Analiz modelleri;

Müşteri gereksinimlerindeki değişimlere iyi adapte olabilir.

- Tasarımdan önce yapıldıklarından, tasarım kısıtlarından etkilenmezler ve değişimlere adaptasyonları kolaydır.

Tasarımda kullanılacak yaklaşımı göstermemelidir.

- Analiz ve tasarımı mümkün olduğunca (etki altında bırakmamak için) bağımsız tutmaya çalışırız

Donanımdaki değişimlere karşı dirençli ve esnektir.

- Sistemin çalışacağı ortamla ilgili varsayımlar yapmamalıdır ve değişimlere ayak uydurabilmelidir.

Sistemin gerektirdiği tüm bileşenlere sahip değildir.

- Koleksiyon sınıfları ve benzeri işlem sınıfları tasarımda eklenecektir.

Resmi Tanım (USP'nin Tanımı)

The set of significant **decisions** about the organization of a software system, the selection of the **structural elements** and their **interfaces** by which the system is composed,

together with their behavior as specified in the **collaborations** among those elements,

the **composition** of these structural and behavioral elements into progressively larger subsystems,

and the architectural **style** that guides this organization: these elements and their interfaces, their collaborations, and their composition.

Software architecture is concerned not only with structure and behavior but with **usage, functionality, performance, resilience, reuse, comprehensibility, economics and technology constraints and trade-offs, and aesthetic concerns.**

Resmi Tanım (USP'nin Tanımı)

Mimari, kararlarla ilgilidir.

- Mimarin seçimleri, hangi bileşenlerin olacağını, nasıl etkileşeceklerini, işlevsel olmayan gereksinimlerin nasıl karşılanacağını belirler.

Bileşenler söz konusu ise, bunlar yapısal elemanlar ve onların arayüzleridir (bulundukları dünyaya sundukları ve dünyadan bekledikleri)

Bileşenler birbirleriyle etkileşir.

- Diğer bileşenlerle işbirliği yaparlar

Mimari, yapısal ve davranışsal bu elemanların birleşip gitgide daha büyük alt sistemler ve sistemler oluşmasını sağlar.

- Bu birleşme mimari stillerin rehberliğinde (deneyim, daha önceden benzer sistemlerin geliştirilmesinde iyi reçeteler) gerçekleştirilebilir.

Yapısal ve davranışsal elemanlar dışında kullanım, performans, anlaşılabilirlik, ekonomi, teknolojik kısıtlar ve ödümler, estetik ile de ilgilidir.

Diğer Tanımlar

Dwayne Perry & Alex Wolf: Elements + forms + **rational**

- Mantıksal temel: Mimarların yapmak için üzerinde anlaştıkları kararlar ve sebepler

IEEE: The fundamental **organization** of a system, embodied in its **components**, their **relationships** to each other and the environment, and the principles governing its design and evolution

Verhoff: The software architecture of a deployed software is determined by those aspects that are the hardest to **change**.

- Bileşenleri nelerin oluşturacağını tespiti, değişmesi en zor olan şeyleri saklamaya dayanır
- Bir sistem 5 yıl sonra aynı kalmayacak, değişecektir. Bu değişimler, sistemin beklenmedik şekillerde kırılmasına yol açacaktır.

Garlan & Shaw: Components + connectors + configurations

Mimariyi İfade Etmek

Component: Bileşen

- İşlemsel veya veri elemanı ve sistemin geri kalanı ile olan arayüzü (interface / port)
- Arayüz, sistemin kalanından bekledikleri ve sistemin kalanına sunduklarıdır

Connector: Bağlayıcı

- Bileşenler arasındaki haberleşme protokolleri
- Protokolü işletmek için kod da içerebilir

Configuration: Düzenleşim

- Bileşen ve bağlayıcıların nasıl bir araya geleceği
- İkili bir bağlayıcı ise, iki ucunda birer bileşen
- Tüm elemanların topolojisi konfigürasyonu oluşturur

Bileşenler

Taylor et al.: A software component is an architectural entity that

- (1) encapsulates a subset of the system's **functionality** and/or **data**
- (2) restricts access to that subset via an explicitly defined **interface**, and
- (3) has explicitly defined **dependencies** on its required execution context
 - Bileşenin istendiği şekilde çalışabilmesini sağlamak için neler gerekiyor?

Szyperski: A software component is a unit of **composition** (bileşenleri alıp bir araya getireceğiz) with **contractually** specified **interfaces** (arayüzlerimiz açık ve diğer bileşenler bunları bilir ve hemfikirdirler) and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties.

Bileşenleri Seçmek

Bileşenleri seçmenin açık yolu, sistemin ne yapacağını / hesaplayacağını ortaya koymak ve bunu parçalara ayırmaktır. Burada bir belirsizlik de söz konusudur.

Bunların dışında pek çok faktör de hangi bileşenlerin sistemimizin parçası olacağını etkiler.

- Beklenen işlevsellik
- Halihazırda var olan (önceden yazdığınız/kütüphanelerden gelen) yeniden kullanılabilir parçalar
- Fiziksel bilgisayar mimarisi (ör. Çok çekirdekli işlemcide paralelleştirme)
- Ekibiniz ve deneyimi
- Conway's Law: Bir sistemin son hali, onu inşa eden organizasyonun yapısına bağlıdır.
- Sistemin değişim yörüngesi

Uygulama Programlama Arayüzü (API)

Bileşen tanımının bir kısmını oluşturur ve bir kısmını gerektirir

Bileşenin uygulama programlama arayüzü olarak tanımlanır.

- İndirdiğiniz bir uygulamadaki sınıflar, metotları, ... bunların tanımı API'yi oluşturur
- Birimdeki elemanları nasıl çağırabileceğinize dair isimler (metot isimleri, argümanlar, argümanların tipleri, geri dönüş değeri, ...)

API belirli bir programlama dili ile tanımlanabilir. (Dile Bağlama: Language binding)

- OCL gibi daha yüksek seviyeli soyutlama dili ile tanımlanabilir
- ADL (Architectural Description Language) gibi özelleştirilmiş notasyonlar da kullanılabilir

Bağlayıcılar (Connectors)

İşlevsellik (gereksinimlerden doğan) sorgusu ile bileşenleri bulmak daha kolaydır.

Bağlayıcılar biraz daha karışıktır.

- Tasarımcının belirli problemlerle başa çıkabilmek için bazı kararlar alması gerekir.

Taylor et al.: A software connector is an architectural element tasked with **effecting** and **regulating interactions** among components.

- Bileşenler arası akış (piping)

Bağlayıcılar, bileşenler arası etkileşim için bir protokol sağlar.

- Protokol: Kimin hangi sıra ile konuşacağını, hangi bilginin hangi yönde iletileceğini, yanlış bir şey olduğunda ne yapılacağını söyleyen bir dil.

Örnek Bağlayıcı

En basit bağlayıcı: Prosedür çağırısı / geri dönüşü

İki rol var : Çağırıcı ve çağrılan

Bu bir mesaj çiftidir. İlkinde bir metod çağrılır, ikincide geri döndürülen bir değer alınır.

Asimetrik ilişkidir. Çağırıcı çağrıyı başlatır ve çağrılanı bekler.

Senkron ilişkidir. Çağırıcı, çağrılan cevap verene kadar durur/işlem yapmaz.

Bağlayıcılar, tipi olan parametreler ve geri dönüş değeri ile bilginin aktarımına imkan tanır.

Düzenleşim (Configuration)

Bileşen ve bağlayıcılar olduğunda, bunları birleştirmek gerekir.

Buna konfigürasyon adı verilir.

Taylor et al.: An architectural configuration is a set of **specific associations between the components and connectors** of a software system's architecture.

Kalan Terminoloji

Kavramsal Mimari (Conceptual Architecture): Kavramsal demek, belirsiz ve yüksek seviyeli anlamına gelir. Geliştirme sürecinin en başlarında, -gereksinimlerin neler olduğu hakkında tam bir fikrimiz olmadan bile önce- üretilir.

- Çok soyut bir seviyede bileşenler ve bağlayıcıların neler olabileceğini düşünmemizle başlar. Takımların neler olabileceğini, son ürünü ne kadar sürede üretebileceğimizi öngörmekle devam eder.

Planlanan vs. inşa edilen mimari (hayaller / gerçekler)

- Planlama süreci boyunca takım mimarının ne olacağını belirler ve belgeler.
- Uygulama gerçekleşirken, başka bir şey inşa edilebilir.
- İyileştirme sürecinde gerçekleştirme ekibi açık kaynaklı veya başka takımdan gelen ve geliştirme süresini çok kısaltan bir bileşene rastlayabilir. Ancak bu bileşen, planlanan sonuca uymuyor olabilir.
- Buna mimari sapma (architectural drift) adı verilir. Bakım sürecinde oluşursa, mimari erozyon olarak adlandırılır. Bakım ekibi hız ve anı kurtarmak adına planlanmayan değişiklikler yapabilir ve bunu genele (dokümantasyona) yansıtmaz.

Mimari Görünümler

Mimari tanım sadece bir diyagram değildir. Bir kararlar kümesidir.

Bu kümeyi tam olarak tanımlamak için pek çok diyagram ve belge üretilebilir.

Bunlara mimari görünüler (architectural views) adını veriyoruz.

Bu küme geniş ve pek çok yönlü derleme içerebilir. Pek çok mimari diyagram bir kısmını ifade eder.

Şu UML diyagramları katkı sağlayabilir:

- Sınıf, Bileşen, Barındırma, Nesne, Paket, Kullanım Senaryosu, Ardışıl, İletişim, Etkileşim

Mimari Stiller

Binalar gibi yazılım sistemleri de çeşit çeşittir. Bunlara stil adını veriyoruz,

Taylor et al.: Named collection of architectural design decisions that

- (1) are applicable in a given development context,
- (2) contain architectural design decisions that are specific to a particular system within that context,
- (3) and elicit beneficial qualities in each resulting system.

İsmlendirilmiş karar koleksiyonu

- Bu kararlar sistem spesifikasyonları ile tanımlı belirli koşullar altında uygundur
- Tasarım kararları olası bileşenlerin neler olduğunu ve etkileşimlerini belirli bir şekilde olmaya zorlar
- Bu stillerden, oluşturduğumuz sistemde yararlı katkılar elde etmemizi sağlar.

Mimari Stiller

Servis veren ve servis alan yazılım bileşenleri fiziksel olarak ayrıdır (ayrı makinelerde). Bu sayede iki taraf da bağımsız olarak ölçeklendirilebilir.

Servis veren bileşenler, alanlar kimliklerini belirtmediği sürece servis alanların kim olduğunu bilmez. Bu sayede şeffaf olarak değişebilecek istem yapanlara hizmet verebilirler.

Servis alanlar birbirinden izoledir, bu sayede bağımsız olarak ekleme, çıkarma, değişim yaşayabilirler. Servis alanlar sadece servis verenlere bağımlıdır.

Servise talep belirli bir değerin üzerinde çıktığında, servis verenler dinamik olarak eklenebilir ve mevcut hizmet verenlerin yükünü azaltabilir.

Hangisi?

Blackboard - Client Server - Implicit Invocation - Layered - Object Oriented - Pipe and Filter

Mimari Stiller

Faydalar:

Problem üzerindeki deneyimimizi kodlamış oluruz.

Örneğin Client server stilinde parçaları bölmek ve bağlamak için diğerlerinden daha iyi çalışacak pek çok seçeneğimiz bulunmaktadır. Ayrıca problemlerin neler olabileceğini ve bunlar çözmek için neler yapabileceğimizi de biliriz.

Bu sayede genel geliştirme yükümüz azalır. Çıkmaz sokaklara dalmayız.

Mimari stiller aynı zamanda standartlar haline de gelebilir. (Referans Mimari)
Seçtiğimiz çözümün uygunluğunu geçerleyebilmemizi de sağlar.

Mimari stiller yeniden kullanımı da destekler. (Hazır bir mysql server kullanabiliriz)

Geliştirme sürecimizi de şekillendirebilir.

Stil Kataloğumuz

Yıllar içinde belirli durumlara çözüm olmak üzere oluşan uzun büyük bir listemiz var.

Tasarım deneyim gerektirir. Deneyim de, olası çözümlerden haberdar olmaktan geçer.

Abstract Data Type (ADT): Temsilin gizlenmesi

Batch sequential: Geçerle, düzenle, güncelle döngüsü

Blackboard: havuz, fırsatçı kontrol, işbirliği yapan ajanlar (istekler ve sonuçlar ortak veri havuzuna)

Big ball of mud: (aslında stil değil, stil eksikliği): Takım, mimari tasarım çalışması yapmadığında oluşur

Client-server: hareket işleme, çok katman

Stil Kataloğumuz

Component-based: iyi tanımlı arayüzlerle haberleşen tekrar kullanılabilir modüller

Coroutines: Simetrik etkileşim. A, B'yi, B, A'yı çağırabilir. A, B'yi 2.kere çağırdığında, B ilk çağrıda kaldığı yerden devam eder. (printf'teki format ve veri ikilisi gibi)

Data centric: Veritabanındaki saklı yordamların kullanılması

Domain Driven Design (DDD): iş odaklı, iş modeli. Veri akışı, iş modelinde otomatik değişikliklere neden olur. (Ör. Vergi hesabı uygulama programı)

Implicit Invocation: Olaylar, tepkiler, abone-yayım

Layered: soyut makineler, kısıtlı görünürlük. Her katman bir soyut makine gibi davranır. Üstündeki katmanlara hizmet eder (Ör. X11)

Daha Fazla Stil...

İlk stil: Master control: hiyerarşik olarak alttaki yordamları çağır ve geri dön

Message bus: asenkron mesaj ortak bir veriyolundan geçer

Mobile code: Özellikle mobil cihazlar için, istek üzerine kod, uzaktan değerlendirme, mobil ajan

Object oriented: Nesneye yönelik programlamadan biraz farklı. asenkron mesaj iletimi, bağımsız kontrol iş parçacığı. Yine nesnelerimiz var ama hepsinin varlığı bağımsız, kontrolleri kendilerinde. Birbirlerine asenkron mesaj yollayarak, işbirliği içinde problemi çözer.

Peer-to-peer: Eşit ortaklar servisin sorumluluğu paylaşır.

Plug-in: Kayıtlı üçüncü parti eklentiler (ör. Eclipse) Sisteme eklentiler yaparak büyütme

Daha fazla Stil...

Pipe-and-filter: Tek yönlü, sıralı, girdi-çıkı, ASCII katarı

Process control: geri besleme döngüsü. Donanımı kontrol eden yazılım. (Ör. Araba hız sabitleme sistemi)

Production systems: (yapay zekadan) kural tabanı, koşullu çalışma. Sistemi gerçekleştirmek üzere kontrol akışı hakkında tam fikrimiz yoksa kullanabiliriz.

Representational state transfer (REST): ör: HTTP uygulamaları. istemci-sunucu ilişkisinde, katmanlı, durumsuz (stateless, her istek bağımsız ele alınır), ön belleğe alınabilir dağıtık hiperortamlar (performans için)

Daha fazla Stil...

Service-oriented (SOA): ;Sistemin işlevleri ayrıık servislerde. Sistem bir servis kümesi. gevşek ilişkili, durumsuz, keşfedilebilir, kontrat-tanımlı

Shared nothing (Sharded): düğümler arası paylaşım olmayan dağıtık veritabanı.

State-transition systems: Reaktif, Asenkron olaylara tepki veren sistemler. Ör. GUI ve kullanıcının yarattığı olaylar. Gerçek-zamanlı da olabilir.

Shared memory: Senkronizasyon için kilit mekanizması. Tüm işlevler ortak bir bilgi havuzunu kullanır.

Table-driven interpreter: Çözümle ve Dağıt. İstekler belirli bir dil ile kodlanır ve bir yorumlayıcı ile bunlar çözümlenir ve ilgili yordama gönderilir.

Stil Sorunları

Sanki bütün problemleri stillerle çözmüşüz gibi konuşsak da, hala pek çok sorun mevcuttur.

Gerçek ve büyük sistemler birden fazla stile ihtiyaç duyar. Bunlara heterojen sistem adını veririz.

- İstemci/sunucu birimlerimiz, reaktif bir GUI, vb çeşitli yaklaşımların kombinasyonu

Bazı durumlar, mimari bir stile uysa da, çalışma alanına çok bağlıdır. Belirli bir askeri uçağı düşünün. Çeşitli versiyonları olacaktır. Ancak kontrol mekanizması çok benzer olacaktır. Farktan çok benzerlik içerir.

- Domain-specific software architecture (DSSA): Referans mimari de denir. Ortak işlevler burada tanımlanır, özelleşenler kendi mimarileri ile tanımlanır.

Anlam (Semantics). Mimari alanı ilerledikçe, stillerin tanımlarının daha kesin olmaları gerekir.

Architecture Description Language (ADL)

Mimariyi tanımlamak için notasyon

Sadece diyagrama sahip olmaktan az da olsa fazla resmiyet ve kesinlik sağlar

Araç desteği sağlar.

- Diyagram çizim aracı, stile uyup uymadığınızı kontrol eder.
- Çözümleyiciler (analyzers) sisteminizin yapısal ve davranışsal özelliklerini saptar.
- Simülatörler

Popüler Mimari Tanım dilleri:

- **Acme**, Wright, Rapide, ArTek, Derneter, CODE, Modechart, PSDL/CAPS, Resolve, UniCon

Mimari Değerlendirme

Acaba doğru mimariyi uygulayabildik mi?

Ürettiğimiz mimariyi doğruluk, bütünlük, tutarlılık ve diğer kalite unsurları açısından yargılayacağımız bir süreçte ihtiyaç duyarız.

Eğer hata yaparsak, bir değişiklik yapmak maliyetli olacaktır.

Mimari gözden geçirme tahtası (Architectural review board): Büyük ve karmaşık sistemlerde, çoklu takımların oluşturduğu sistemlerde, hatta sistemlerin sistemlerinde bir yerde değişiklik yapmak, hiç beklenmedik bir kısımda beklenmedik bir etki yaratabilir. En iyisi, geliştiricilerin bir araya gelip bu tarz etkileri gözden geçirmesidir. Bunu periyodik olarak yapan kurumlar da vardır.

Software Architecture Assessment Method (SAAM): Biraz daha gayriresmi

Architecture Tradeoff Analysis Method (ATAM): Daha resmi. Carnegie Mellon Software Engineering Institute'ta üretildi.

Software Architecture Assessment Method (SAAM)

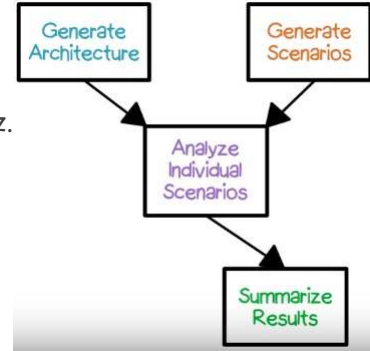
Mimari tasarım sürecinden geçip diyagram gibi çıktılar üretmiş olmalıyız. Mimariyi yarattık.

Senaryolar yaratırız. (Kullanım senaryosu değil, sistemin içinden bakarak detaylı senaryolar)

Kullanım senaryosu bir hesabın yapılması ise, burada mimarinin hangi elemanları beklenen işlevselliği sağlamak için hangi sırada gerekli onları ortaya koyarız.

Diyagramda adım adım yürüyerek, belirli kullanım mimariyi nasıl etkiler ortaya çıkarırız. Özellikle yeni eklenen işlevlerin mimariyi nasıl etkilediği için yapılır.

- Tasarım alternatifleri de değerlendirilerek en az etkiyi yaratan seçilir.



Özet

Yazılım mimarisine ve Mimari stillere bir giriş yaptık.

Nihai hedefimiz, yüksek kaliteli sistemler üretmek ve maliyetleri azaltmak.

Bunu yapmanın en iyi yolu, problemlerin erkenden belirlenmesidir.

- Bunun için de sistem neye benzeyecek en baştan belirlemek gerekir. Sorunlar ve mantıklı çözümleri, ortaya koymak gerekir.

Elimizdeki hazır çözümleri/araçları çözümde kullanabilmeliyiz.

Tüm bunlarla, tüm paydaşlara hızlı ve etkin şekilde fikirlerimizi aktarabileceğimiz kadar soyut bir mimari inşa etmek isteriz.

Mimari Görünümler

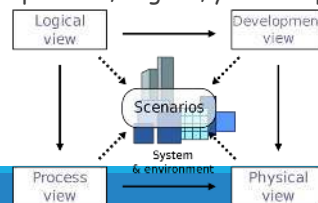
Mimari görünüler

Bina mimarları eskiz ve taslaklar ile önerilen bir binanın mimarisini ifade ederler.

Yazılım mimarları da görünüm (view) adı verilen gösterim araçlarını kullanır.

Mimari bir diyagram DEĞİLDİR. Sistemin çözeceği problemi nasıl çözeceği için önemli tasarım kararları kümesidir.

- Gereksinimlerde yer alan problem için belirli bir çözümü, işlevsel olmayan gereksinimleri de karşılayacak şekilde hesaplamak için bir araya gelen bileşen kümesini ifade eder.
- Grafik ve metin türünde mimari görünümü nasıl ifade edebildiğimize bakacağız.
- Kruchten'in makalesini baz alacağız. Sonrasında farklı görünümüleri de (özellik, işlevsel olmayan gereksinimler, hata raporlama, bağlam, yardımcı program) görerek destekleyeceğiz.



Mantıksal Görünüm (Logical View)

İşlemsel, iletişimsel ve davranışsal sorumlulukların yapısal parçalanması

Diyagramlar:

- Kutu ve ok (Box and Arrow)
- UML sınıf modeli
- UML genel etkileşim
- UML İşbirliği
- Bileşenler ve Bağlayıcılar (ADL)

Mantıksal Görünüm: Kutu ve Ok

Kutular sistemin işlevsel bileşenleri

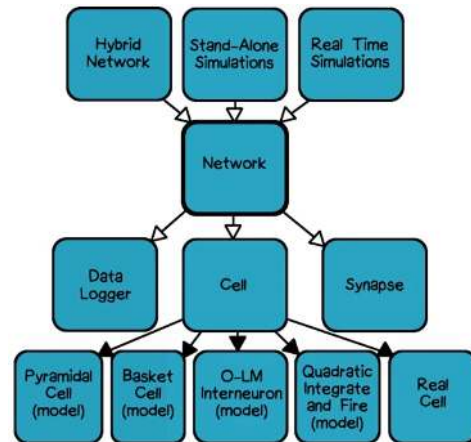
Oklar, aralarındaki bir çeşit bağımlılık ilişkisi

Tek diyagramda yüksek seviyeli mimari yapı

Anlaşılması kolay

Kesin olmayabilir.

- Oklar ne anlama geliyor?



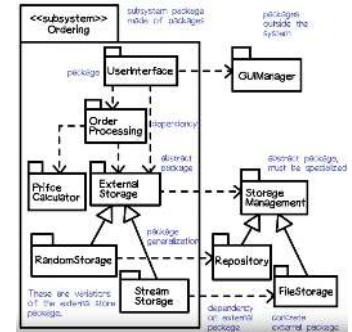
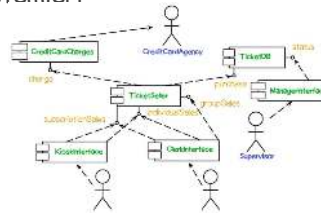
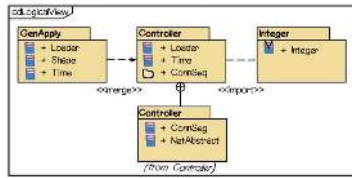
Gelişimsel Görünüm (Developmental View)

Burada kaynak kodu ile ilgiliyiz.

İlgilendiğimiz kaynak kodu birimleri: Paketler, sınıflar, altsistemler, kütüphaneler, dosyalar

Diyagramlar:

- UML Paket diyagramı
- UML Bileşen diyagramı
- CVS, SVN, Github vb kod kontrol sistemleri



Diyagramlardaki Bağlantılar

Bileşen Diyagramı: Çağrı, Kullanabilme

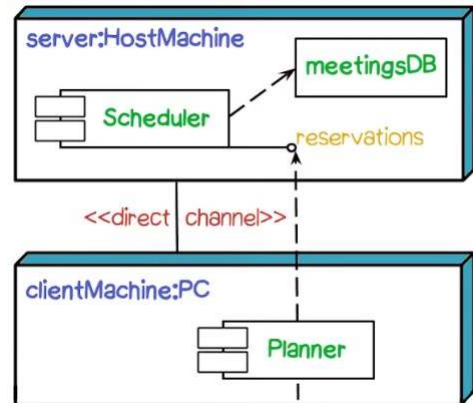
Paket Diyagramı: içeri aktarma (import), alt bileşen, bir araya gelme (merge)

Sınıf diyagramı: Genelleştirme, Bağımlılık

İşlem Görünümü (Process View)

Çalışmanın ayrıldığı eşzamanlı işlemler ve iş parçaları (processes & threads)

UML Barındırma (Deployment) Diyagramı



Fiziksel Görünüm (Physical View)

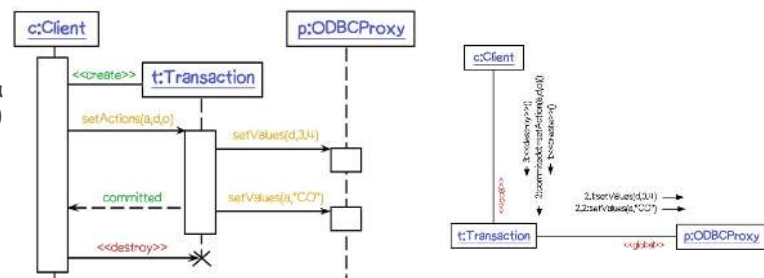
Bir öncekine çok yakın

Sistemin çalışması için gerekli işletme birimleri (makinelar) ve,

İşlemlerin çeşitli işletme birimlerine nasıl dağıldığı/yerleştirildiği

Diyagramlar:

- UML Barındırma Diyagramı
- UML Ardışıl (Sequence) veya UML İşbirliği (Collaboration) Diyagramı

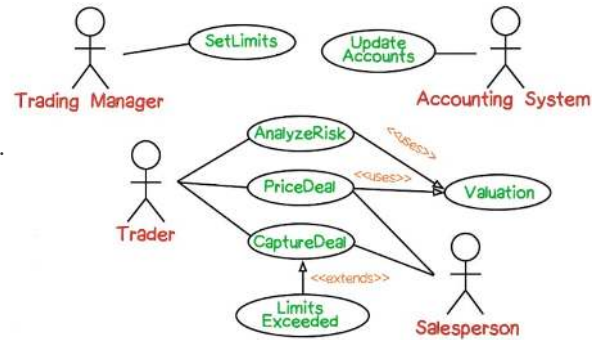


Kullanım Senaryosu Görünümü (Use Case View)

Harici aktörlerin gözünden/bakış açısından önemli işletim dizgeleri

Diyagramlar:

- UML Kullanım Senaryosu diyagramı
- UML İletişim (Communication) diyagramı
- UML Ardışıl diyagram
- UML Aktivite Diyagramı
- Bağlam Diyagramı (Taslak)
- Kullanım senaryoları: Metin ya da tablo olabilir.



Bağlam Görünümü (Context View)

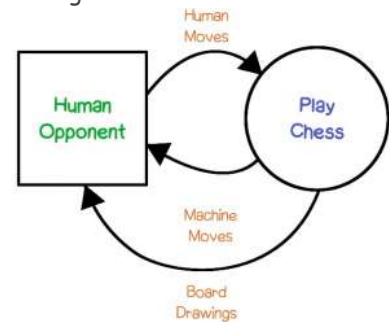
OMT'de yapısal ve davranışsal ve işlevsel görünüm vardı.

İşlevsel görünüm: Veri Akış Diyagramı (DFD)

- UML'de yok. Yerine Use-case diyagramı kullanılır.
- Sistemi tanımlarken oldukça faydalıdır. Aktiviteler ve oluşum sıraları gösterilebilir.

En üst seviyeli DFD bağlam diyagramı olarak da bilinir.

- Sistemin bulunduğu çevre ile olan etkileşimini vurgular.
- Harici aktörler/sistemler, veri akışları ile sisteme bağlıdır.
- Detaylar alt seviyeli (içiçe) diyagramlar ile gösterilir.



Özellik Görünümü (Feature View)

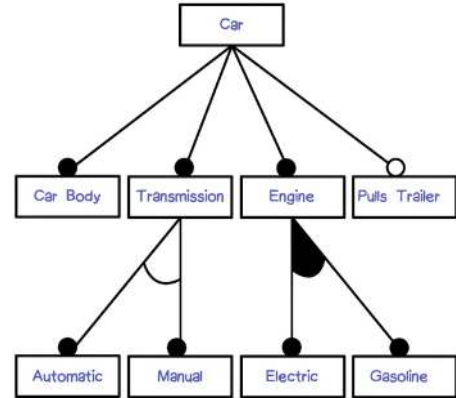
4+1 görünüm bitti ama sistemi mimari seviyede modellerken faydalı olabilecek başka görünüm de mevcuttur.

Özellik: Kullanıcının bakış açısından, kavramsal bir sistem davranışı

- Kamera: Zum özelliği (zum özelliği yok da olabilir.)

Özellik modellemesi: Birbiriyle ilişkili uygulamalar koleksiyonunun sağladığı bir özellik kümesini tanımlamak için kullanılır.

- Özellik diyagramı (Bir ürün kümesinde belirli bir ürüne uygulanabilecek özellikler)
 - Gerekli ve opsiyonel özellikler
 - Sadece biri seçilebilir / kombine edilebilir özellikler
- Özellikler arası kısıtlar
 - (diyagramda yok ama) Römork çekme sadece otomatik vites ile alınabilir)



Özellik Görünümü (Feature View)

Kaç çeşit araba oluşturulabilir?

Olasılıkları çarpmamız gerekir.

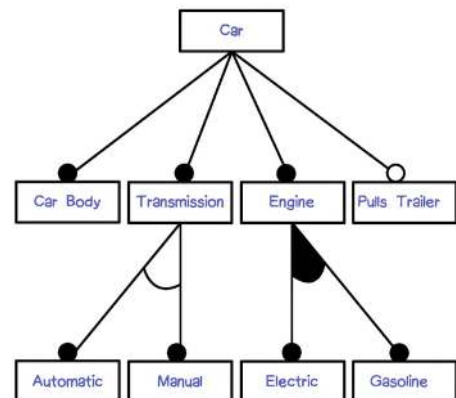
Gövde lazım ve alt özelliği yok.

Vites lazım ve 2 seçeneği var.

Motor lazım ve 3 seçeneği var (elektrik, benzin, elektrik+benzin)

İstersek römork çekebiliriz

$1 \times 2 \times 3 \times 2 = 12$ çeşit araba



İşlevsel Olmayan Görünüm (Non Functional View)

Mimaride unutmamamız gereken husus: Sistem hesaplaması gerekeni, işlevsel olmayan gereksinimleri de karşılayacak şekilde hesaplamalıdır.

- Zordur ve birşeylerden ödün vermeyi gerektirir.
- Tablo yapısı kullanılabilir.

Bir web tarayıcısının esas sorumluluğu nedir?

- Web sayfalarını görüntülemek
- Ancak kaynak kodlarına bakarsak, «performans kısıtları için» pek çok önbellek yönetim mekanizması implementasyonu görürüz.
 - Sayfa önbelleği, bağlantı önbelleği, görüntü önbelleği, ...
- Performans → Önbellek
- Güvenlik → SSL desteği
- Genişletilebilirlik → plug-in desteği
- Taşınabilirlik → cross-platform

Hata Raporlama Görünümü (Bug Reporting View)

Hataların (geliştirme ve kullanımda ortaya çıkan) yer aldığı birimler

Hata raporlama aracı bileşenleri

Hangi özelliği kullanırken ortaya çıktı, geliştirmede kimin ilgilendiği de takip edilebilir.

Before reporting a bug, please read the [bug writing guidelines](#), please look at the list of most frequently reported bugs, and please search for the bug.

Show Advanced Fields

Product: CZ

Reporter: [Redacted]

Component: [Dropdown menu]

Component Description: Select a component to read its description.

Version: [Dropdown menu]

Severity: [Dropdown menu]

Hardware: [Dropdown menu]

OS: [Dropdown menu]

We've made a guess at your operating system and platform. Please check them and make any corrections if necessary.

Summary: [Text input field]

Description: [Text input field]

Attachment: [Add an attachment] [Cancel]

Yardımcı Program Görünümü (Utility View)

Aslında tam bir görünüm değil ama diğer görünüm kadar yeterince üzerinde durulmayan sistem yapı özellikleri ile ilgili bilgiler içerir.

Sistemin bir parçası olabilecek ancak çalışan uygulamada yer almayan destek yazılımları.

- Kurulum betikleri (Installation scripts)
- Kayıt dosyası analizi (Log file analysis)
- İstatistiksel işleme
- Make dosyaları ve diğer kurulum betikleri
- Konfigürasyon dosyaları
- Belegelendirme (herhangi bir)
- Proje manifestoları (Geniş dağıtılabılır paketteki yapı tanımları)
- Destek araçları

Özet

Mimaride somut ve tekil bir şey yoktur.

Mimari, bir karar kümesidir.

- Bu kararlar hakkındaki bilgiyi çeşitli (grafik/metin) görünüm üzerinden verebiliriz.
- Bunların toplamı mimariyi ifade eder.

Geliştirici olarak, sistemin yapısına, karmaşıklığına, uygulama alanına ve ilgililerine uygun görünümünü seçmemiz gerekir.

